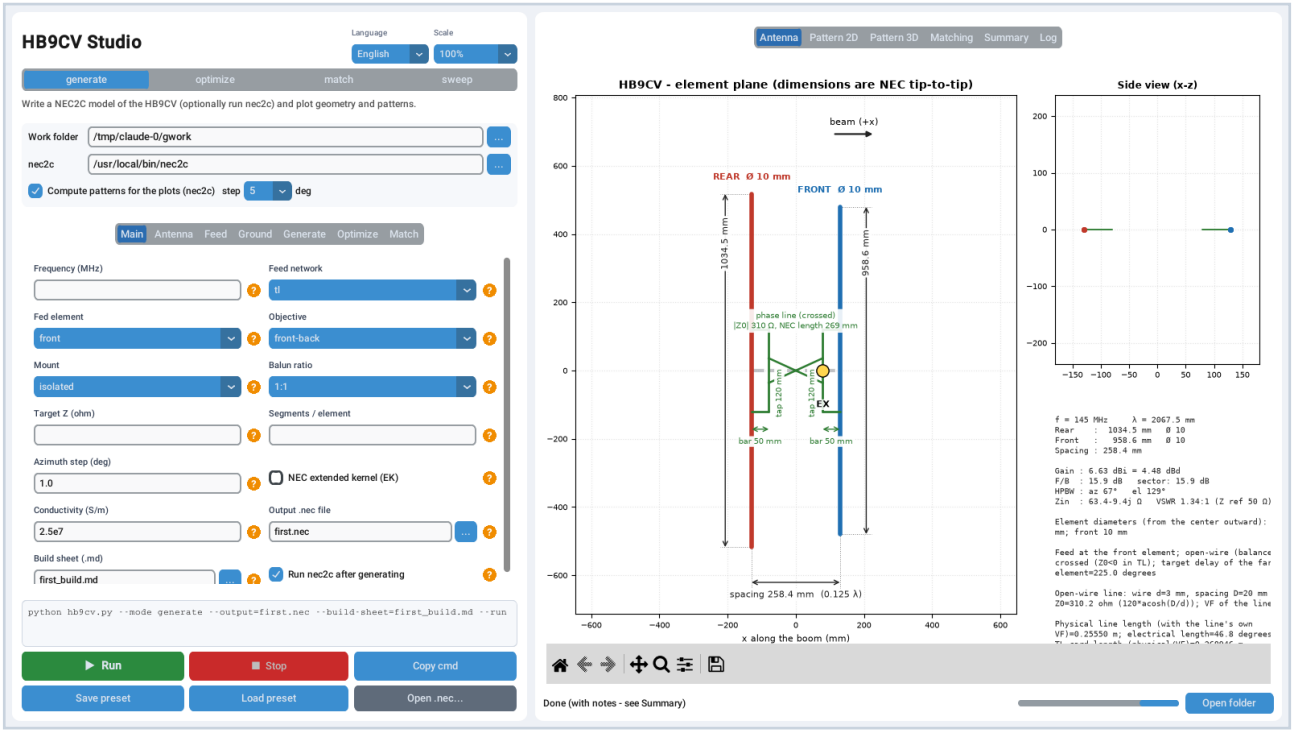
# HB9CV Studio

## User Manual

Design, simulate, optimize and match a two-element HB9CV beam with hb9cv_gui.py, hb9cv.py and the nec2c antenna simulator

**Emiliano Gonzalez - LU3VEA**




A step-by-step guide for first-time users. No previous antenna-modelling experience is needed: every window, button and result is shown and explained.

Edition: October 2026

**HB9CV Studio**

Author: Emiliano Gonzalez - LU3VEA

Copyright © 2026 Emiliano Gonzalez - LU3VEA

This manual and the software it describes are free software: you can redistribute them and/or modify them under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

They are distributed in the hope that they will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License (file LICENSE of the project, or https://www.gnu.org/licenses/) for more details.

SPDX-License-Identifier: GPL-3.0-or-later

# Contents

**1. Welcome . . . . . 6**

1.1 What is an HB9CV antenna? . . . . . 6

1.2 What does the program do? . . . . . 7

1.3 How this manual is organised . . . . . 7

**2. Key ideas in plain words . . . . . 8**

2.1 Frequency and wavelength . . . . . 8

2.2 Elements, boom and spacing . . . . . 8

2.3 Phasing line, feed and T-match . . . . . 8

2.4 Gain: dBi and dBd . . . . . 8

2.5 Front-to-back ratio (F/B) . . . . . 9

2.6 Radiation pattern, beamwidth and take-off angle . . . . . 9

2.7 Input impedance (Zin), R and X . . . . . 9

2.8 VSWR and matching . . . . . 9

2.9 Balun and common-mode current . . . . . 9

2.10 Simulation versus reality . . . . . 10

**3. Installation . . . . . 11**

3.1 What is installed . . . . . 11

3.2 Getting the files . . . . . 11

3.3 Linux — with the installer (recommended) . . . . . 11

3.4 Linux — manual installation . . . . . 12

3.5 Windows — with the installer (recommended) . . . . . 12

3.6 Windows — manual installation . . . . . 13

3.7 Starting the program . . . . . 14

3.8 Updating and uninstalling . . . . . 14

**4. A tour of the main window . . . . . 15**

4.1 Empty boxes mean 'use the default' . . . . . 16

4.2 Greyed-out options . . . . . 16

4.3 Getting help inside the program . . . . . 16

4.4 Languages . . . . . 17

**5. The option tabs . . . . . 18**

5.1 Main tab — the basics . . . . . 18

5.2 Antenna tab — the geometry . . . . . 19

5.3 Feed tab — the phasing line and feed . . . . . 19

5.4 Ground tab — height, mast and checks . . . . . 20

5.5 Generate tab — frequency sweep . . . . . 21

5.6 Optimize tab — the search . . . . . 21

5.7 Match tab — the matching network . . . 22
**6. Tutorial: your first antenna . . . 23**
6.1 Set up the run . . . 23
6.2 Run it . . . 24
**7. Reading the results . . . 25**
7.1 Antenna tab — the drawing and the key numbers . . . 25
7.2 Pattern 2D tab — the radiation pattern . . . 26
7.3 Pattern 3D tab — the shape in space . . . 27
7.4 Summary tab — the report . . . 28
7.5 Matching and Log tabs . . . 28
7.6 Opening an existing model . . . 28
**8. Adding reflectors and directors . . . 29**
**9. The antenna over real ground . . . 32**
**10. Frequency sweep and bandwidth . . . 35**
**11. The optimizer: let the program find the best design . . . 36**
11.1 A ready-made example . . . 36
11.2 Running . . . 37
11.3 The result . . . 38
11.4 Important messages of the optimizer . . . 40
11.5 Monte-Carlo tolerance analysis . . . 41
**12. Matching network: match and sweep modes . . . 42**
12.1 Match mode . . . 42
12.2 Sweep mode . . . 44
**13. Build sheet and tuning . . . 47**
13.1 Tuning the real antenna . . . 48
**14. Quality checks . . . 49**
**15. Using the command line . . . 50**
15.1 Getting help . . . 50
15.2 Generate and simulate . . . 50
15.3 Frequency sweep . . . 51
15.4 Match and sweep . . . 52
15.5 Optimize . . . 53
15.6 Robustness and convergence . . . 53
**16. Troubleshooting and frequent questions . . . 54**
16.1 Problems . . . 54
16.2 Common NOTE and WARNING lines . . . 55
16.3 Frequent questions . . . 55

**Appendix A — Complete option reference . . . . . 56**

Main tab . . . . . 56

Antenna tab . . . . . 58

Feed tab . . . . . 60

Ground tab . . . . . 63

Generate tab . . . . . 64

Optimize tab . . . . . 64

Match tab . . . . . 68

**Appendix B — Files created by the program . . . . . 70**

# 1. Welcome

This manual teaches you how to use **HB9CV Studio**, a program that designs a small directional antenna called the **HB9CV**, simulates how it will behave, improves it automatically and tells you how to build it. It is written for people who have never used an antenna simulator. Every chapter shows the real screens of the program and explains, in plain words, what each control does and how to read each result.

## 1.1 What is an HB9CV antenna?

An HB9CV is a beam antenna with only **two elements**: two straight metal tubes or rods mounted parallel to each other on a support called the **boom**. Unlike a simple dipole, which sends the same signal to the front and to the back, the HB9CV concentrates the energy in **one direction**. It does this by feeding *both* elements with the radio signal, but with a small time delay between them. The delay is created by a short piece of transmission line called the **phasing line**, which crosses over between the two elements.

The result is a compact, light antenna with useful gain (about 4 to 5 dB more than a dipole in the forward direction) and a good rejection of signals arriving from behind. It is very popular on the 2 m (144–148 MHz) and 70 cm bands, for portable operation, fox hunting (radio direction finding) and as a small fixed beam.

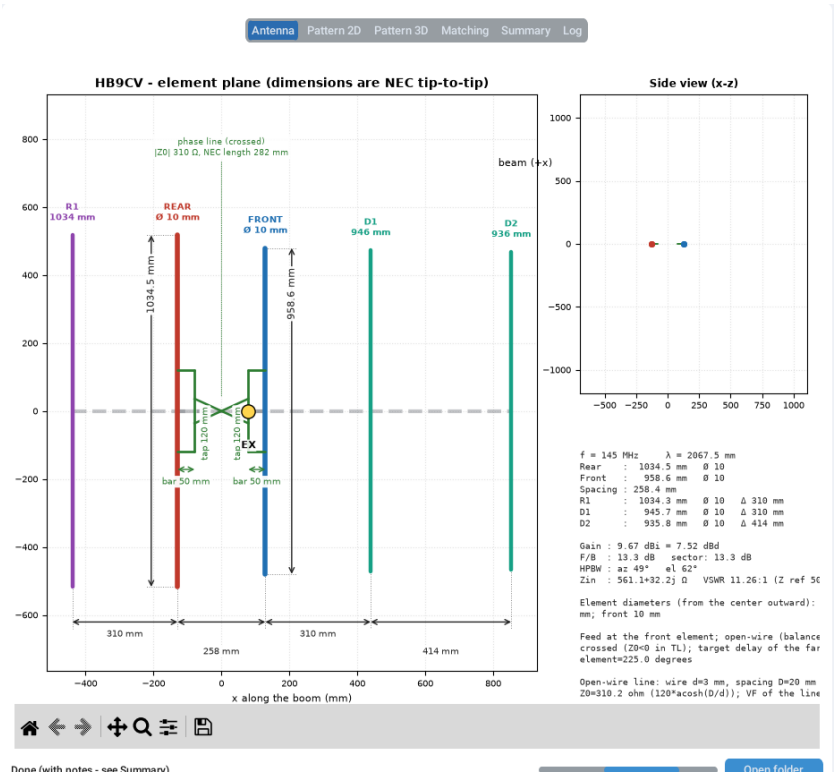



*Figure 1-1. How the program draws an antenna: the red tube is the rear element, the blue tube the front element, the green lines the crossed phasing line and the T-match feed. (This example also has an extra reflector R1 and two directors D1, D2.)*

## 1.2 What does the program do?

HB9CV Studio is made of two files that work together:

- `hb9cv.py` — the **engine**. It calculates the antenna dimensions, writes a model file for the simulator, runs the simulator and analyses the results. It can be used alone from a terminal (chapter 15).
- `hb9cv_gui.py` — the **window** (graphical user interface, GUI). You fill in boxes and press **Run**; the window builds the command for the engine, runs it, and shows drawings and graphs of the result. This manual focuses on the window.

The calculations are done by **nec2c**, a free implementation of NEC-2, the classic antenna simulator developed for the US Navy. NEC divides every metal wire into short pieces called **segments**, computes the electric current in each one, and from those currents obtains the radiation pattern and the impedance seen by the radio. You never have to write a NEC file yourself: the program does it for you.

The window has four **modes**, one for each kind of job:

| Mode | What it does | When to use it | Chapter |
|---|---|---|---|
| generate | Builds one antenna with the dimensions you give and simulates it. | To see how a given design performs, to draw it, and to get a build sheet. | 6–10 |
| optimize | Tries many combinations of dimensions and keeps the best one. | To find the best design for your frequency, band and goals. | 11 |
| match | Designs a small coil-and-capacitor circuit (an L-network) so the antenna presents 50 Ω to the radio. | When the antenna impedance is not close to 50 Ω. | 12 |
| sweep | Shows how that matching circuit behaves over a range of frequencies. | To check the bandwidth and the effect of real component tolerances. | 12 |

## 1.3 How this manual is organised

- Chapter 2 explains the few technical words you will meet. Read it once even if you skip the rest.
- Chapter 3 installs everything. Chapter 4 is a guided tour of the window. Chapter 5 describes the option tabs.
- Chapters 6 and 7 are a hands-on tutorial: you build your first antenna and learn to read every result.
- Chapters 8 to 14 cover each remaining function, one at a time, with screenshots.
- Chapter 15 shows the same work from the command line; chapter 16 answers common problems.
- Appendix A lists **every option** of the program with its full explanation; Appendix B lists the files the program creates.

> **TIP** Every option in the window has an orange **?** badge next to it. Click it to read a full explanation inside the program. The same text is printed in Appendix A.

# 2. Key ideas in plain words

You do not need to be an engineer to use the program, but a few words appear everywhere. This chapter explains them simply. Come back here whenever a result is unclear.

## 2.1 Frequency and wavelength

The **frequency** (in MHz, megahertz) is the channel your radio uses, for example 145 MHz. Every frequency has a **wavelength**, written λ (lambda): the physical length of one cycle of the radio wave. It is about 300 / f metres, so at 145 MHz λ ≈ 2.07 m. Antenna sizes are naturally expressed as fractions of λ. An HB9CV element is about half a wavelength long (roughly 1 m at 145 MHz) and the elements are typically 0.125 λ apart (about 26 cm).

## 2.2 Elements, boom and spacing

- The **rear element** (drawn in red) is the longer one, at the back.
- The **front element** (drawn in blue) is slightly shorter and points toward the direction of maximum signal, called the **beam** direction (the +x axis in the drawings).
- The **spacing** is the distance between the two elements, centre to centre.
- The **boom** is the support that holds them. If the boom is metal, the way the elements are fixed to it matters (see *Mount* in chapter 5).
- Optional extra elements: **reflectors** (behind the rear element) and **directors** (in front of the front element) turn the HB9CV into a longer beam with more gain (chapter 8).

## 2.3 Phasing line, feed and T-match

Both elements receive power. The radio cable is connected to one of them (the **fed element**, usually the front one) and a short crossed **phasing line** carries part of the signal to the other element, delayed. That delay is what makes the antenna directional. The point where the cable connects is the **feed point**. In the classic mechanical design each element is a continuous tube, and the line connects to it through a **T-match**: two short bars (taps) that touch the element at some distance from its centre. Moving the taps changes the impedance.

## 2.4 Gain: dBi and dBd

**Gain** tells how much stronger the signal is in the best direction compared with a reference antenna. It is given in decibels (dB), a logarithmic scale: +3 dB is double the power, +10 dB is ten times. **dBi** compares with an imaginary antenna that radiates equally in all directions (isotropic). **dBd** compares with a half-wave dipole. The two differ by 2.15 dB: dBd = dBi − 2.15. A typical HB9CV in free space gives about 6.3–6.6 dBi (4.2–4.5 dBd).

## 2.5 Front-to-back ratio (F/B)

The **F/B ratio** is the difference, in dB, between the signal sent forward and the signal sent backward. A high F/B (15–25 dB or more) means the antenna ignores stations or noise behind it — very useful for direction finding. The program reports three versions, because the back of an antenna is rarely a single clean point:

| Name | Meaning |
|---|---|
| F/B (point) | Forward gain minus the gain exactly at 180° (straight behind). |
| F/B (sector) | Forward gain minus the *worst* gain anywhere in the rear sector, normally 150° to 210°. More honest than the point value; this is what the optimizer uses by default. |
| F/B (cone) | Like the sector, but also looks up and down at every elevation inside the rear cone. |

## 2.6 Radiation pattern, beamwidth and take-off angle

The **radiation pattern** is a map of the gain in every direction. It is shown as polar plots (a 'top view' and a 'side view' cut) and as a 3-D shape. The **half-power beamwidth (HPBW)** is the angular width of the main lobe between the points where the power has fallen to half (−3 dB). A narrow beam means more directivity. The **take-off angle** is the elevation angle of the strongest radiation; over real ground it is a few degrees above the horizon.

## 2.7 Input impedance (Zin), R and X

The radio 'sees' the antenna as an electrical load called the **input impedance**, Zin, written as R + jX ohms (Ω). **R** (resistance) is the part that takes power and radiates it. **X** (reactance) is a part that stores energy and sends it back; ideally X = 0. For example 63.4 − 9.4j Ω means R = 63.4 Ω and X = −9.4 Ω. Radios and coaxial cables are designed for **50 Ω**.

## 2.8 VSWR and matching

The **VSWR** (voltage standing wave ratio) measures how well the antenna impedance fits 50 Ω. 1.0:1 is perfect; up to 1.5:1 is excellent; up to 2:1 is acceptable for most radios. A high VSWR means part of the power is reflected back to the radio. If the antenna impedance is far from 50 Ω you can add a **matching network**: a small coil (inductor, nH) and capacitor (pF) that transform the impedance. The *match* mode designs it (chapter 12). The **bandwidth** is the range of frequencies where VSWR stays below 2:1.

## 2.9 Balun and common-mode current

The HB9CV is a **balanced** antenna (both sides symmetrical) while coaxial cable is **unbalanced**. Without a **balun** or common-mode choke (for example a few turns of coax or ferrite beads at the feed point) current flows on the outside of the coax shield, the cable radiates, and the F/B and impedance get worse. The program warns you about this in every run; it is a reminder, not an error.

## 2.10 Simulation versus reality

A simulation is an idealised model. NEC assumes perfect straight wires, ideal connections, a perfect balun and an ideal phasing line. Real tubes have clamps, insulators, bends and nearby objects. The numbers are an excellent guide for design and comparison, but the final antenna must be **tuned on the bench** with an antenna analyzer (or VNA). The build sheet (chapter 13) explains how.

# 3. Installation

HB9CV Studio runs on **Linux** (Fedora, Debian, Ubuntu, Linux Mint and their derivatives) and on **Windows 10 and 11**. Each system has an installer that does all the work: you answer a few questions and get a menu entry or Start menu shortcut. The manual steps are also explained, for those who prefer to see every command.

## 3.1 What is installed

| Component | What it is for | Linux | Windows |
|---|---|---|---|
| Python 3.9 or newer, with Tk | Runs the program and draws its window. | Distribution packages (python3, python3-tk / python3-tkinter, python3-venv). | python.org installer (Tk included); the installer can install it with winget. |
| customtkinter, matplotlib, numpy | The window, the plots and the calculations of the GUI. | Python virtual environment inside the program folder. | Same. |
| nec2c | The antenna simulator. | Distribution package nec2c, or compiled from source. | nec2c.exe included in the project folder, installed automatically. |

Nothing is installed system-wide except the distribution packages on Linux: the program itself lives in your user folder and can be removed completely (section 3.8).

## 3.2 Getting the files

Download the project from its GitHub page with **Code → Download ZIP** and extract it, or clone it with Git:

```
git clone https://github.com/<user>/<repo>.git hb9cv-studio
```

The folder contains, among others: *hb9cv.py*, *hb9cv_gui.py*, *install.sh* and *uninstall.sh* (Linux), *install.bat* and *uninstall.bat* (Windows), *requirements.txt*, the *tools*, *assets* and *docs* folders.

## 3.3 Linux — with the installer (recommended)

1. Open a terminal in the project folder. In most file managers: right-click inside the folder → *Open in Terminal*.
2. Run `./install.sh` as your **normal user** (not with sudo). If you get *Permission denied* (files extracted from a ZIP lose their execute permission), run `bash install.sh` instead.
3. When asked, confirm the installation of the system packages and type your password for sudo.
4. Wait until the message **HB9CV Studio is installed** appears.

```
cd ~/Downloads/hb9cv-studio
./install.sh
```

What the installer does:

- Detects the distribution and installs the system packages:
  Fedora: `sudo dnf install python3 python3-pip python3-tkinter nec2c`
  Debian / Ubuntu / Mint: `sudo apt install python3 python3-venv python3-pip python3-tk`

`nec2c`

- If the distribution has no nec2c package, it compiles nec2c from source (it installs gcc and downloads the source code automatically).
- Copies the program to `~/.local/share/hb9cv-studio` and creates a Python environment there with customtkinter, matplotlib and numpy.
- Creates the commands `hb9cv-studio` (window) and `hb9cv` (command line) in `~/.local/bin`.
- Adds **HB9CV Studio**, with its icon, to the applications menu (category Science / Engineering).

| Option | Effect |
|---|---|
| --prefix DIR | Install in another folder. |
| --no-system-deps | Do not use dnf/apt (install Python, Tk and nec2c yourself). |
| --build-nec2c | Always compile nec2c from source. |
| --no-build-nec2c | Never compile nec2c. |
| --no-desktop | Do not create the menu entry. |
| -y | Do not ask questions. |

## 3.4 Linux — manual installation

```
# Fedora
sudo dnf install python3 python3-pip python3-tkinter nec2c
# Debian / Ubuntu / Linux Mint
sudo apt install python3 python3-venv python3-pip python3-tk nec2c

cd hb9cv-studio
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python hb9cv_gui.py         # the window
.venv/bin/python hb9cv.py --help      # the command line
```

If your distribution has no nec2c package, `bash tools/build_nec2c.sh` compiles it into the *bin* folder of the project, where the program finds it automatically (it needs gcc: Fedora `sudo dnf install gcc`, Debian/Ubuntu/Mint `sudo apt install gcc`).

## 3.5 Windows — with the installer (recommended)

1. Extract the ZIP file, for example to *Downloads\hb9cv-studio*.
2. Double-click **install.bat**. If Windows shows *Windows protected your PC*, click *More info* → *Run anyway*.
3. A console window opens and asks some questions; answer with Enter (yes) or N (no). No administrator rights are needed: everything is installed for your user.
4. If Python is missing, the installer offers to install Python 3.12 with **winget** (the Windows package manager, included in Windows 10 and 11).
5. It installs the **nec2c.exe** included in the project folder (with any runtime DLLs next to it) and checks that it runs. Only if the project has no working nec2c.exe does it offer to install **MSYS2** (free development tools, about 500 MB) to compile nec2c from its source code; that takes several minutes the first time.
6. When **HB9CV Studio is installed** appears, press a key to close the window.

The result:

- The program in `%LOCALAPPDATA%\Programs\HB9CV-Studio`, with its own Python environment and `bin\nec2c.exe`.
- A **HB9CV Studio** shortcut on the desktop and a **HB9CV Studio** folder in the Start menu with the program, the three manuals and *Uninstall HB9CV Studio*.
- `hb9cv.cmd` (command line) and `hb9cv-studio.cmd` (window) in the program folder.

Options, when running the PowerShell script directly, for example `powershell -ExecutionPolicy Bypass -File install.ps1 -AddToPath`:

| Option | Effect |
|---|---|
| -InstallDir DIR | Install in another folder. |
| -Nec2c C:\path\nec2c.exe | Install this nec2c.exe instead of the one in the project (its runtime DLLs are copied too). |
| -SkipNec2c | Do not look for or compile nec2c. |
| -AddToPath | Add the program folder to your PATH: hb9cv and hb9cv-studio work in any terminal. |
| -NoShortcuts | Do not create shortcuts. |
| -Yes | Do not ask questions. |

## 3.6 Windows — manual installation

1. Install Python 3 from *python.org/downloads*. In the first screen tick **Add python.exe to PATH**; keep **tcl/tk and IDLE** selected.
2. Nothing to do for nec2c: the **nec2c.exe** in the project folder is found automatically. (To rebuild it from source: install MSYS2 from *msys2.org*, open *MSYS2 MSYS*, go to the project folder and run `pacman -S --needed gcc tar curl git` and `bash tools/build_nec2c.sh bin`.)
3. Open a terminal (PowerShell or Command Prompt) in the project folder and run:

```
py -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\pythonw hb9cv_gui.py      # the window
.venv\Scripts\python hb9cv.py --help    # the command line
```

**TIP** On Windows 11 you can also use WSL (*wsl --install* in an administrator terminal) and follow the Linux instructions inside Ubuntu: the program window appears on the Windows desktop.

## 3.7 Starting the program

| | Linux | Windows |
|---|---|---|
| Window (GUI) | Applications menu → HB9CV Studio, or the command hb9cv-studio | Start menu or desktop → HB9CV Studio |
| Command line | hb9cv --help | hb9cv.cmd in the program folder (or hb9cv with -AddToPath) |
| With the example loaded | hb9cv-studio --load-default-example | hb9cv-studio.cmd --load-default-example |
| From the project folder (manual installation) | .venv/bin/python hb9cv_gui.py | .venv\Scripts\pythonw hb9cv_gui.py |

On Linux, if the terminal answers *hb9cv-studio: command not found* right after installing, log out and in once (most distributions add ~/.local/bin to the PATH at login), or run ~/.local/bin/hb9cv-studio.

The program looks for nec2c in this order: the **nec2c** field of the window (or the --nec2c option), the `HB9CV_NEC2C` environment variable, the system PATH and finally the *bin* folder next to the program, where the installers put it, and on Windows the nec2c.exe in the program folder itself. Usually the field can stay empty.

> **NOTE** Without nec2c the program can still *write* the model file and design matching networks from values you type, but it cannot simulate. Most functions (patterns, optimizer, checks) need nec2c.

## 3.8 Updating and uninstalling

- **Update**: get the new version (git pull, or a new ZIP) and run the installer again. Your settings and work folders are kept.
- **Uninstall on Linux**: `./uninstall.sh` (add `--purge` to also delete the settings file ~/.hb9cv_gui.json). The distribution packages are kept.
- **Uninstall on Windows**: Start menu → HB9CV Studio → *Uninstall HB9CV Studio*, or uninstall.bat. Python and MSYS2 are kept; remove them from *Settings* → *Apps* if you no longer need them.

Your choices of language and scale are stored in the file .hb9cv_gui.json in your home folder. The first time, the language follows the one of the system; it can be changed at any time (chapter 4).

# 4. A tour of the main window

The window is divided in two halves. The **left half** is where you choose what to do and type the settings. The **right half** shows the results. The numbered picture below names every part; the table after it explains each one.

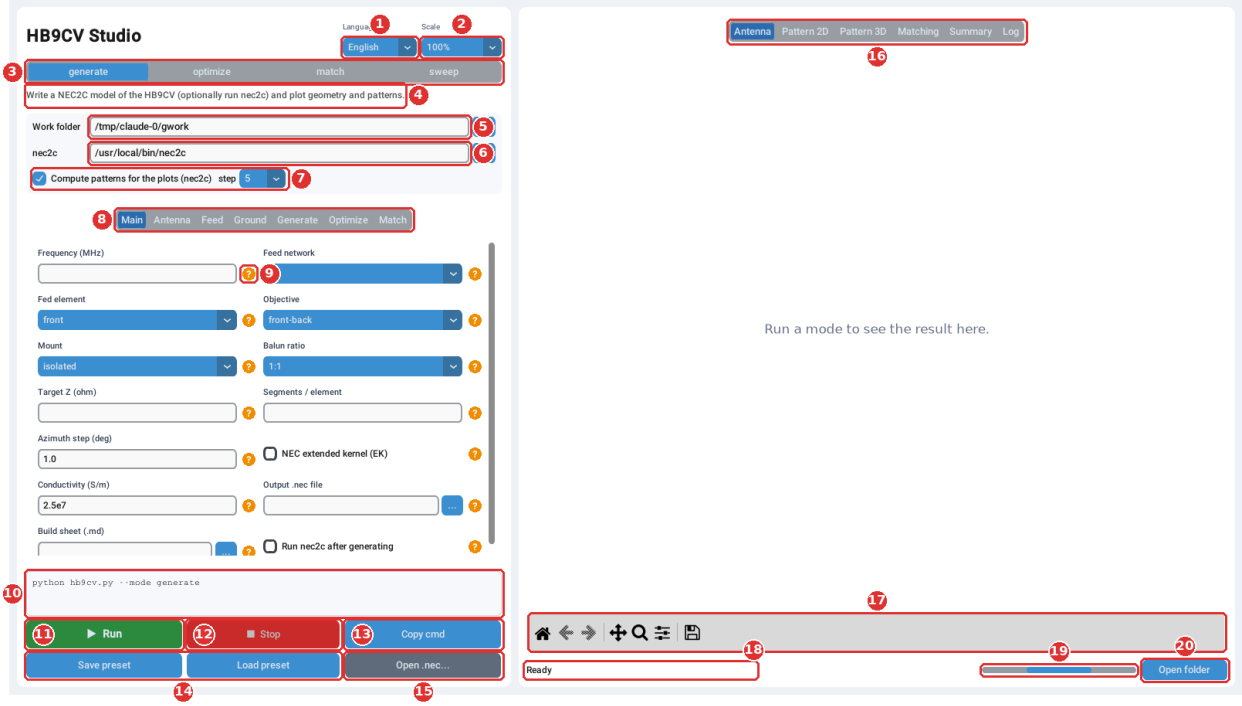



*Figure 4-1. The main window when the program starts. The numbers are explained in the table below.*

| # | Part | What it does |
|---|---|---|
| 1 | Language | Interface language: English, Español or Italiano. The change is applied immediately. |
| 2 | Scale | Size of all text and controls (for small or very large screens). Keyboard: Ctrl + to enlarge, Ctrl − to shrink, Ctrl 0 to return to automatic. |
| 3 | Mode selector | Chooses the job: generate, optimize, match or sweep (see chapter 1). |
| 4 | Mode description | One line that reminds you what the selected mode does. |
| 5 | Work folder | The folder where all files are written (models, results, build sheets). The **...** button opens a folder picker. It must exist. |
| 6 | nec2c | Path of the nec2c simulator. Leave empty if nec2c is in the system PATH. |
| 7 | Compute patterns | When ticked, after the run the window asks nec2c for a full 3-D pattern so the plots can be drawn. **step** is the angular resolution in degrees: 5 is fast, 2 or 1 is finer but slower. |
| 8 | Option tabs | The settings, grouped in tabs: Main, Antenna, Feed, Ground, Generate, Optimize, Match (chapter 5). Scroll inside a tab to see all its boxes. |
| 9 | ? help badge | Click to open a full explanation of that option. |
| 10 | Command preview | The exact command the window will run. It updates as you type. Useful to learn the command-line options. |
| 11 | Run | Starts the job. |
| 12 | Stop | Interrupts a running job (for example a long optimisation). |
| 13 | Copy cmd | Copies the command preview to the clipboard, to paste in a terminal or a note. |

| # | Part | What it does |
|---|---|---|
| 14 | Save / Load preset | Save all current settings to a file and load them later. Ideal for keeping one preset per band or design. |
| 15 | Open .nec… | Opens an existing NEC model file (made by this program or by another one), draws it and simulates its patterns. |
| 16 | Result tabs | Antenna, Pattern 2D, Pattern 3D, Matching, Summary, Log (chapter 7). |
| 17 | Plot toolbar | Home, back/forward, pan, zoom, subplot settings and save picture (PNG, PDF, SVG). |
| 18 | Status | What the program is doing: Ready, … running, Done, Done (with notes), or an error. |
| 19 | Progress | A moving bar while a job runs. |
| 20 | Open folder | Opens the work folder in your file manager to see the files created. |

## 4.1 Empty boxes mean 'use the default'

Most boxes start **empty**. An empty box is **not** an error: it means the program uses its own default value, which is shown in light grey inside the box (for example 145 in *Frequency*) and in the help text. Type a value only when you want something different. Only the options you fill in are added to the command preview.

## 4.2 Greyed-out options

Options that do not apply to the selected mode are shown in grey and cannot be edited. For example, the *Design frequency* of the Match tab is grey in generate mode. Switch the mode and they become active.

## 4.3 Getting help inside the program

Click any orange **?** badge. A yellow panel opens with the option name, its command-line flag, a full explanation, the modes in which it is used and its default value. Click anywhere else to close it.

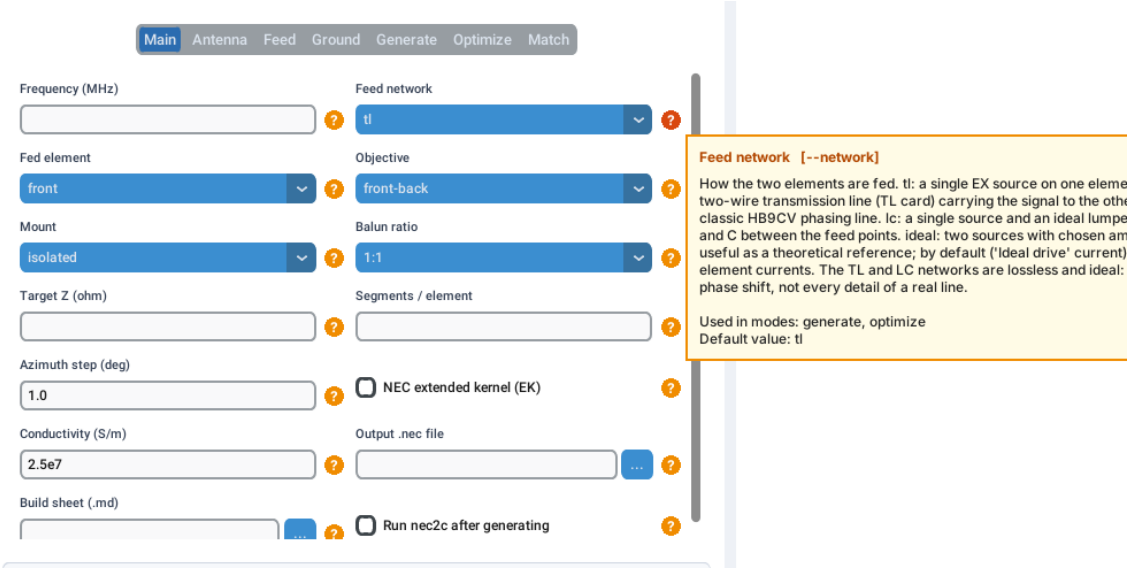



*Figure 4-2. The help panel of the Feed network option. Every option has one.*

## 4.4 Languages

The whole interface, including the help texts, is available in English, Spanish and Italian. The pictures below show the same window in Spanish. Results and logs produced by the engine stay in English.

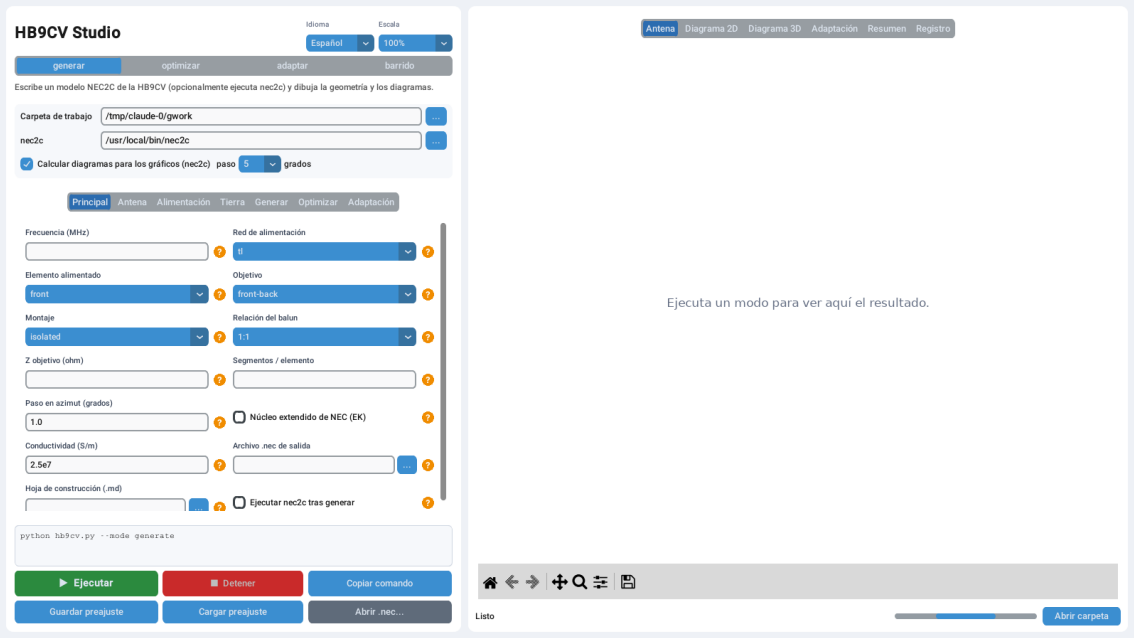



*Figure 4-3. The window in Spanish (Idioma: Español).*

# 5. The option tabs

The settings are grouped in seven tabs. This chapter shows each tab and describes the most important boxes. You do not need to understand all of them to start: the defaults produce a good classic HB9CV. The complete list, with every detail, is in Appendix A.

## 5.1 Main tab — the basics

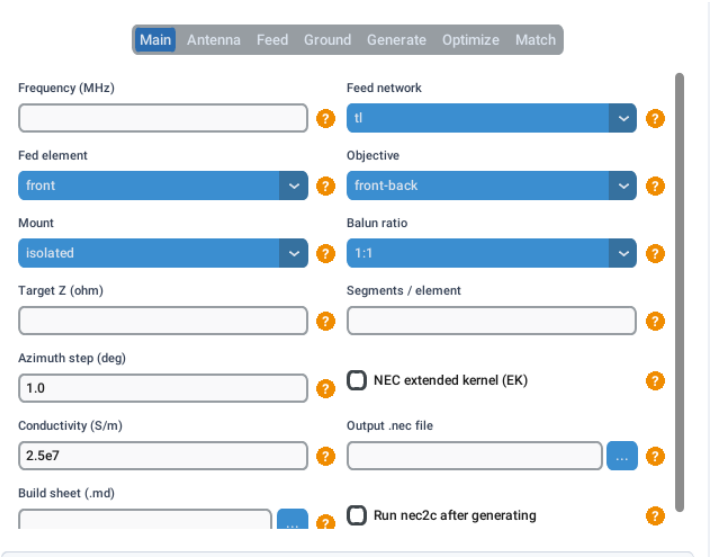



*Figure 5-1. The Main tab.*

| Box | Meaning |
|---|---|
| Frequency (MHz) | Design frequency. Default 145 MHz. All dimensions are scaled to it. |
| Feed network | How the two elements are connected: **tl** = transmission phasing line (classic HB9CV, default); **lc** = a lumped coil/capacitor network instead of a line; **ideal** = theoretical drive with exact currents (a reference, not buildable). A coax phasing line is chosen in the Feed tab. |
| Fed element | Which element the radio cable connects to: front (default) or rear. |
| Objective | What 'best' means for the optimizer: **front-back** (F/B), **gain**, or **balance** (a compromise). |
| Mount | **isolated**: elements insulated from the boom (default). **contact**: the element centres are clamped to a metal boom (classic HB9CV on a metal boom; needs the T-match feed). |
| Balun ratio | Ratio of the balun at the feed point. 1:1 (default) is the usual current balun / choke; 4:1 is an option to evaluate. |
| Target Z (ohm) | Impedance you want the antenna to present (default 50 Ω). |
| Segments / element | How finely NEC divides each element (odd number; default 41 with the T-match). More segments = more precise but slower. |
| Azimuth step (deg) | Resolution of the horizontal pattern used for the numbers. |
| NEC extended kernel (EK) | A different NEC calculation method for thick wires. Leave it **off**: the normal thin-wire kernel is more reliable for this antenna. |
| Conductivity (S/m) | Metal of the elements: 2.5e7 = 6061 aluminium (default), 1.5e7 brass, 5.8e7 copper, 0 = perfect. Used to compute losses. |
| Output .nec file | Name of the model file to write. Empty = automatic name. |
| Build sheet (.md) | Name of the construction document to write (chapter 13). |

| Box | Meaning |
|---|---|
| Run nec2c after generating | In generate mode: simulate immediately. Tick it to see results. |

## 5.2 Antenna tab — the geometry

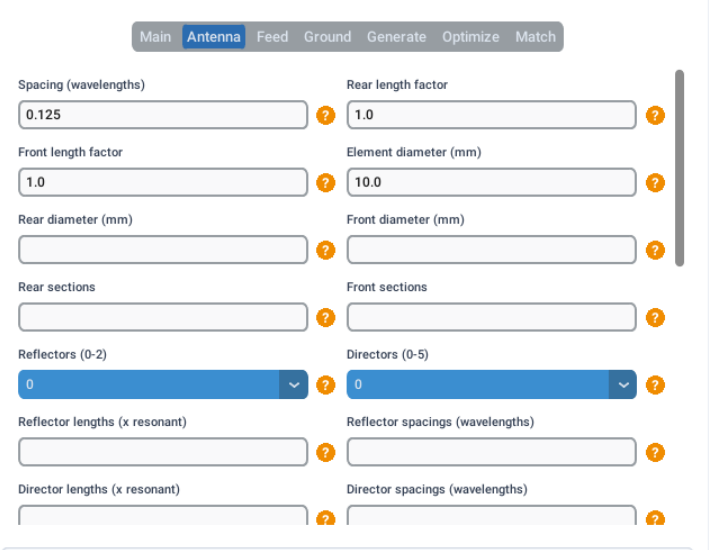



*Figure 5-2. The Antenna tab.*

- **Spacing (wavelengths)**: distance between the elements as a fraction of λ (0.125 = classic).
- **Rear / Front length factor**: multiplies the calculated lengths. 1.0 = calculated length; 1.02 = 2 % longer. Use this to fine-tune.
- **Element diameter (mm)**, or separate **Rear / Front diameter**: tube thickness.
- **Rear / Front sections**: for telescopic (stepped) tubes, e.g. 300:12,200:10,8 (see Appendix A).
- **Reflectors (0–2)** and **Directors (0–5)**: extra parasitic elements, with their lengths and spacings.
- Further down: boom shape, size and length, gap between feed bars, etc.

## 5.3 Feed tab — the phasing line and feed

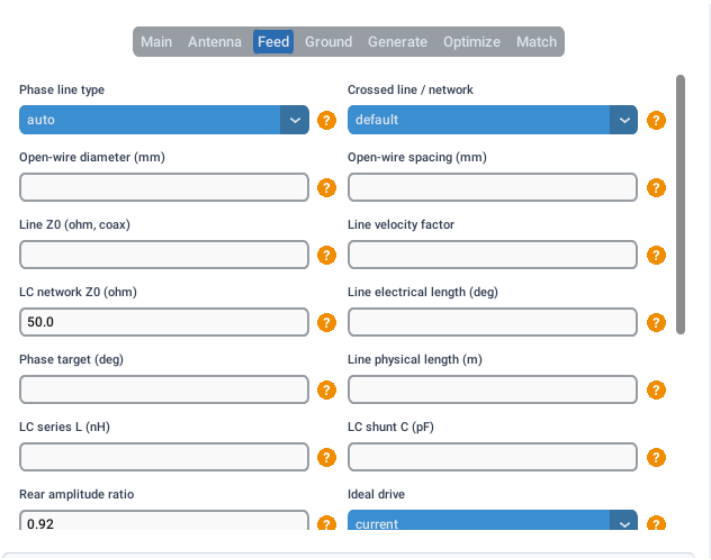



*Figure 5-3. The Feed tab.*

- **Phase line type**: auto, openwire (two parallel wires — the classic HB9CV line) or coax.
- **Crossed line / network**: the line is normally crossed (the two wires swap sides); leave *default*.
- **Open-wire diameter / spacing**: wire thickness and centre distance; together they set the line impedance Z0 = 120·acosh(D/d).
- **Line Z0, Line velocity factor**: characteristic impedance and velocity factor of the line.
- **Line electrical length / physical length**: normally calculated automatically.
- **LC network** boxes: values for the lumped network (only with Feed network = lc).
- **Rear amplitude ratio, Phase target, Ideal drive**: used with Feed network = ideal.
- Further down: T-match tap position, bar offset, series capacitor, feedline and common-mode checks.

## 5.4 Ground tab — height, mast and checks

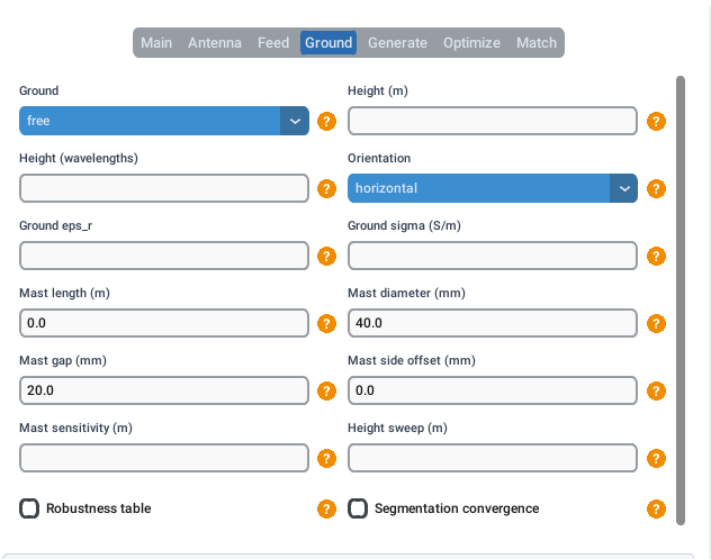



*Figure 5-4. The Ground tab.*

- **Ground**: *free* (free space, no earth — the default, best for comparing designs), *average*, *good*, or *perfect* ground.

- **Height (m)** or **Height (wavelengths)**: how high the antenna is above the ground.
- **Orientation**: horizontal (elements parallel to the ground, horizontal polarisation, usual for SSB) or vertical (vertical polarisation, usual for FM repeaters).
- **Ground eps_r / sigma**: electrical properties of the soil, to override the preset.
- **Mast length, diameter, gap, side offset**: model a metal mast below the antenna.
- **Mast sensitivity, Height sweep**: run a series of simulations varying the mast length or the height.
- **Robustness table** and **Segmentation convergence**: quality checks (chapter 14).

## 5.5 Generate tab — frequency sweep

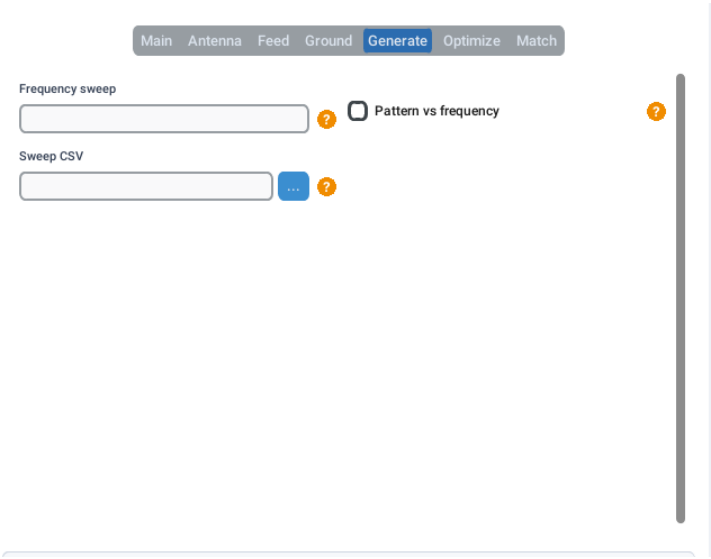



*Figure 5-5. The Generate tab.*

- **Frequency sweep**: FMIN,FMAX,STEP in MHz, e.g. 140,150,0.5. Simulates the impedance at every frequency (chapter 10).
- **Pattern vs frequency**: also computes the pattern at every frequency.
- **Sweep CSV**: optional spreadsheet file with the sweep results.

## 5.6 Optimize tab — the search

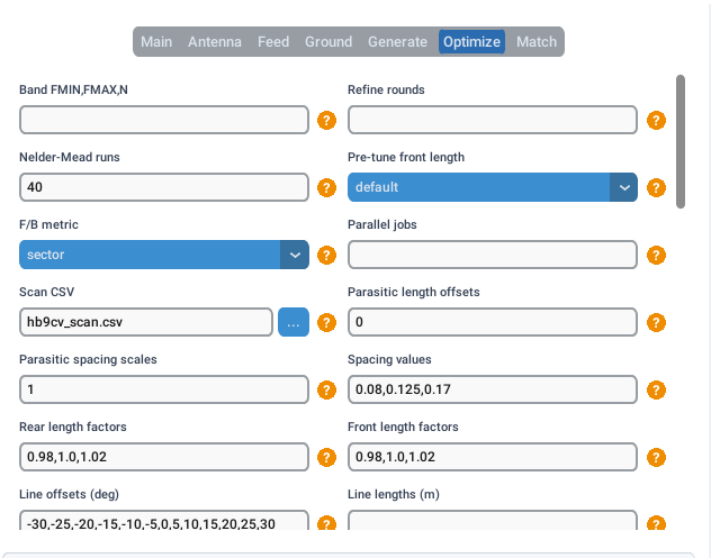



*Figure 5-6. The Optimize tab.*

These boxes define which designs the optimizer tries (chapter 11): the frequency band to evaluate, the lists of spacings and length factors to scan, the line offsets and impedances, the number of refinement rounds, the F/B metric, and the Monte-Carlo tolerance analysis. Lists are written as comma-separated values, e.g. 0.10,0.125,0.15.

## 5.7 Match tab — the matching network

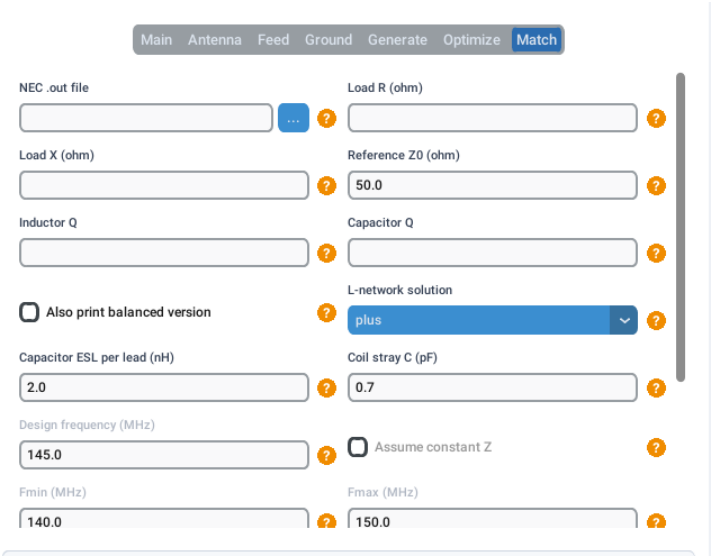



*Figure 5-7. The Match tab.*

- **NEC .out file**: the simulator result to read the impedance from (e.g. *sweep.out*).
- **Load R / Load X**: alternatively, type an impedance by hand (no NEC file needed).
- **Reference Z0**: the impedance to match to (50 Ω).
- **Inductor Q / Capacitor Q**: quality of real components, to estimate their losses (150 and 500 are typical).
- **L-network solution**: plus (low-pass) or minus (high-pass) — the two possible circuits.
- **Design frequency, Fmin, Fmax**: for the sweep mode.
- Further down: commercial values, tolerances and parasitic values of the components.

# 6. Tutorial: your first antenna

Let's design, simulate and look at a classic HB9CV for 145 MHz. This takes about one minute.

## 6.1 Set up the run

1. Start **HB9CV Studio** from the applications menu (Linux) or the Start menu (Windows); see section 3.7.
2. Check the **Work folder** (number 5 in figure 4-1). Click **…** and choose an empty folder of your own, for example *Documents/hb9cv*.
3. Check the **nec2c** box. If nec2c is in the PATH, leave it empty; otherwise click **…** and select the nec2c program.
4. Make sure **generate** is selected in the mode selector and **Compute patterns for the plots** is ticked.
5. In the **Main** tab leave *Frequency* empty (145 MHz is the default) or type your frequency.
6. Type **first.nec** in **Output .nec file** and **first_build.md** in **Build sheet (.md)**.
7. Tick **Run nec2c after generating**.
8. Look at the **command preview**: it now reads `python hb9cv.py --mode generate --output=first.nec --build-sheet=first_build.md --run`.

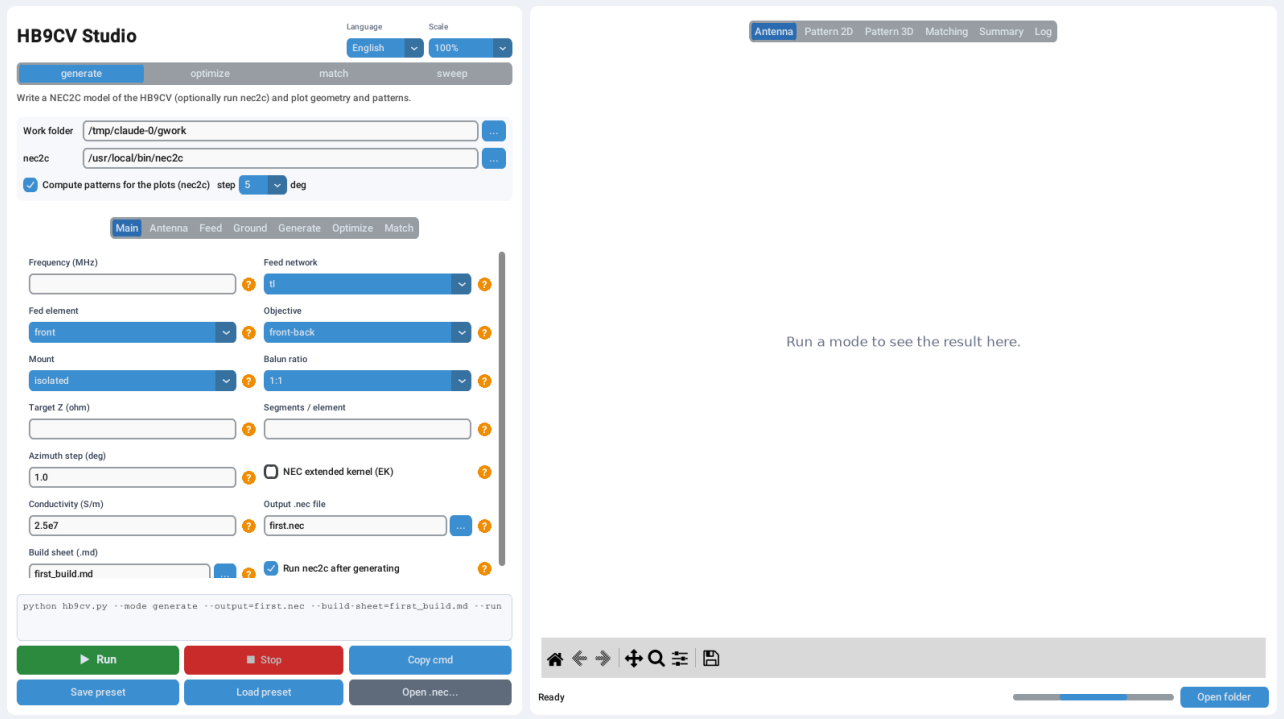



*Figure 6-1. Ready to run the first antenna.*

## 6.2 Run it

Press the green **Run** button. The status bar shows *generate running...* and the progress bar moves. After a few seconds the status changes to **Done (with notes – see Summary)** and the right half shows the drawing of the antenna. 'With notes' only means that the program has comments for you (for example the balun reminder); it is not an error.

The **Log** tab shows everything the engine printed, line by line, exactly as in a terminal:

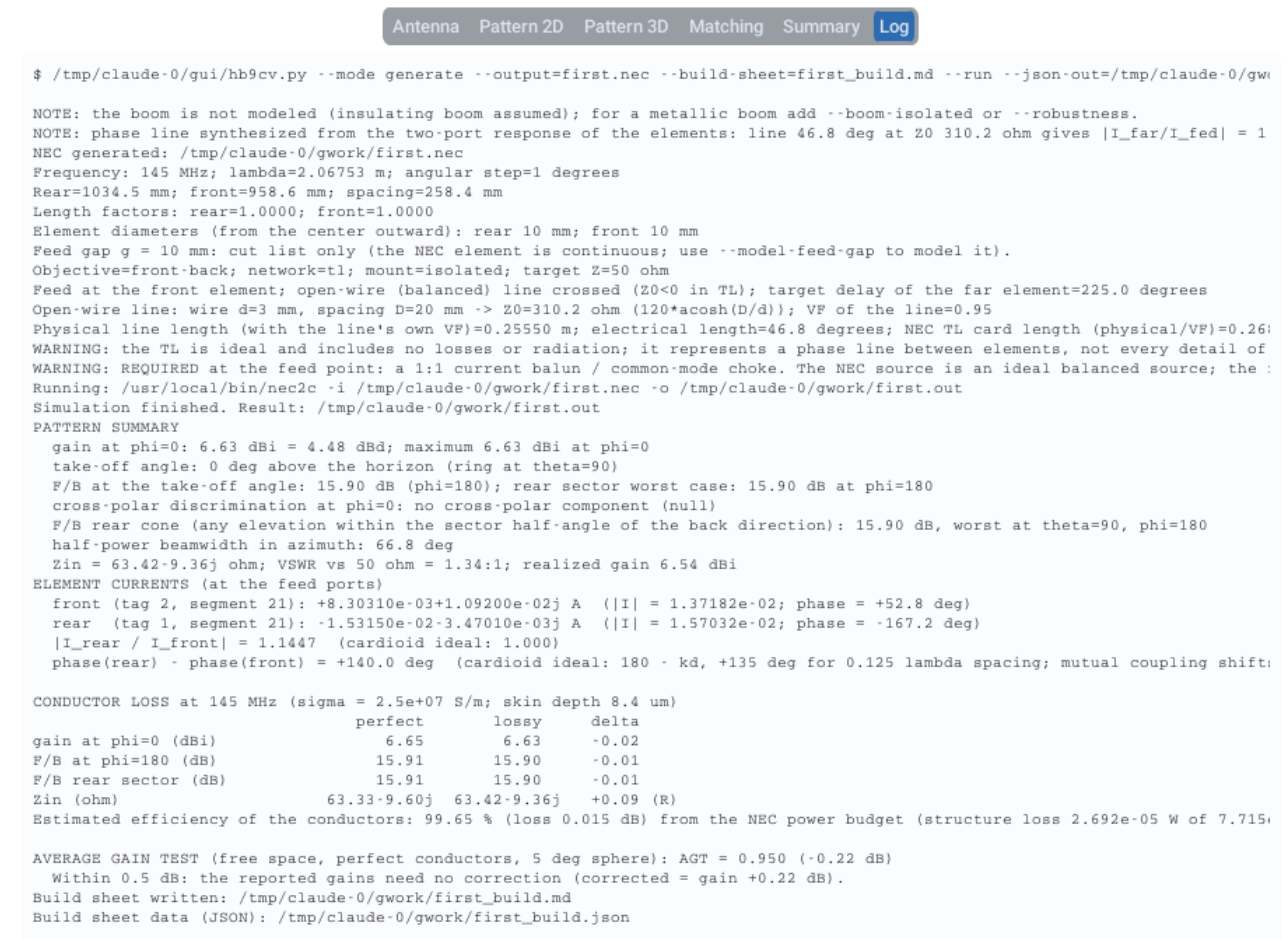



*Figure 6-2. The Log tab after the run. Lines starting with NOTE and WARNING are explained in chapter 16.*

Among other things the log says: the frequency and wavelength (145 MHz, λ = 2.068 m), the dimensions (rear 1034.5 mm, front 958.6 mm, spacing 258.4 mm), the phasing line (open-wire line of 3 mm wire at 20 mm spacing, Z0 = 310 Ω), and a PATTERN SUMMARY with gain, F/B, beamwidth and impedance.

> **TRY IT** Change the frequency to 435 and press Run again. All the dimensions shrink by about 3 times: the program scales the whole antenna to the new wavelength.

# 7. Reading the results

After a run, six result tabs on the right show different views of the same antenna. This chapter explains each one using the first antenna from chapter 6.

## 7.1 Antenna tab — the drawing and the key numbers

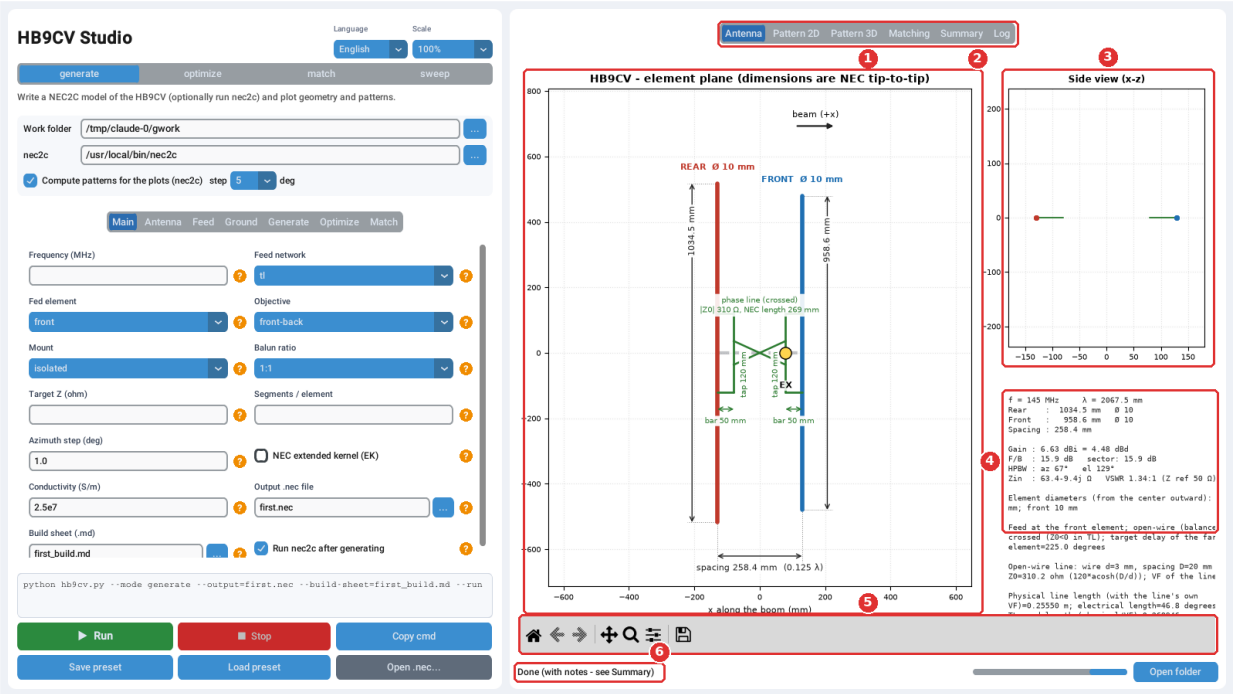



*Figure 7-1. The Antenna tab.*

| # | Part | What it shows |
|---|---|---|
| 1 | Result tabs | Switch between the six views. |
| 2 | Element plane | Top view of the antenna, to scale, in millimetres. Red = rear element, blue = front element, green = phasing line and T-match bars, yellow dot = feed point (EX). The arrow marks the beam direction. All lengths are labelled: element lengths (tip to tip), spacing, tap distance, bar length and the phasing line impedance and length. |
| 3 | Side view | The antenna seen from the side (x–z plane). Over ground it shows the height. |
| 4 | Figures panel | The essential numbers: frequency and wavelength, lengths and diameters, gain in dBi and dBd, F/B (point and sector), half-power beamwidth (az = horizontal, el = vertical), Zin and VSWR. Scroll it for the details of the line and feed. |
| 5 | Toolbar | Zoom into the drawing, pan, return home, or save the picture. |
| 6 | Status | Result of the run. |

In this example: gain 6.63 dBi (4.48 dBd), F/B 15.9 dB, beamwidth 67° horizontally, Zin = 63.4 − 9.4j Ω, VSWR 1.34:1 against 50 Ω. That is a good, usable antenna with no matching network needed.

**TIP** Use the zoom tool (magnifier) in the toolbar to draw a rectangle around the feed area and see the T-match details. Press the house icon to return to the full view.

## 7.2 Pattern 2D tab — the radiation pattern

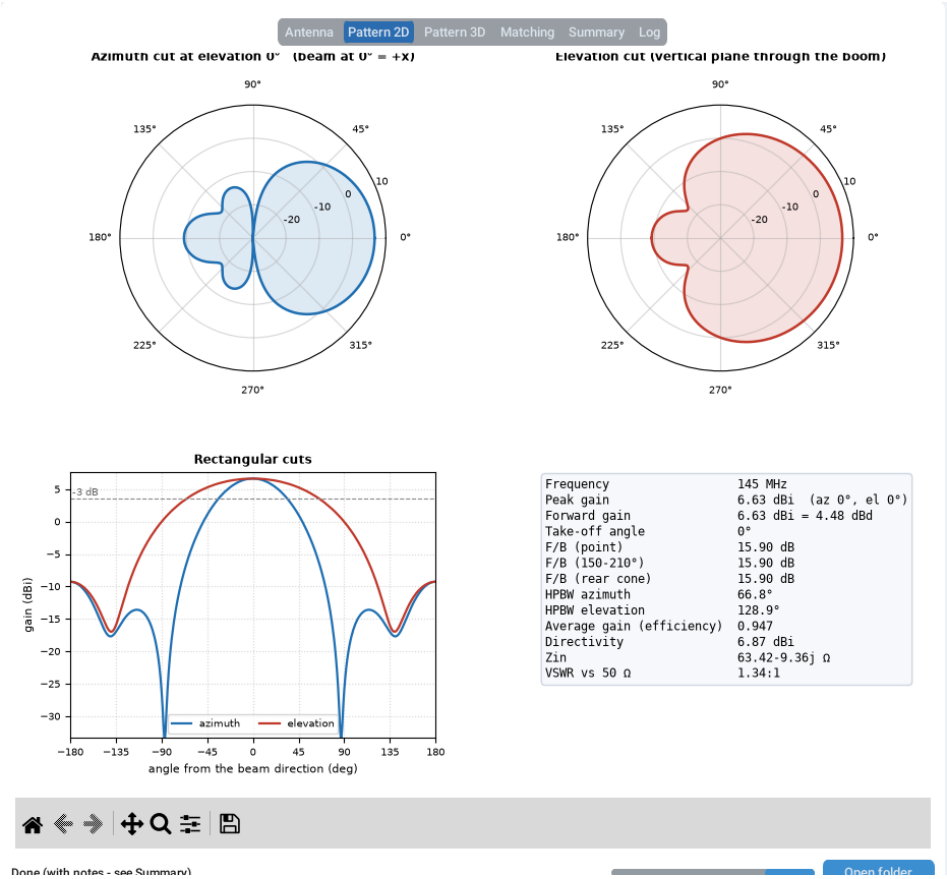



*Figure 7-2. The Pattern 2D tab.*

- **Top-left — Azimuth cut**: the pattern seen from above, at the elevation of the main beam. 0° (right) is the beam direction. The big blue lobe on the right is the forward signal; the small lobe on the left is the signal sent backward. The rings are dB levels (0, −10, −20).
- **Top-right — Elevation cut**: the pattern in the vertical plane through the boom (a side view).
- **Bottom-left — Rectangular cuts**: the same two cuts unrolled on a straight axis from −180° to +180°, easier to read precisely. The dashed line marks −3 dB below the peak: where each curve crosses it gives the beamwidth.
- **Bottom-right — table**: all figures: peak and forward gain, take-off angle, the three F/B values, HPBW, average gain (efficiency), directivity, Zin and VSWR.

## 7.3 Pattern 3D tab — the shape in space

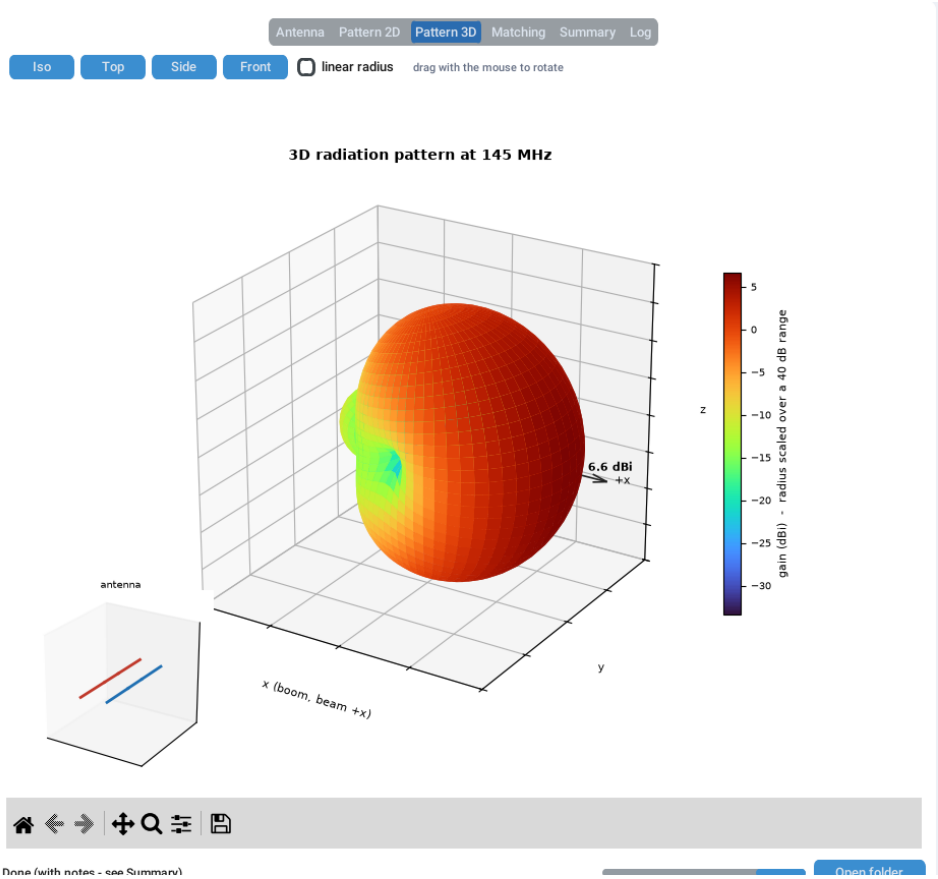



*Figure 7-3. The 3-D pattern (isometric view).*

The coloured surface is the radiation pattern in every direction. Its distance from the centre and its colour indicate the gain (scale on the right, in dBi). Red = strongest. The small box at the bottom-left shows the antenna orientation. Controls at the top:

- **Iso**, **Top**, **Side**, **Front**: preset viewpoints.
- Drag with the mouse to rotate freely.
- **linear radius**: by default the radius is proportional to dB over a 40 dB range, which makes small lobes visible. Ticked, the radius is proportional to the real power, which shows how dominant the main lobe really is.

*Figure 7-4. Left: the Top view. Right: the same pattern with linear radius ticked.*

## 7.4 Summary tab — the report

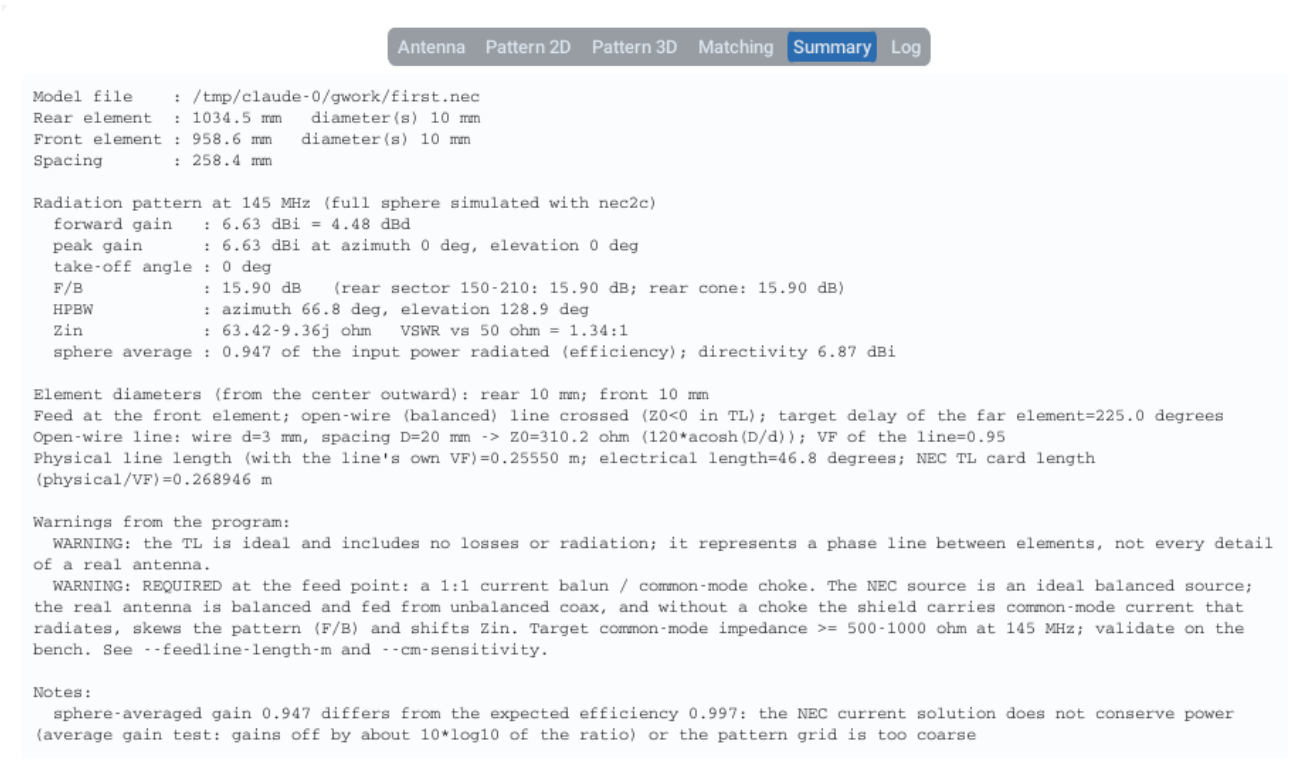



*Figure 7-5. The Summary tab.*

A clean, readable report of the run: files, dimensions, the radiation figures, the description of the feed and line, and two important sections: **Warnings from the program** and **Notes**. Read them every time. Chapter 16 explains the most common ones.

## 7.5 Matching and Log tabs

The **Matching** tab is used by the match and sweep modes (chapter 12). The **Log** tab (figure 6-2) contains the full text output. If something goes wrong, the explanation is always in the Log.

## 7.6 Opening an existing model

The **Open .nec…** button lets you load any NEC model file, for example one you saved earlier or one made by another program. The window draws it and runs nec2c to compute its patterns, filling the result tabs exactly as after a run.

# 8. Adding reflectors and directors

You can add up to 2 extra **reflectors** behind the rear element and up to 5 **directors** in front of the front element. These are not fed (they are 'parasitic'): they are excited by the field of the HB9CV and make the beam narrower and the gain higher, at the cost of a longer boom.

1. Select generate mode and open the **Antenna** tab.
2. Choose **1** in **Reflectors (0-2)** and **2** in **Directors (0-5)**.
3. Leave the lengths and spacings empty to use the defaults, or type lists such as 1.0,0.98 (lengths as a factor of the resonant length) and 0.15,0.2 (spacings in wavelengths).
4. Name the output (e.g. *yagi.nec*), tick Run nec2c after generating, and press **Run**.

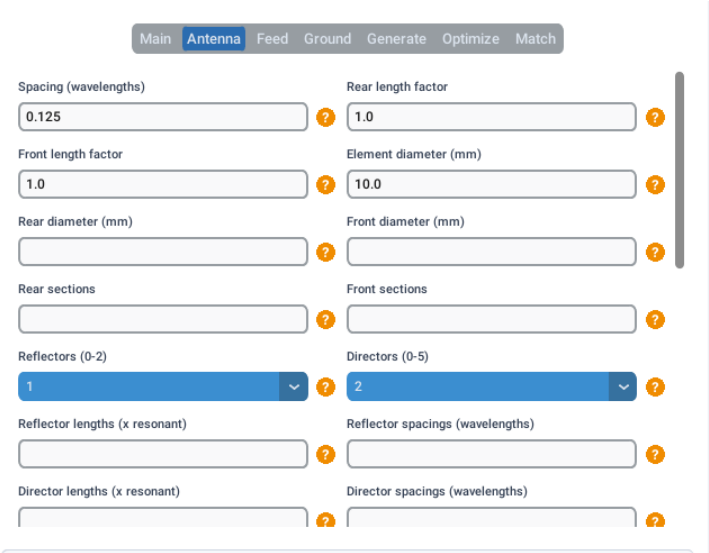



*Figure 8-1. One reflector and two directors selected.*

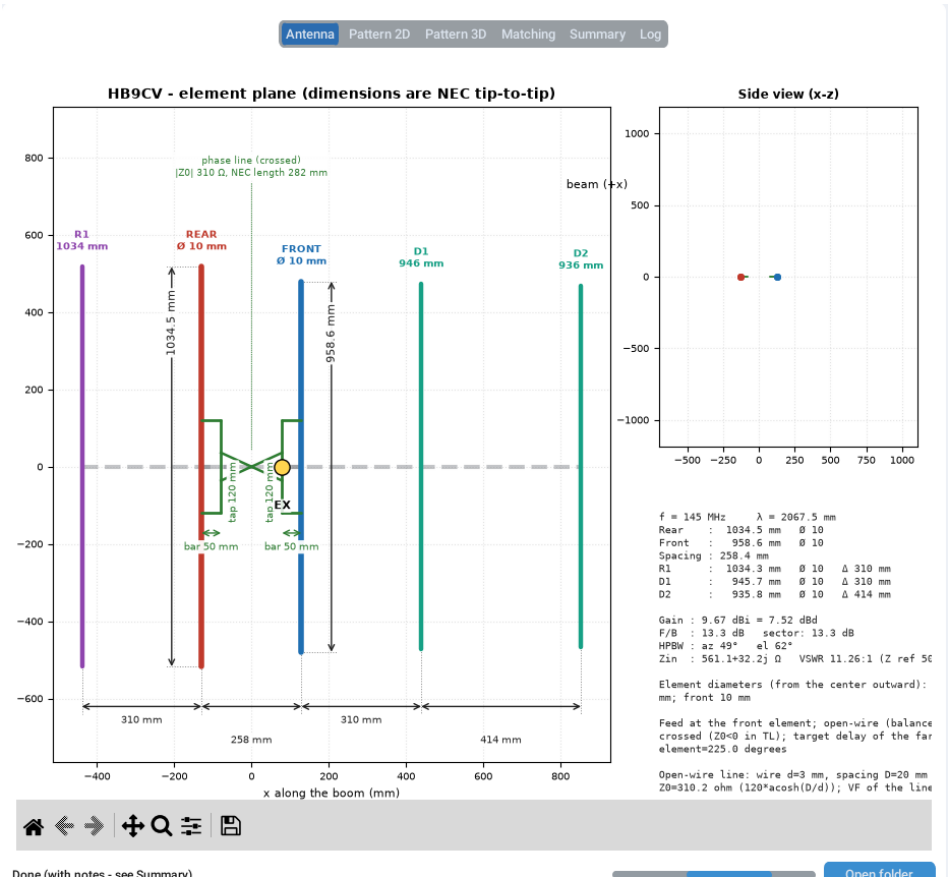



*Figure 8-2. The resulting antenna: R1 (purple) behind, D1 and D2 (green) in front.*

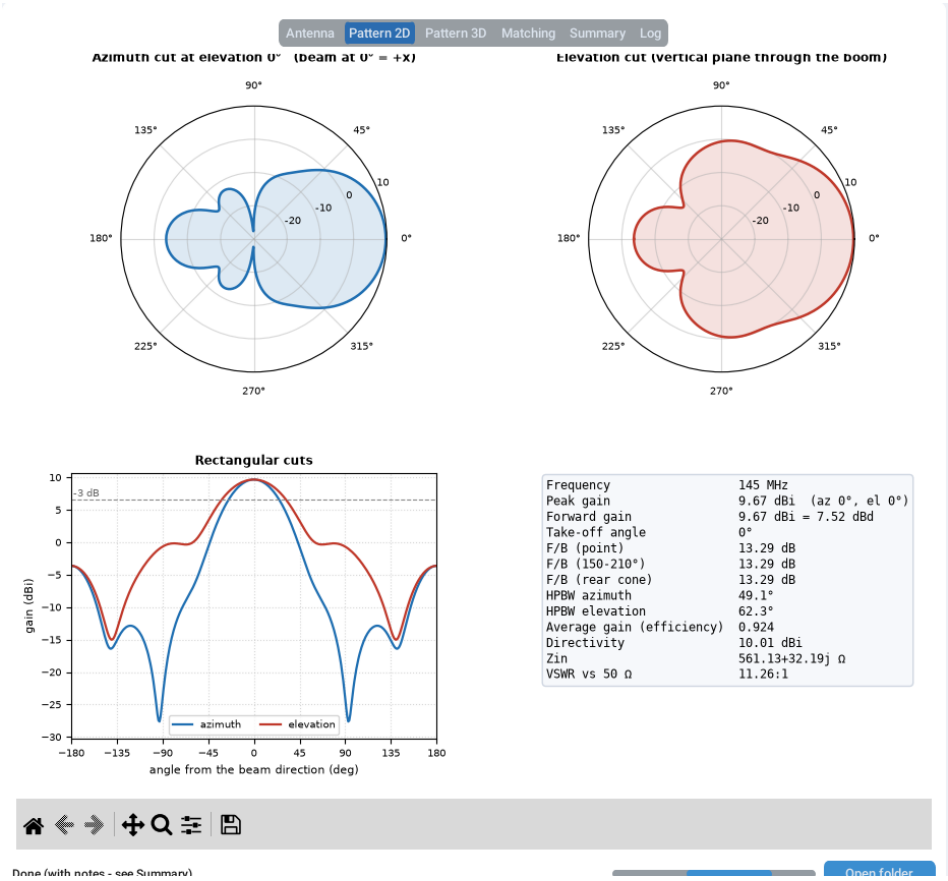



*Figure 8-3. Its pattern: gain rises to 9.67 dBi and the beam is narrower (49°).*

**CAUTION** Look at the impedance in figure 8-3: Zin = 561 + 32j Ω, VSWR 11:1. Adding elements changes the impedance a lot. A design like this must be re-optimised (chapter 11, set the reflector/director counts in optimize mode) and/or matched (chapter 12) before building it.

# 9. The antenna over real ground

By default the antenna is simulated in **free space**, far from everything. That is the best way to compare designs. But a real antenna is installed at some height above the earth, which reflects part of the signal. The reflection changes the vertical pattern a lot (and slightly the impedance).

1. Open the **Ground** tab. Choose **Ground** = **average** (typical soil).
2. Type the **Height (m)**, for example **10**.
3. Choose the **Orientation**: horizontal or vertical polarisation.
4. Press **Run**.

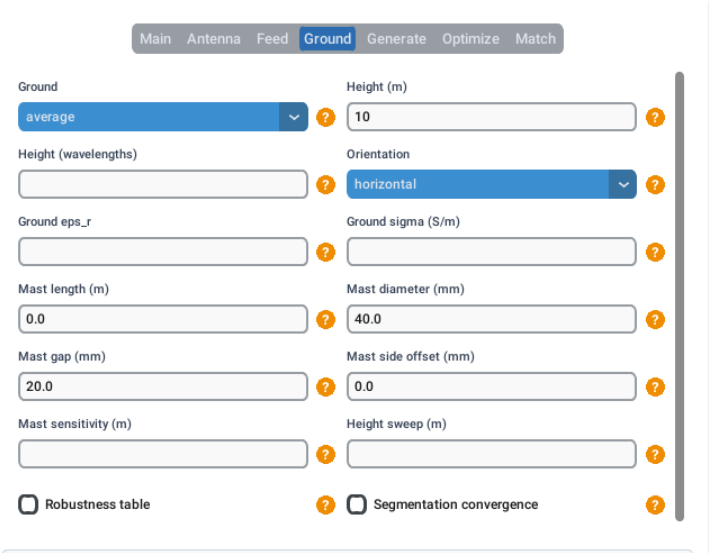



*Figure 9-1. Average ground, 10 m height.*

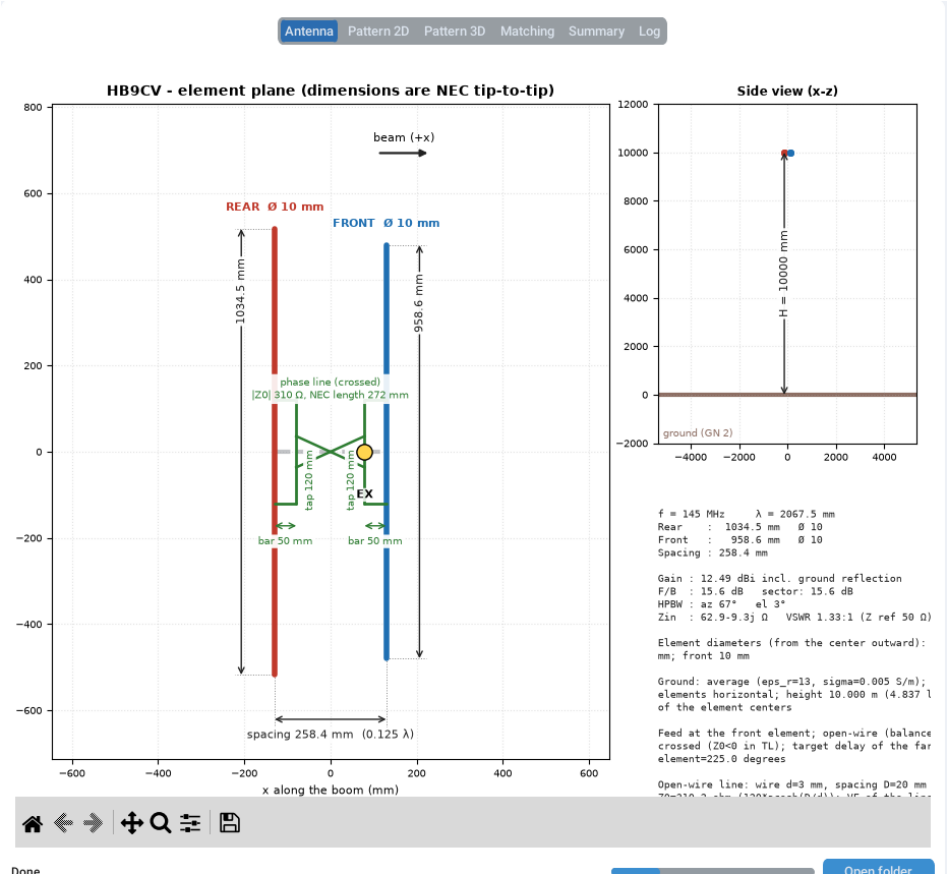



*Figure 9-2. The side view now shows the antenna 10 m above the ground line.*

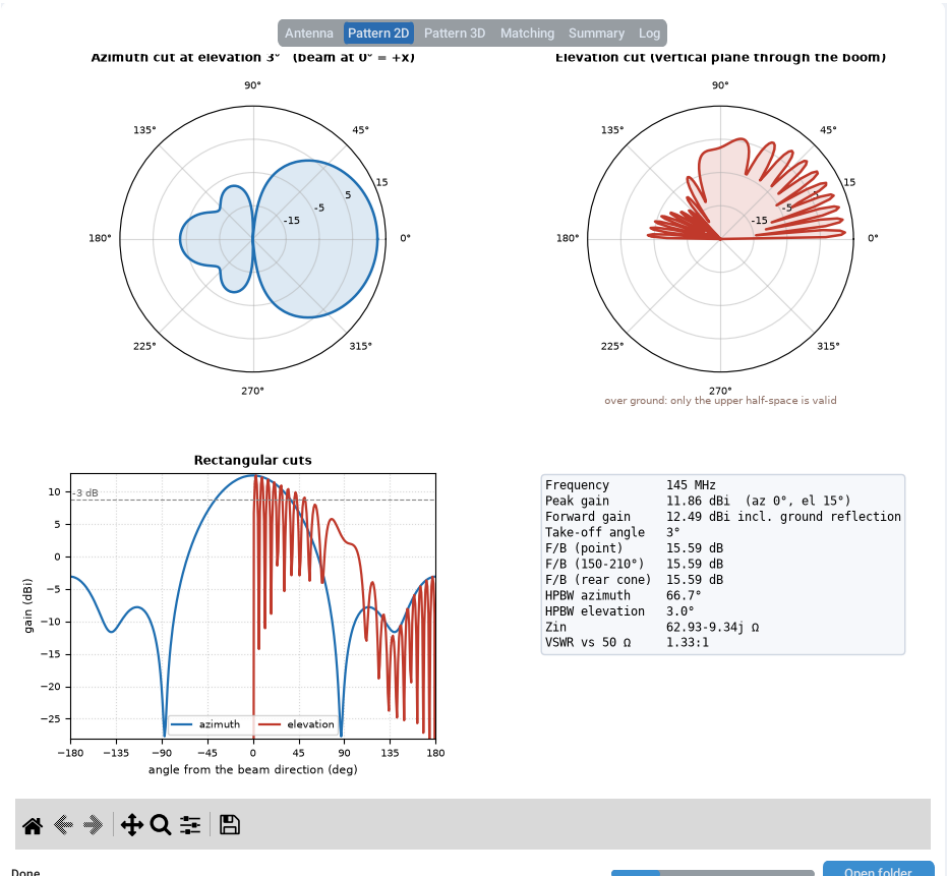



*Figure 9-3. Pattern over ground: the elevation cut breaks into many lobes.*

Look at the elevation cut (top-right): instead of one smooth lobe there are many narrow lobes. They are caused by the ground reflection adding to or cancelling the direct wave. The lowest lobe is the useful one for long-distance contacts: its elevation is the **take-off angle** (3° here). Gain now includes the ground reflection (11.86 dBi peak). Note that only the upper half of the space is valid over ground.

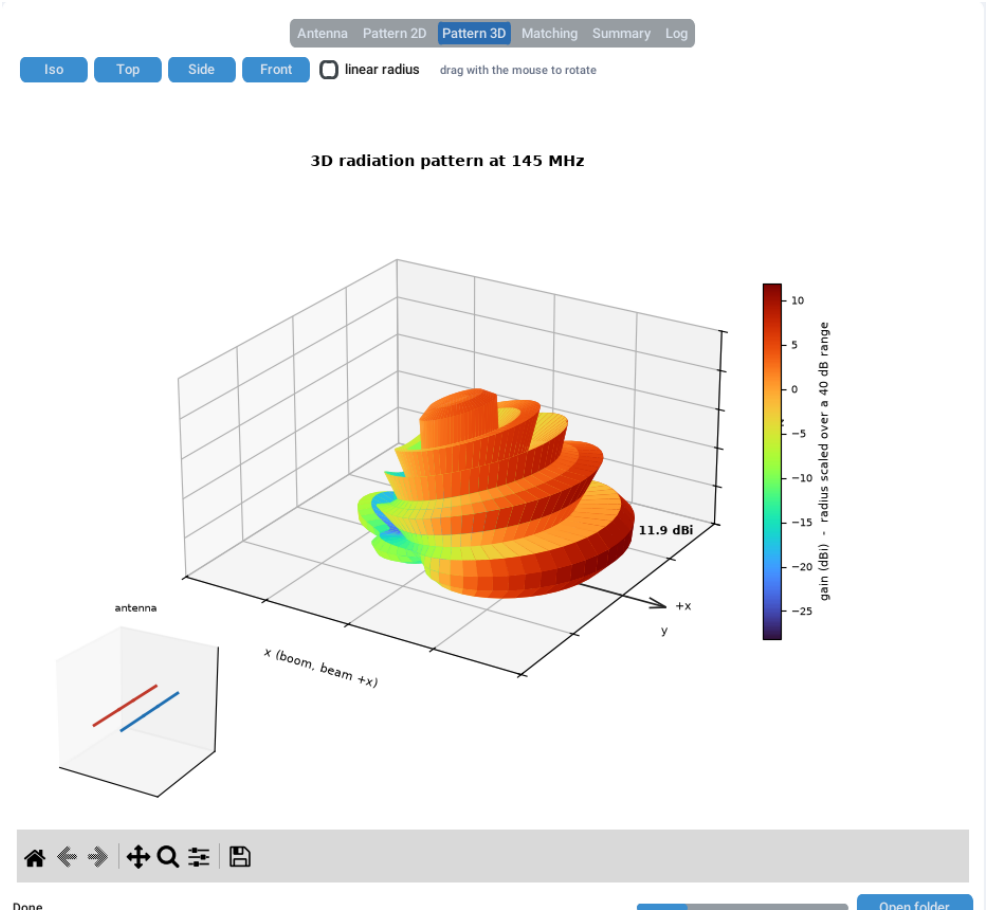



*Figure 9-4. The 3-D pattern over ground, with its layered lobes.*

**TIP** Use **Height sweep (m)** in the Ground tab (for example 4,12,5) to simulate 5 heights between 4 and 12 m and get a table of take-off angle, gain and impedance for each height.

# 10. Frequency sweep and bandwidth

A design is made for one frequency, but you will use it over a band. The frequency sweep simulates the same antenna at many frequencies to see how the impedance and VSWR change.

1. In generate mode open the **Generate** tab.
2. In **Frequency sweep** type **140,150,0.5** (from 140 to 150 MHz in 0.5 MHz steps).
3. Give an output name in the Main tab, e.g. **sweep.nec**, tick Run nec2c after generating, and press **Run**.

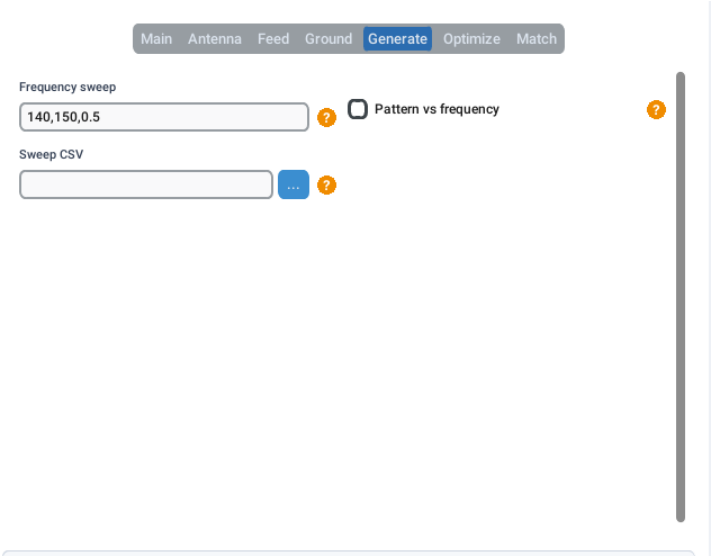



*Figure 10-1. A frequency sweep from 140 to 150 MHz.*

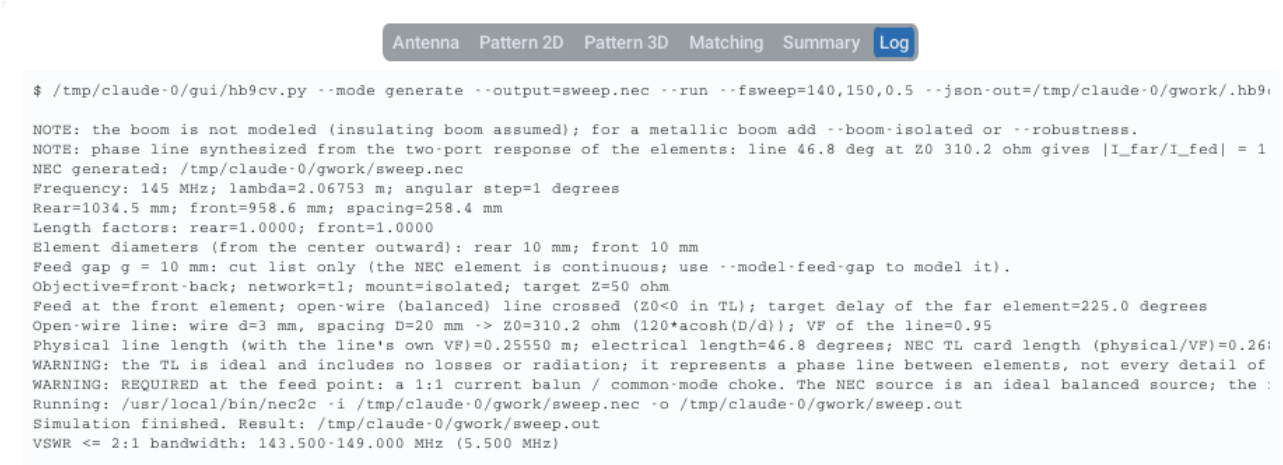



*Figure 10-2. The last line of the Log gives the 2:1 bandwidth.*

The last line reads **VSWR ≤ 2:1 bandwidth: 143.500–149.000 MHz (5.500 MHz)**: the antenna can be used without a tuner over the whole 2 m band. The simulator also wrote **sweep.out**, with the impedance at every frequency; the match and sweep modes use it in chapter 12.

# 11. The optimizer: let the program find the best design

Instead of guessing dimensions, the optimizer tries many combinations, simulates each one with nec2c, scores them and keeps the best. It then refines the best candidates with a finer search. The result is written as a normal model file plus a build sheet.

## 11.1 A ready-made example

The quickest way to try it: start the program with `python3 hb9cv_gui.py --load-default-example`. The boxes are filled with a complete optimisation for 146 MHz, maximising gain, over the band 145–147 MHz.

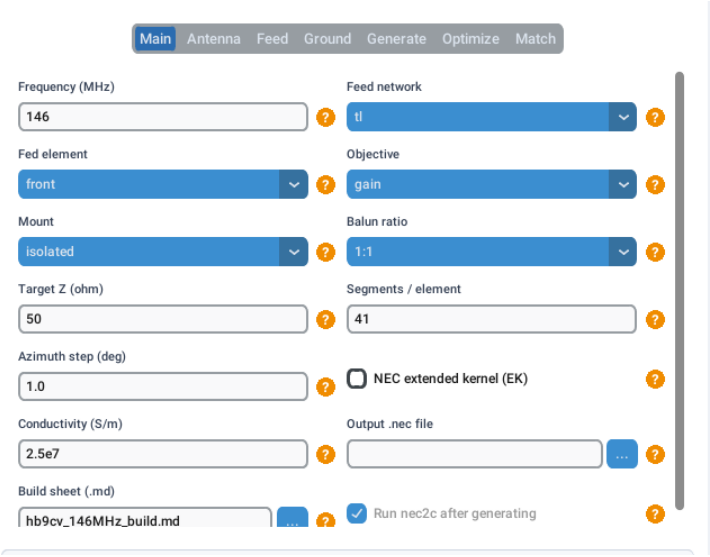



*Figure 11-1. The Main tab of the example: 146 MHz, objective gain, target 50 Ω, 41 segments.*

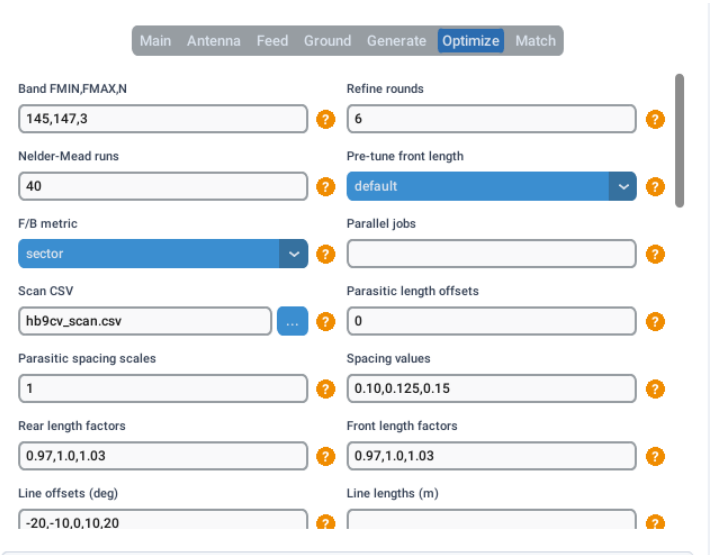



*Figure 11-2. The Optimize tab of the example.*

The Optimize tab defines the search:

| Box (example value) | Meaning |
|---|---|
| Band FMIN,FMAX,N (145,147,3) | Score each candidate at 3 frequencies (145, 146, 147 MHz) and keep the worst case, so the design works across the band. |
| Refine rounds (6) | How many rounds of local refinement around the best candidates. |
| Nelder-Mead runs (40) | Iterations of the final fine-tuning algorithm. |
| F/B metric (sector) | Which F/B definition is used in the score (see chapter 2). |
| Spacing values (0.10,0.125,0.15) | Element spacings to try, in wavelengths. |
| Rear / Front length factors (0.97,1.0,1.03) | Element length multipliers to try. |
| Line offsets (deg) (-20,-10,0,10,20) | Changes of the phasing line electrical length to try. |
| Scan CSV (hb9cv_scan.csv) | Spreadsheet with every candidate and its score. |
| Parallel jobs | How many nec2c runs at the same time (empty = one per CPU core). Lower it if the computer becomes slow to respond. |

The number of candidates multiplies quickly: 3 spacings × 3 rear × 3 front × 5 offsets × 3 line impedances = 405 combinations, each simulated at 3 frequencies. Start with short lists.

## 11.2 Running

Press **Run**. The Log shows progress lines such as *[40/191] valid=40; best score=…*. You can press **Stop** at any time. An optimisation typically takes from one to several minutes.

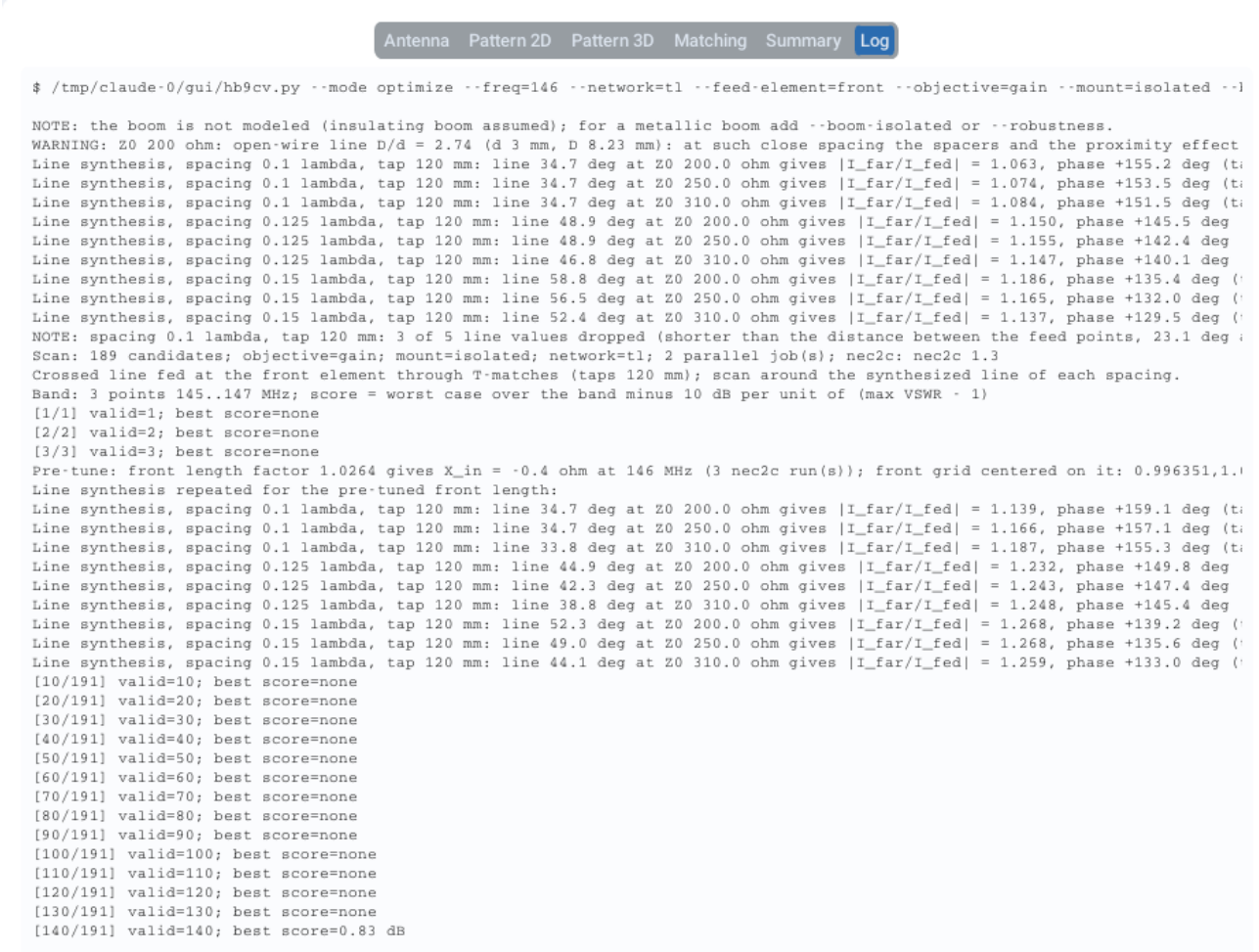



*Figure 11-3. An optimisation in progress.*

## 11.3 The result

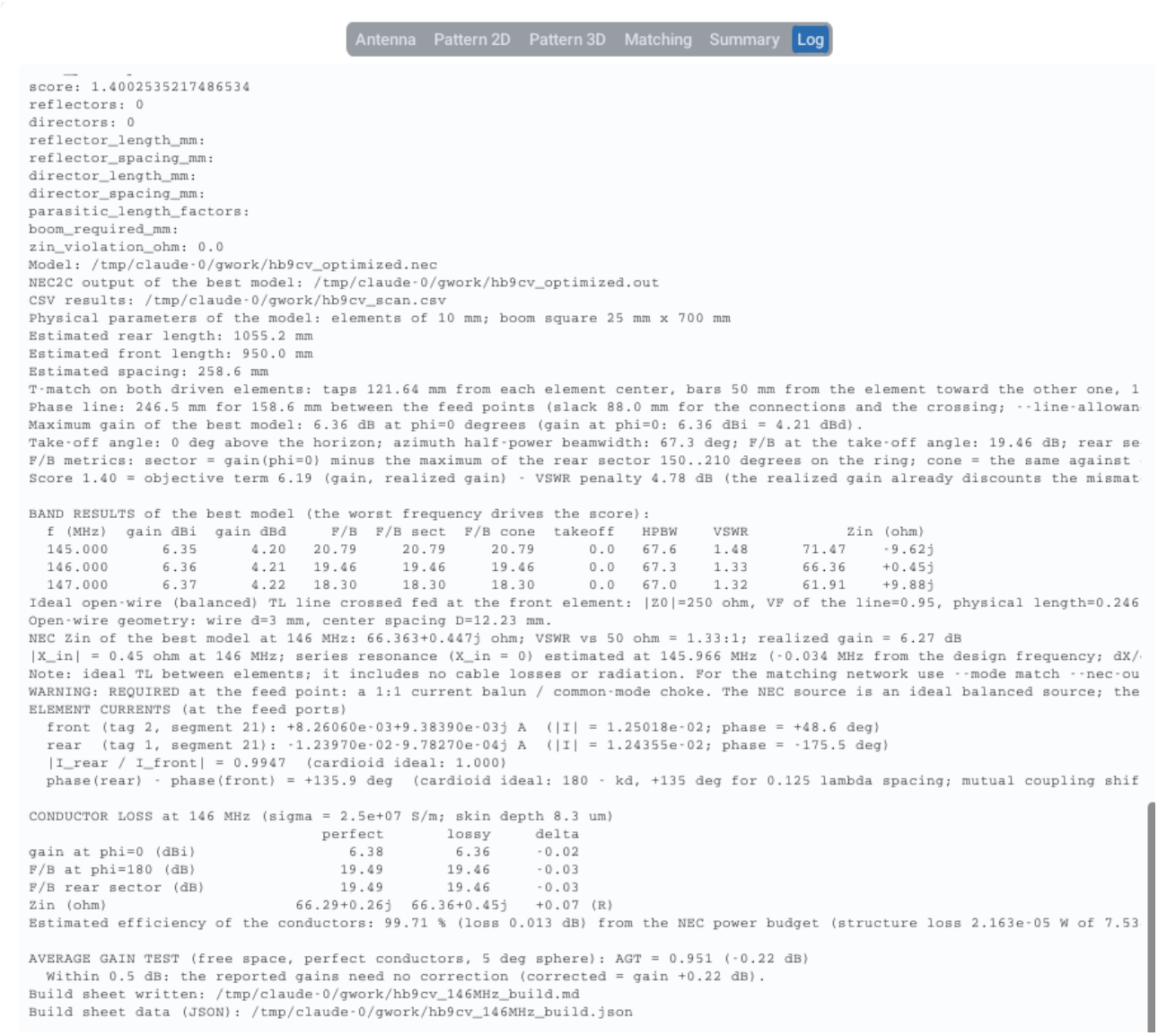



*Figure 11-4. The end of the optimisation log.*

The end of the log describes the winner: estimated element lengths and spacing, the T-match tap positions, the phasing line length, the maximum gain, the F/B values, and a **BAND RESULTS** table with gain, F/B, beamwidth, VSWR and Zin at each band frequency. The best model is drawn and its patterns computed:

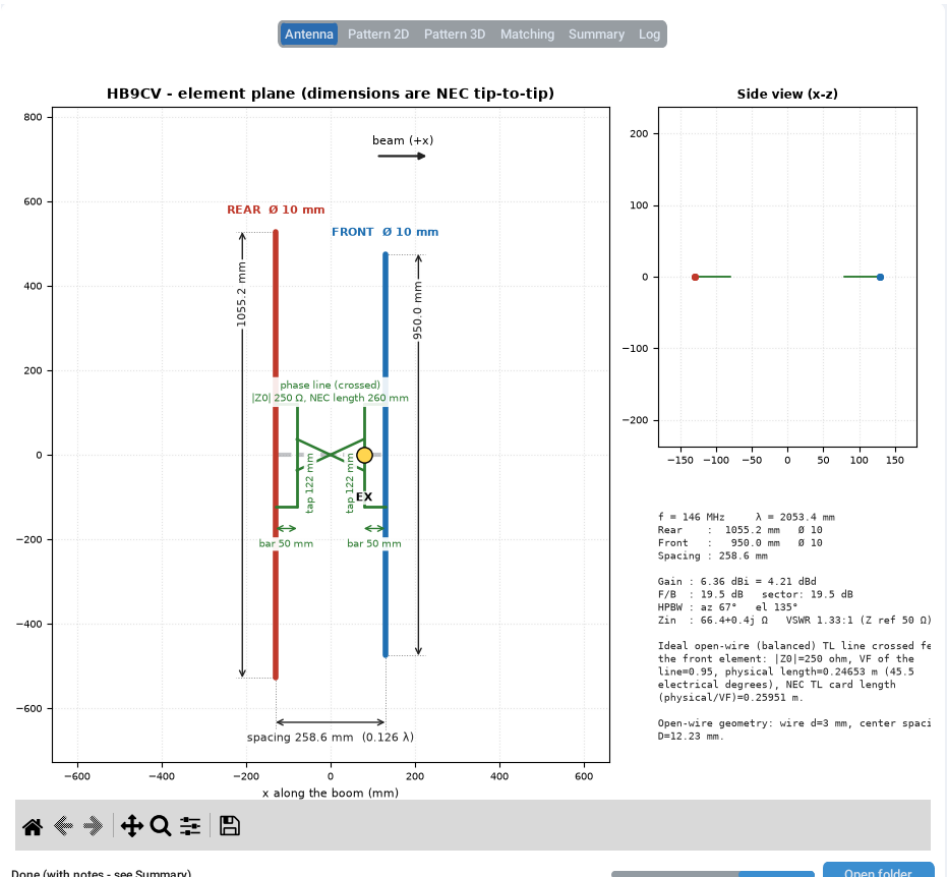



*Figure 11-5. The optimised antenna.*

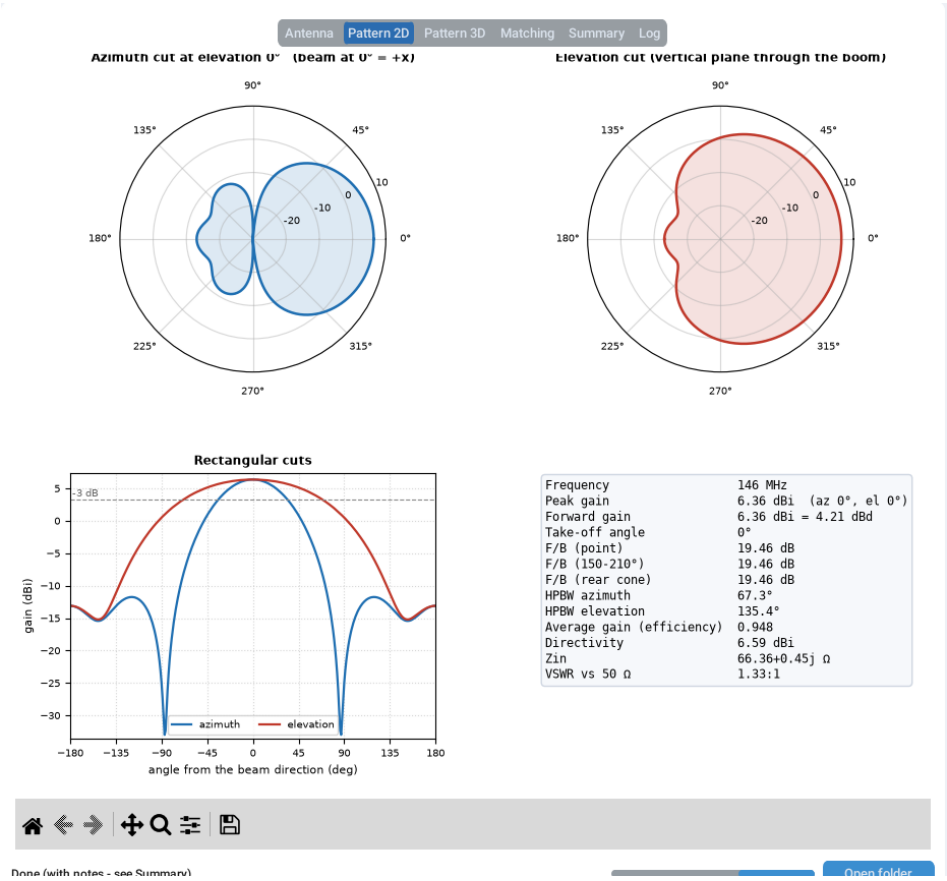



*Figure 11-6. Its pattern: 6.36 dBi gain and a much better F/B of 19.5 dB.*

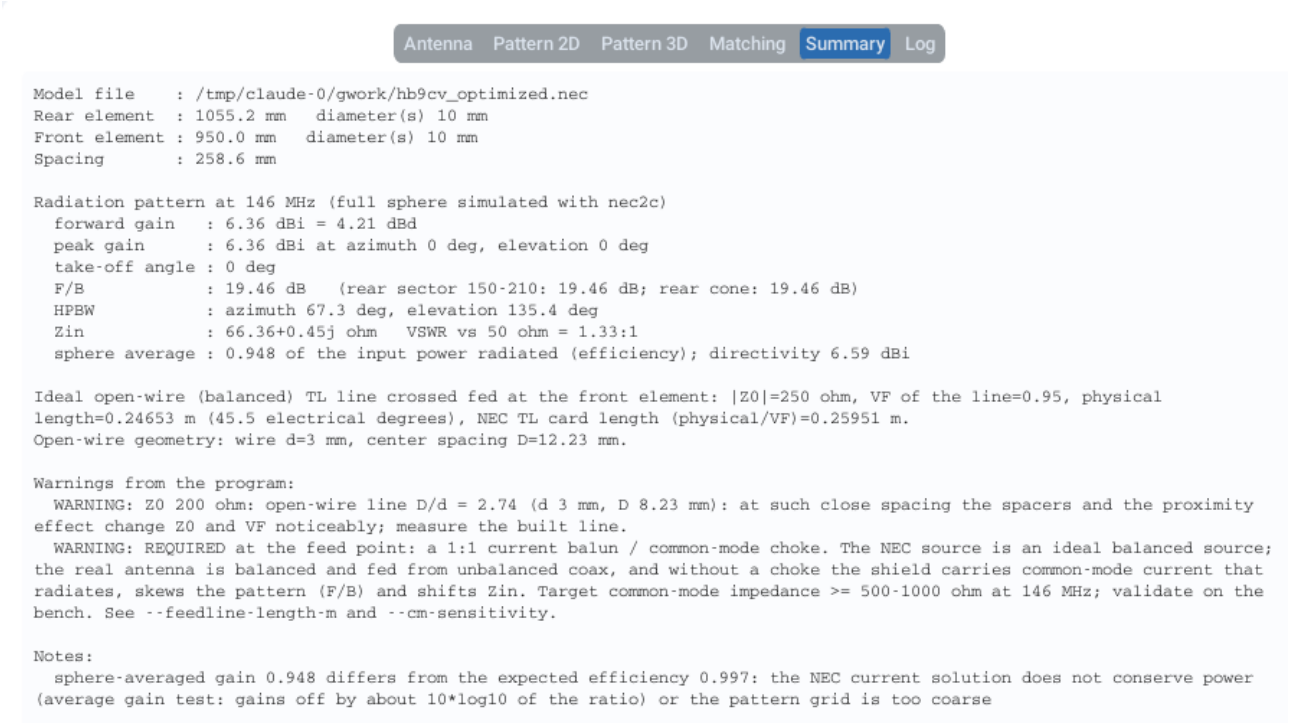



*Figure 11-7. The Summary of the optimised design.*

## 11.4 Important messages of the optimizer

- **Zin window**: the options *Admissible R_in (min,max)* and *Max |X_in|* ask for an impedance window (for example R between 30 and 75 Ω, |X| below 10 Ω). If no candidate reaches it, the program still writes the **closest** candidate and prints a WARNING explaining how far it is. Widen the search lists or the window, or match the result (chapter 12). Tick *Stop if no Zin match* if you prefer an error instead.
- **Matching network used by the score**: when matching is allowed in the score, the optimizer reports the L-network it assumed, and a NOTE like *the raw VSWR of the best model is 7.04:1 … it is meant to be fed through the matching network above*. In that case build the matching network with match mode, or set *Score through the L-network = no* to search for designs that connect directly to 50 Ω.
- **Unphysical runs rejected**: rarely, a geometry makes NEC produce impossible results (negative resistance or efficiency above 100 %). Those candidates are discarded automatically and counted in the log.

## 11.5 Monte-Carlo tolerance analysis

No antenna is built exactly to the millimetre. With *Monte-Carlo samples* (100 or more) and the *MC:* tolerance boxes (elements ±mm, spacing ±mm, tap ±mm, line VF ±%, Z0 ±%, capacitor ±%) the program builds many slightly-wrong copies of the best candidates and reports, for each one, the score that 90 % of the copies beat (p10), the typical F/B and the VSWR. Prefer a candidate whose p10 score stays close to its nominal score: it is less sensitive to construction errors.

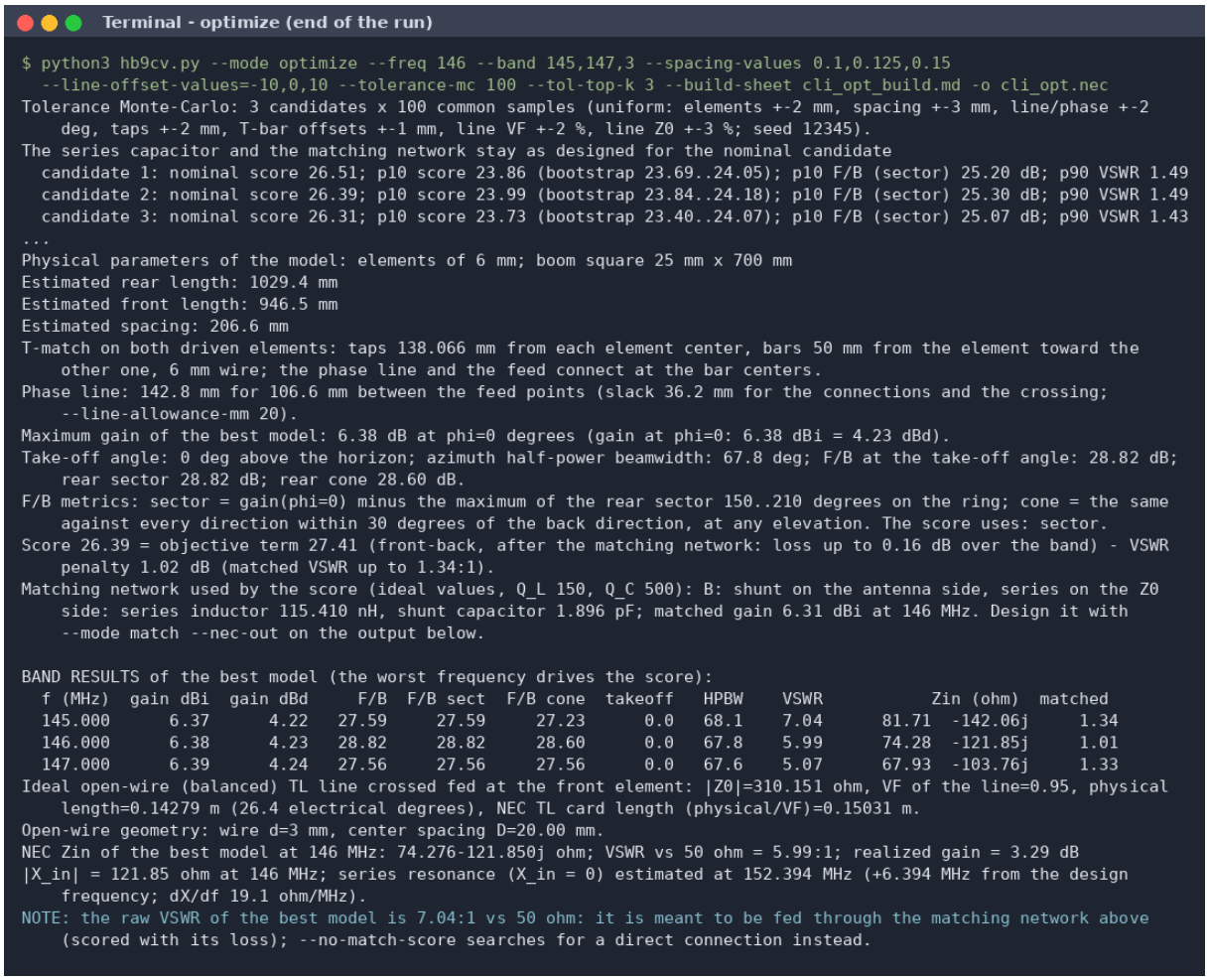



*Figure 11-8. The Monte-Carlo section and the final report of an optimisation run from the terminal.*

# 12. Matching network: match and sweep modes

If the antenna impedance is far from 50 Ω (VSWR above about 1.5:1) you can add an **L-network** at the feed point: one inductor and one capacitor, one in series and one in parallel (shunt). The *match* mode computes the values; the *sweep* mode checks them over a range of frequencies.

## 12.1 Match mode

1. First produce a NEC result with the impedance, e.g. the frequency sweep of chapter 10 (**sweep.out**).
2. Select **match** in the mode selector and open the **Match** tab.
3. In **NEC .out file** choose **sweep.out** (or type Load R and Load X by hand instead).
4. In the Main tab type the **Frequency** to match (it must be one of the swept frequencies, e.g. 145).
5. Optional: **Inductor Q** 150 and **Capacitor Q** 500 to include component losses.
6. Press **Run**.

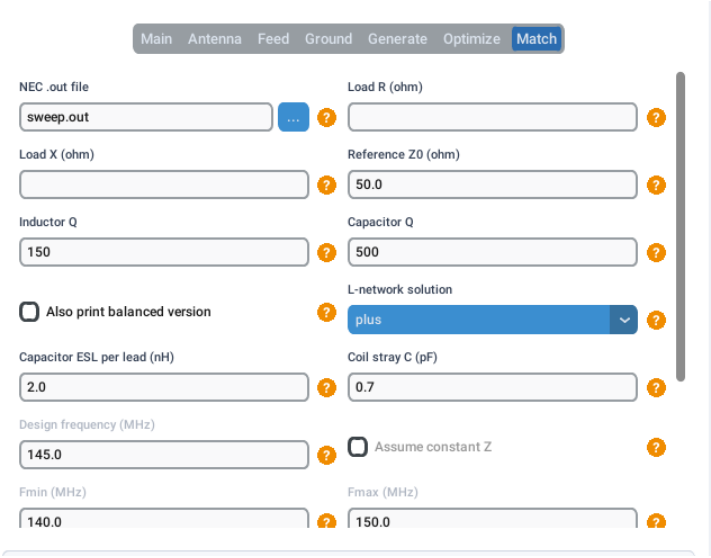



*Figure 12-1. Match settings.*

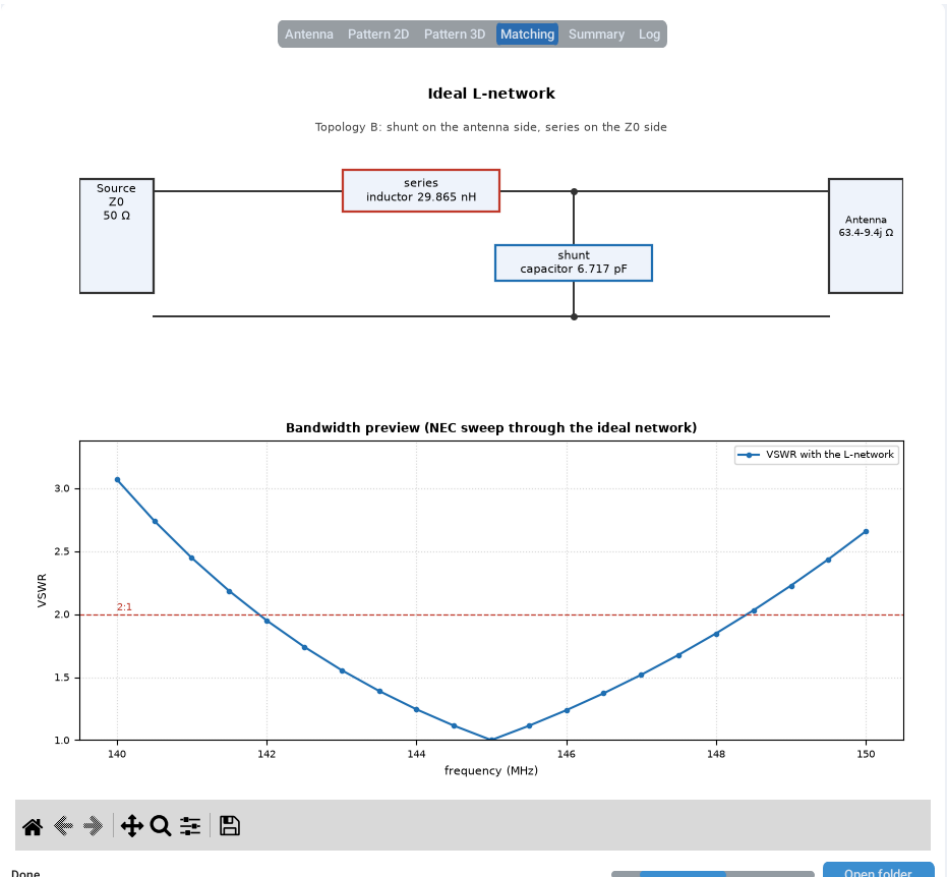



*Figure 12-2. The Matching tab: the circuit and its bandwidth preview.*

The upper drawing shows the circuit: **Source** (your 50 Ω cable) on the left, the **Antenna** (63.4 − 9.4j Ω) on the right, and between them a **series inductor of 29.865 nH** and a **shunt capacitor of 6.717 pF**. The graph below is the VSWR of the antenna with this network across the sweep: perfect 1.0:1 at 145 MHz and below 2:1 from about 142 to 148.5 MHz.

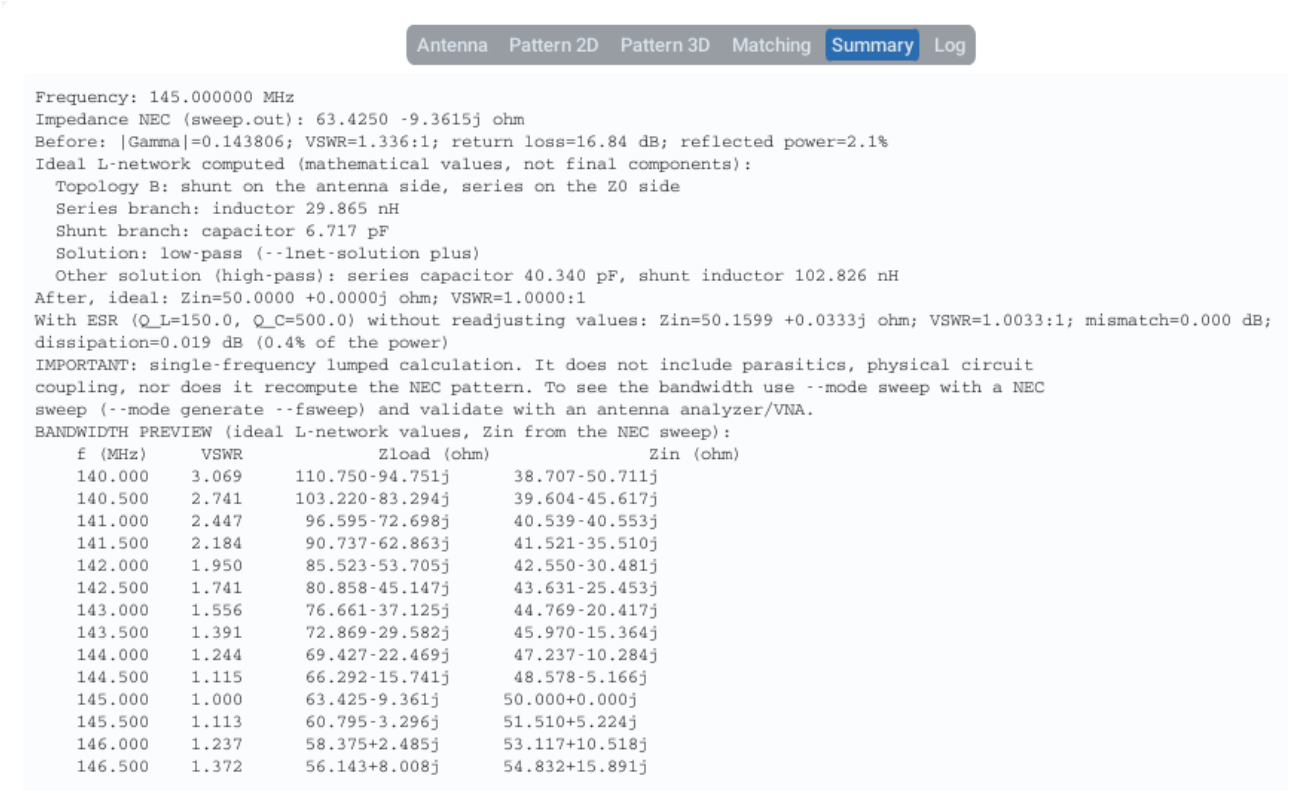



*Figure 12-3. The Summary of match mode.*

The summary lists the VSWR and return loss before and after, both possible solutions (low-pass with a series inductor, or high-pass with a series capacitor), the losses caused by real components, and a table frequency by frequency. Read the IMPORTANT note: the values are mathematical; the wires and pads of the real circuit add small inductances and capacitances, so the circuit must be adjusted with an analyzer.

## 12.2 Sweep mode

The sweep mode takes the network designed at one frequency and evaluates it at all the frequencies of the NEC file, with real (commercial) component values and their tolerances.

1. Select **sweep** in the mode selector.
2. In the Match tab: **NEC .out file** = sweep.out, **Design frequency** = 145, Fmin 140, Fmax 150.
3. Optional: commercial values (e.g. 30,6.8 meaning 30 nH and 6.8 pF) and tolerances (e.g. 5 % for L and C).
4. Press **Run**.

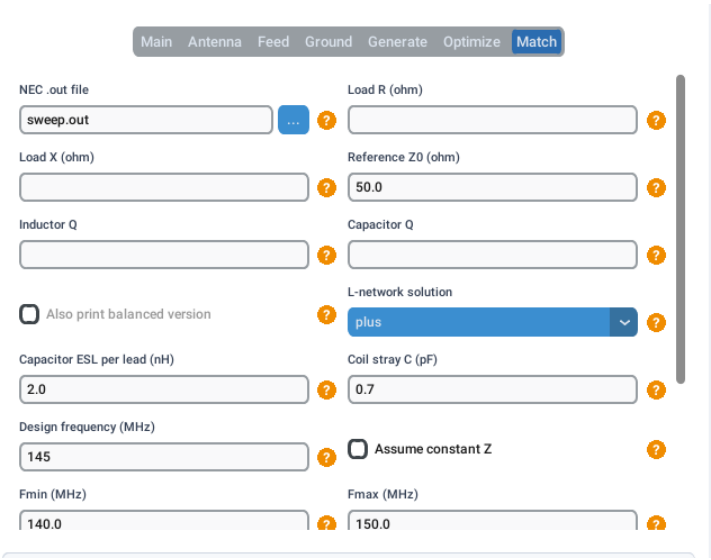



*Figure 12-4. Sweep settings.*

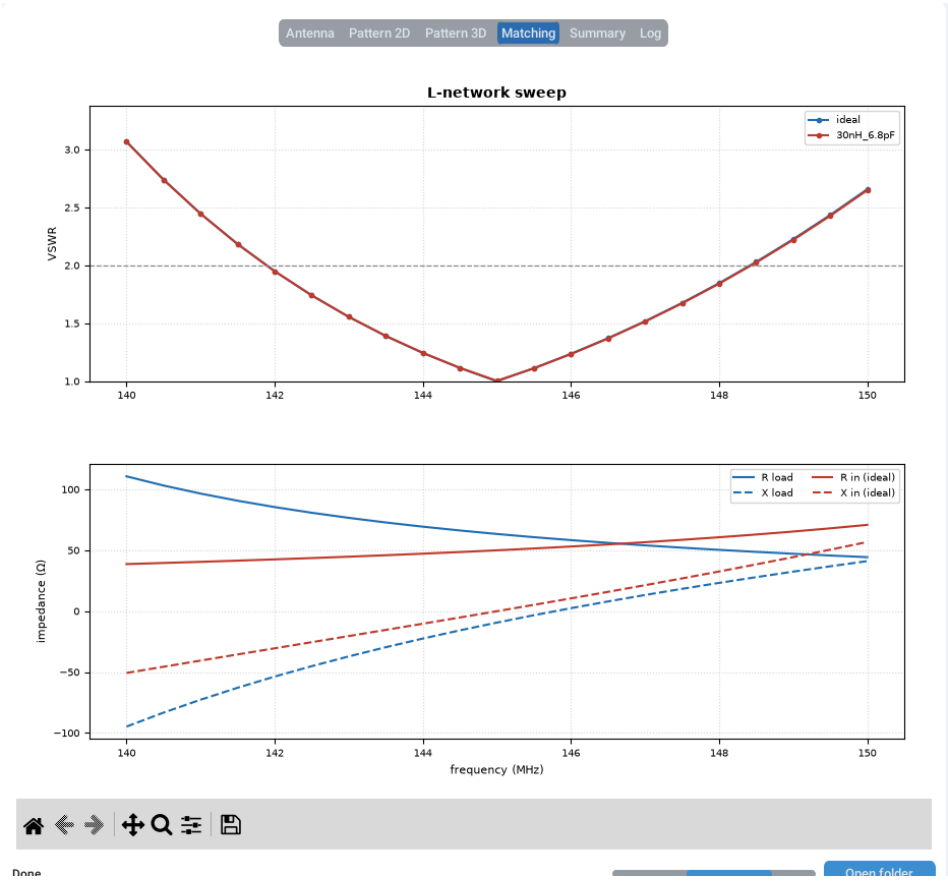



*Figure 12-5. The sweep result: VSWR (top) and impedances (bottom) versus frequency.*

The top graph compares the ideal network with the nearest standard values (30 nH and 6.8 pF): they are almost identical. The bottom graph shows the antenna impedance (R load, X load) and the impedance after the network (R in, X in). The Log adds a **TOLERANCE ANALYSIS**: with ±5 % components the worst-case VSWR stays below 2:1 from 142.5 to 148 MHz.

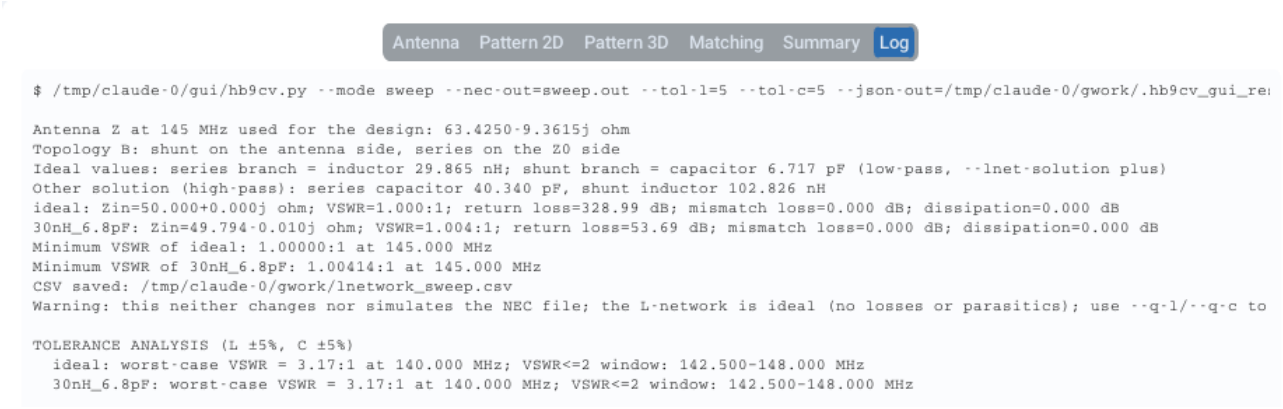



*Figure 12-6. The sweep log with the tolerance analysis.*

# 13. Build sheet and tuning

When a **Build sheet (.md)** name is given, the program writes a construction document (Markdown text, readable with any text editor, or nicely formatted by Markdown viewers) and a .json file with the same data. It contains everything needed to build the antenna:

- **1. Elements**: NEC length, cut length of each half, tube sections and which element is fed.
- **2. Spacing and boom**: element-to-element distance, positions along the boom, overhangs.
- **3. Phasing line**: type, wire diameter and spacing, Z0, velocity factor, electrical and physical length, slack for connections and the crossing, which end goes where.
- **3b. T-match**: tap distance from the centre, bar offsets, wire diameters.
- **4. Feed point**, balun recommendation, matching network (if any) and a tuning procedure.

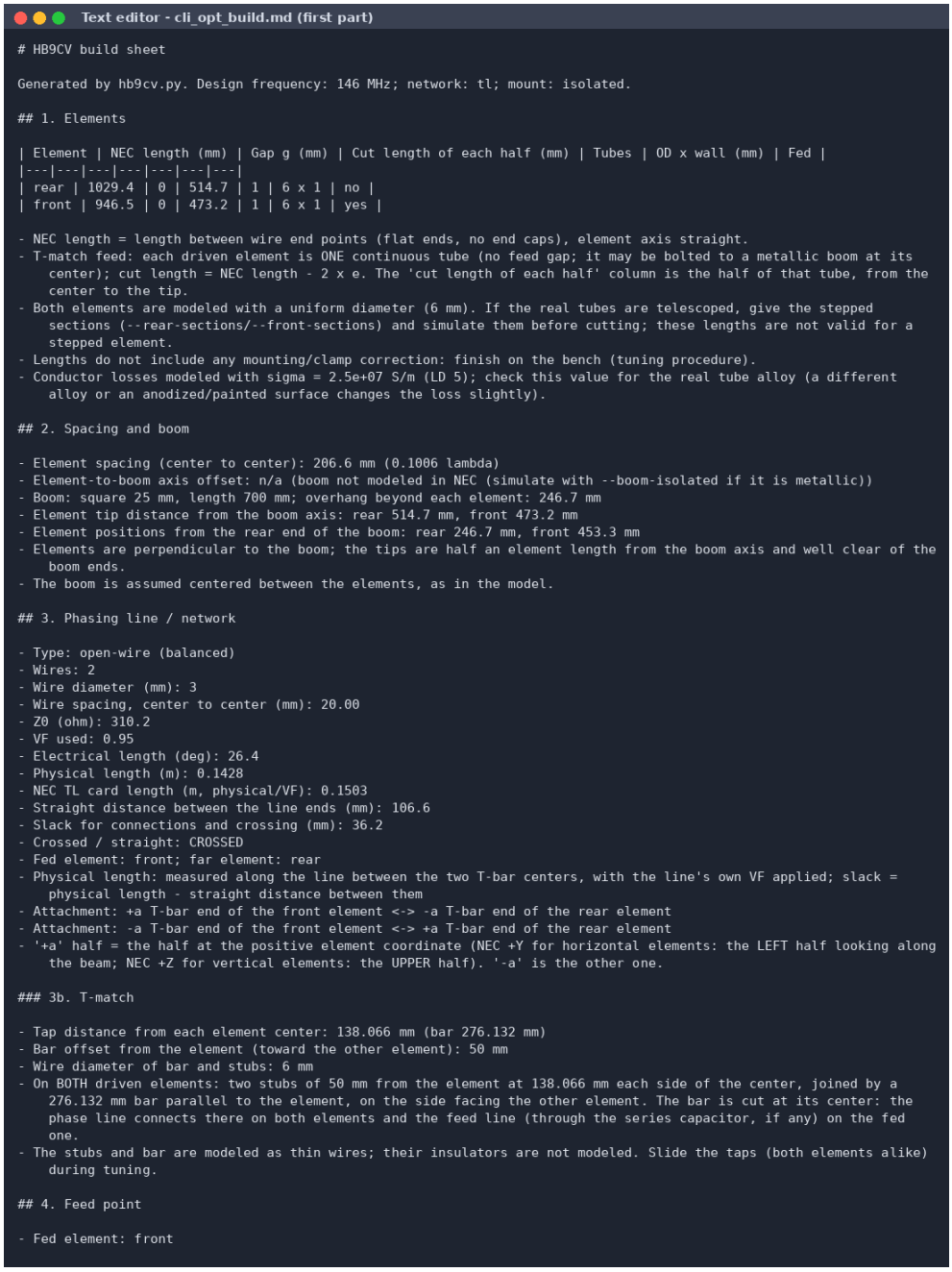



*Figure 13-1. The first part of a build sheet produced by an optimisation.*

## 13.1 Tuning the real antenna

1. Build the antenna slightly **long** (a few mm per element tip): it is easy to shorten, impossible to lengthen.
2. Install the balun / common-mode choke at the feed point as recommended.
3. Mount the antenna in the clear, at least a couple of metres from metal objects, pointing up or away.
4. With an antenna analyzer, measure the VSWR across the band. Move the T-match taps (both elements equally) to set the impedance; trim the elements to centre the frequency of best VSWR.
5. Check the F/B with a signal source or a friend's station: turn the antenna and compare front and back.

# 14. Quality checks

The Ground tab has tools that check how much you can trust a model and how sensitive it is to real-world details. They all need nec2c and add simulation time.

| Check | What it does | How to read it |
|---|---|---|
| Segmentation convergence | Runs the model again with about twice the segments per element. | If gain changes by more than 0.1 dB, F/B by more than 1 dB or R by more than 5 %, a WARNING says it has not converged: increase *Segments / element*. |
| Robustness table | Simulates the model with a metal boom, a mast with side clamp, and a coax shield with no choke, one effect at a time. | Shows the change of gain, F/B and Zin for each case. Large changes mean the real antenna needs care in that aspect. |
| Mast sensitivity | Repeats the model with each mast length you list. | Use it to see if your mast disturbs the antenna. |
| Height sweep | Repeats the model at several heights over ground. | Choose a height with a good take-off angle and stable impedance. |
| Average gain test | Always computed: integrates the pattern over a whole sphere. | Should be close to the efficiency (0.95–1.0 for a lossless model). If not, the model or the pattern grid has a problem; the program reports a correction. |
| Clearance check | Always done: checks that no wires overlap or come too close. | Errors stop the run; warnings mean reduced accuracy. |

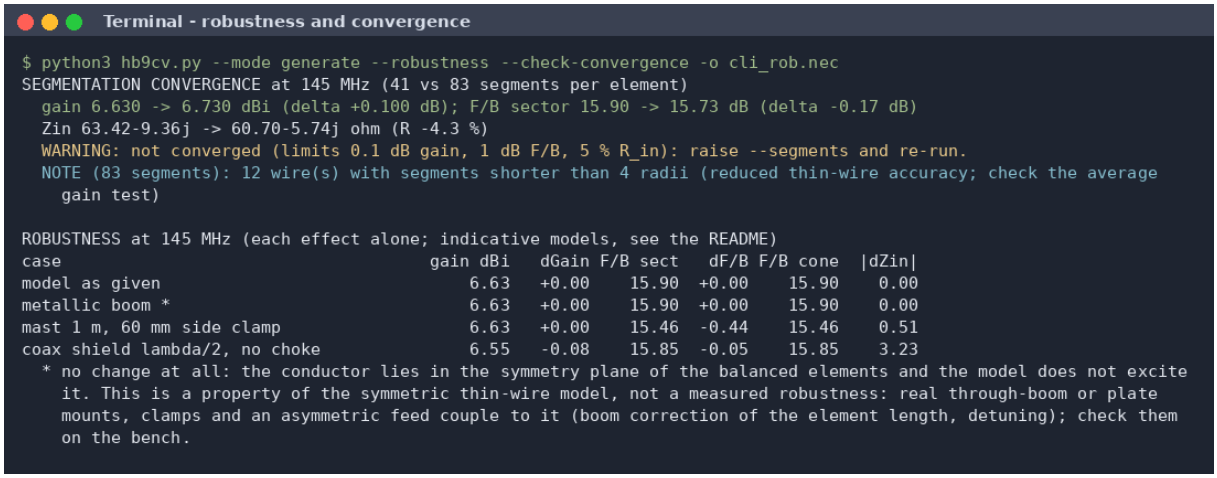



*Figure 14-1. Convergence and robustness output of a model.*

In this example the segmentation is **not converged** (Zin moves 4.3 %), so more segments should be used for the final design. The robustness table shows that a 1 m mast changes F/B by 0.44 dB and that a coax with no choke changes Zin by 3.2 Ω — another reason to use a balun.

# 15. Using the command line

Everything done in the window can also be done by typing commands in a terminal. The window's **Copy cmd** button gives you the exact command of the current settings, which is the easiest way to learn. The command line is handy for scripts and repeated runs.

## 15.1 Getting help

```
python3 hb9cv.py --help
```

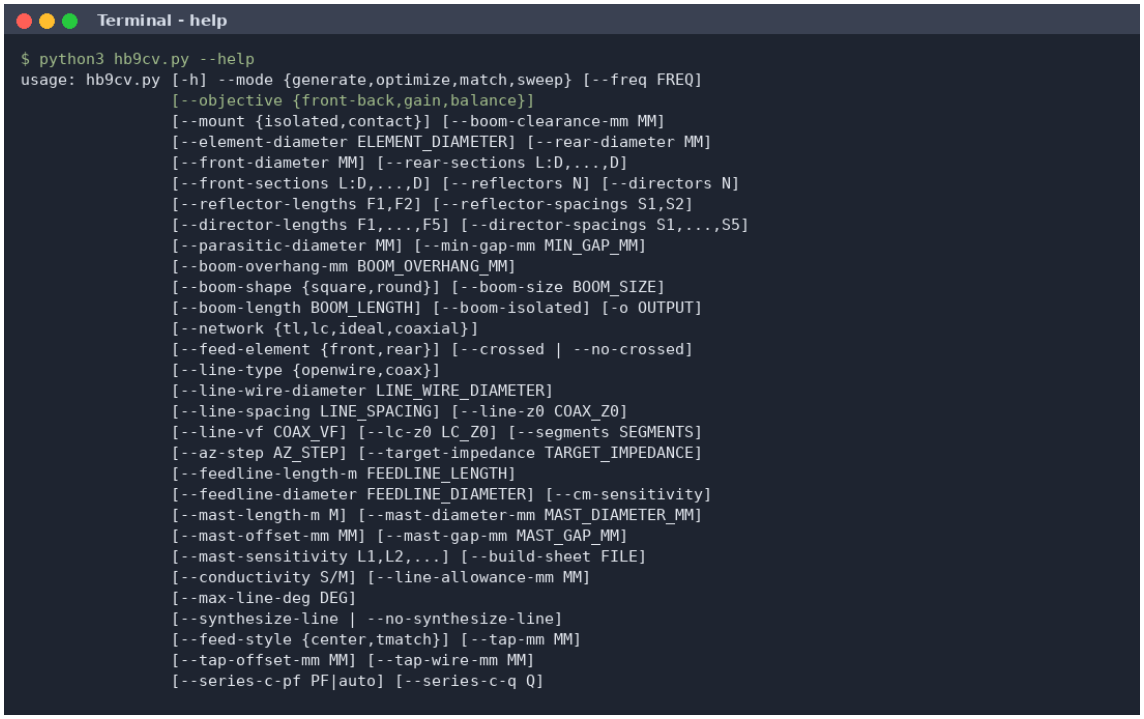



*Figure 15-1. The beginning of the built-in help, listing all the options.*

## 15.2 Generate and simulate

```
python3 hb9cv.py --mode generate --run -o cli_first.nec
```

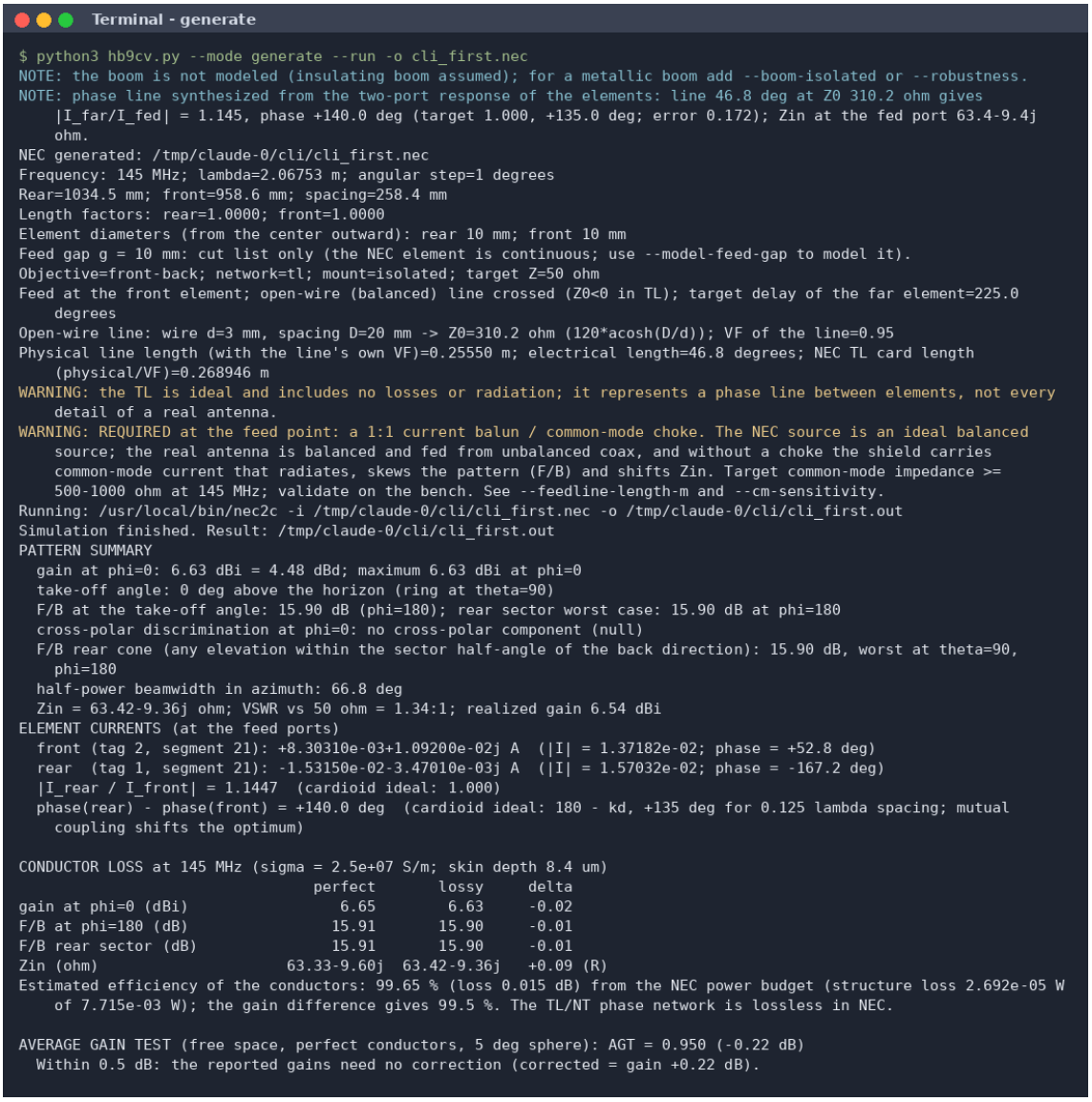



*Figure 15-2. Output of a generate run (the same information as the Log tab).*

## 15.3 Frequency sweep

```
python3 hb9cv.py --mode generate --fsweep 140,150,0.5 --run -o cli_sweep.nec
```

Terminal - generate with frequency sweep

```
$ python3 hb9cv.py --mode generate --fsweep 140,150,0.5 --run -o cli_sweep.nec
NOTE: the boom is not modeled (insulating boom assumed); for a metallic boom add --boom-isolated or --robustness.
NOTE: phase line synthesized from the two-port response of the elements: line 46.8 deg at Z0 310.2 ohm gives
    |I_far/I_fed| = 1.145, phase +140.0 deg (target 1.000, +135.0 deg; error 0.172); Zin at the fed port 63.4-9.4j
    ohm.
NEC generated: /tmp/claude-0/cli/cli_sweep.nec
Frequency: 145 MHz; lambda=2.06753 m; angular step=1 degrees
Rear=1034.5 mm; front=958.6 mm; spacing=258.4 mm
Length factors: rear=1.0000; front=1.0000
Element diameters (from the center outward): rear 10 mm; front 10 mm
Feed gap g = 10 mm: cut list only (the NEC element is continuous; use --model-feed-gap to model it).
Objective=front-back; network=tl; mount=isolated; target Z=50 ohm
Feed at the front element; open-wire (balanced) line crossed (Z0<0 in TL); target delay of the far element=225.0
    degrees
Open-wire line: wire d=3 mm, spacing D=20 mm -> Z0=310.2 ohm (120*acosh(D/d)); VF of the line=0.95
Physical line length (with the line's own VF)=0.25550 m; electrical length=46.8 degrees; NEC TL card length
    (physical/VF)=0.268946 m
WARNING: the TL is ideal and includes no losses or radiation; it represents a phase line between elements, not every
    detail of a real antenna.
WARNING: REQUIRED at the feed point: a 1:1 current balun / common-mode choke. The NEC source is an ideal balanced
    source; the real antenna is balanced and fed from unbalanced coax, and without a choke the shield carries
    common-mode current that radiates, skews the pattern (F/B) and shifts Zin. Target common-mode impedance >=
    500-1000 ohm at 145 MHz; validate on the bench. See --feedline-length-m and --cm-sensitivity.
Running: /usr/local/bin/nec2c -i /tmp/claude-0/cli/cli_sweep.nec -o /tmp/claude-0/cli/cli_sweep.out
Simulation finished. Result: /tmp/claude-0/cli/cli_sweep.out
VSWR <= 2:1 bandwidth: 143.500-149.000 MHz (5.500 MHz)
```

*Figure 15-3. A frequency sweep.*

## 15.4 Match and sweep

```
python3 hb9cv.py --mode match --nec-out cli_sweep.out --freq 145 --q-l 150 --q-c 500
python3 hb9cv.py --mode sweep --nec-out cli_sweep.out --design-freq 145 --tol-l 5 --tol-c 5
```

Terminal - match

```
$ python3 hb9cv.py --mode match --nec-out cli_sweep.out --freq 145 --q-l 150 --q-c 500
Frequency: 145.000000 MHz
Impedance NEC (cli_sweep.out): 63.4250 -9.3615j ohm
Before: |Gamma|=0.143806; VSWR=1.336:1; return loss=16.84 dB; reflected power=2.1%

Ideal L-network computed (mathematical values, not final components):
  Topology B: shunt on the antenna side, series on the Z0 side
  Series branch: inductor 29.865 nH
  Shunt branch: capacitor 6.717 pF
  Solution: low-pass (--lnet-solution plus)
  Other solution (high-pass): series capacitor 40.340 pF, shunt inductor 102.826 nH
After, ideal: Zin=50.0000 +0.0000j ohm; VSWR=1.0000:1
With ESR (Q_L=150.0, Q_C=500.0) without readjusting values: Zin=50.1599 +0.0333j ohm; VSWR=1.0033:1; mismatch=0.000
    dB; dissipation=0.019 dB (0.4% of the power)

IMPORTANT: single-frequency lumped calculation. It does not include parasitics, physical circuit
coupling, nor does it recompute the NEC pattern. To see the bandwidth use --mode sweep with a NEC
sweep (--mode generate --fsweep) and validate with an antenna analyzer/VNA.

BANDWIDTH PREVIEW (ideal L-network values, Zin from the NEC sweep):
    f (MHz)     VSWR            Zload (ohm)              Zin (ohm)
    140.000    3.069     110.750-94.751j      38.707-50.711j
    140.500    2.741     103.220-83.294j      39.604-45.617j
    141.000    2.447      96.595-72.698j      40.539-40.553j
    141.500    2.184      90.737-62.863j      41.521-35.510j
    142.000    1.950      85.523-53.705j      42.550-30.481j
    142.500    1.741      80.858-45.147j      43.631-25.453j
    143.000    1.556      76.661-37.125j      44.769-20.417j
    143.500    1.391      72.869-29.582j      45.970-15.364j
    144.000    1.244      69.427-22.469j      47.237-10.284j
    144.500    1.115      66.292-15.741j      48.578-5.166j
    145.000    1.000      63.425-9.361j      50.000+0.000j
    145.500    1.113      60.795-3.296j      51.510+5.224j
    146.000    1.237      58.375+2.485j      53.117+10.518j
    146.500    1.372      56.143+8.008j      54.832+15.891j
    147.000    1.518      54.077+13.297j      56.662+21.358j
    147.500    1.676      52.162+18.371j      58.623+26.928j
    148.000    1.847      50.383+23.250j      60.726+32.615j
    148.500    2.030      48.726+27.951j      62.987+38.434j
    149.000    2.226      47.181+32.488j      65.423+44.395j
    149.500    2.435      45.738+36.875j      68.053+50.516j
    150.000    2.657      44.387+41.125j      70.900+56.813j
```

*Figure 15-4. Match mode in the terminal.*

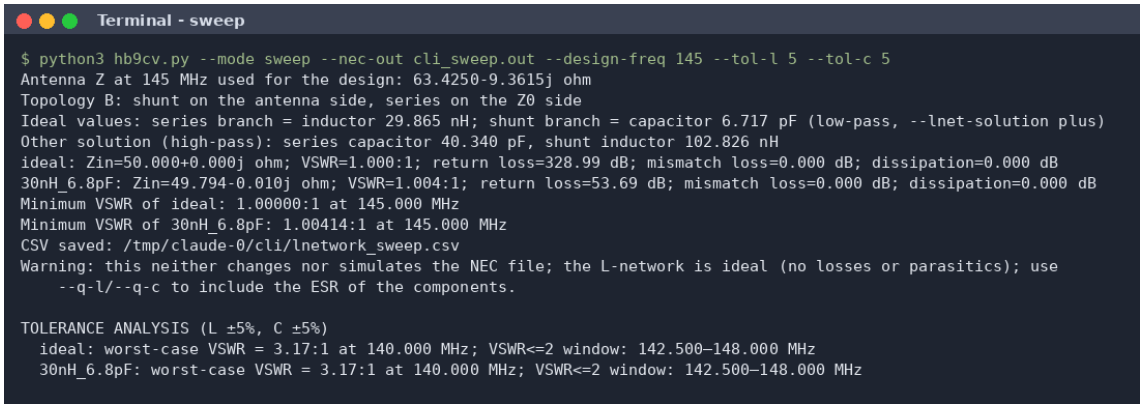



*Figure 15-5. Sweep mode in the terminal.*

## 15.5 Optimize

```
python3 hb9cv.py --mode optimize --freq 146 --band 145,147,3 --spacing-values
0.1,0.125,0.15 \
    --line-offset-values=-10,0,10 --tolerance-mc 100 --tol-top-k 3 --build-sheet
cli_opt_build.md -o cli_opt.nec
```

Note the `=` in `--line-offset-values=-10,0,10`: when a value starts with a minus sign it must be joined to the option with '=' so it is not mistaken for another option.

## 15.6 Robustness and convergence

```
python3 hb9cv.py --mode generate --robustness --check-convergence -o cli_rob.nec
```

# 16. Troubleshooting and frequent questions

## 16.1 Problems

| Problem | Solution |
|---|---|
| The window does not open: *No module named tkinter* or *customtkinter*. | Run the installer again (chapter 3). Manual installation: install Tk (Fedora: sudo dnf install python3-tkinter; Debian/Ubuntu/Mint: sudo apt install python3-tk) and pip install -r requirements.txt in the virtual environment. |
| Linux: *./install.sh: Permission denied*. | Files extracted from a ZIP lose the execute permission: run bash install.sh. |
| Linux: *hb9cv-studio: command not found*. | Log out and in once, or run ~/.local/bin/hb9cv-studio. |
| Windows: *Windows protected your PC* when opening install.bat. | Click More info → Run anyway. |
| Windows: the Microsoft Store opens when typing python. | That is only a shortcut of Windows: install Python from python.org (or let install.bat do it) and tick Add python.exe to PATH. |
| Windows: the installer could not compile nec2c. | Check the internet connection and run install.bat again (the first MSYS2 update sometimes has to be repeated). Or use an existing nec2c.exe: install.ps1 -Nec2c C:\path\nec2c.exe, or write its path in the nec2c box. |
| *nec2c was not found*. | Run the installer again, or install nec2c and write its path in the nec2c box (or put it in the PATH). |
| *Work folder does not exist*. | Choose an existing folder with the ... button. |
| *Please fix: ... is not a valid number*. | Use a dot for decimals (2.5, not 2,5). Lists use commas between values: 0.1,0.125,0.15. |
| *Finished with errors (exit code ...)*. | Open the Log tab: the last lines explain the reason, usually an option combination that is not allowed (the message names the options). |
| The plots are empty. | Tick *Compute patterns for the plots* and, in generate mode, *Run nec2c after generating*. |
| Runs are slow. | Use a coarser pattern step (5° or 10°), fewer segments, shorter optimisation lists, and more Parallel jobs. |
| Text or buttons too small/large. | Change Scale at the top, or use Ctrl + / Ctrl − / Ctrl 0. |
| Optimizer: *no candidate satisfies the Zin window*. | The closest candidate is still written. Widen Admissible R_in or Max \|X_in\|, enlarge the search lists, or use a matching network. |
| A run is reported as *not physical*. | NEC produced an impossible power budget for that geometry. Keep the extended kernel off, check that no wires are too close, and increase the segments. |

## 16.2 Common NOTE and WARNING lines

| Message (short) | What it means |
|---|---|
| NOTE: the boom is not modeled | The boom is assumed insulating. For a metal boom use the metal-boom options (Mount = contact, or *Model the boom* in the Antenna tab) or the robustness table. |
| NOTE: phase line synthesized | The program computed the best phasing line length from the actual behaviour of the two elements. Informative. |
| WARNING: the TL is ideal | The NEC transmission line has no losses or radiation; a real line is close but not identical. Informative. |
| WARNING: REQUIRED at the feed point: a 1:1 current balun | Use a balun or common-mode choke in the real antenna. Shown every time on purpose. |
| WARNING: … open-wire line D/d … | The wires of the phasing line are very close; spacers and proximity shift its impedance. Measure the built line. |
| WARNING: not converged | Increase Segments / element and run again. |
| Notes: sphere-averaged gain … differs | The average gain test is a little off; the program reports how much to correct the gain figures. Usually small. |
| NOTE: the raw VSWR of the best model is … | The optimised antenna is meant to be used with the matching network the optimizer reported. |

## 16.3 Frequent questions

### Should I use free space or real ground?

Free space to design and compare antennas; real ground to see the elevation pattern and take-off angle at your actual height.

### Which objective should I choose?

**front-back** for direction finding or to reject interference from behind; **gain** for maximum range; **balance** for a general-purpose compromise.

### Why are the NEC lengths different from the cut lengths?

NEC lengths are tip to tip of a continuous element. The build sheet converts them into the lengths you actually cut, taking the feed gap and tube sections into account.

### Can I keep my settings?

Yes: **Save preset** writes a file with all your settings; **Load preset** restores it.

# Appendix A — Complete option reference

Every box of the window, grouped by tab, in the order shown in the program. **Flag** is the matching command-line option. **Modes**: G = generate, O = optimize, M = match, S = sweep. The explanation is the same text as the in-program help.

## Main tab

| Option | Explanation |
|---|---|
| **Frequency (MHz)**<br>`--freq`<br>Modes: G O M | Design frequency in MHz. In generate/optimize it sets the wavelength used to size the elements (rear about 150/f metres, front about 139/f metres) and the element spacing, and it is the frequency at which nec2c computes the pattern. In optimize, if you give a band, the geometry is still fixed at this frequency (default: band centre). In match mode with a NEC output file it must be one of the frequencies stored in that file; leave it empty to use the only one it contains.<br>Default shown: 145 |
| **Feed network**<br>`--network`<br>Modes: G O | How the two elements are fed. tl: a single EX source on one element and an ideal two-wire transmission line (TL card) carrying the signal to the other element; it is the classic HB9CV phasing line. lc: a single source and an ideal lumped pi network of L and C between the feed points. ideal: two sources with chosen amplitude and phase, useful as a theoretical reference; by default ('Ideal drive' current) they prescribe the element currents. The TL and LC networks are lossless and ideal: they model the phase shift, not every detail of a real line.<br>Choices: tl, lc, ideal |
| **Fed element**<br>`--feed-element`<br>Modes: G O | Chooses which element receives the single EX source when the network is tl or lc. In the classic HB9CV the front (director side) element is fed and a crossed line of about 45 electrical degrees carries the signal to the rear one, giving the 225 degree delay needed for a cardioid pattern. Feeding the rear element instead changes the required delay (180 minus the spacing in degrees) and is mainly an alternative to evaluate.<br>Choices: front, rear |
| **Objective**<br>`--objective`<br>Modes: G O | Scoring criterion used by the optimizer. front-back maximizes the front-to-back ratio, gain maximizes forward gain, balance combines both using the weights 'Weight: gain' and 'Weight: F/B'. In generate mode it is informative only: it is written to the comments of the NEC file but does not change the geometry or the excitation.<br>Choices: front-back, gain, balance |
| **Mount**<br>`--mount`<br>Modes: G O | How the elements are fixed to the boom. isolated (default): the elements are insulated from the boom; the boom is a separate conductor (modelled only with 'Model the boom'). contact: the element centres (driven, reflectors and directors) are bonded to a metallic boom by their clamps; the boom is modelled with a short bond wire per element. It needs the T-match feed, where the element centres are at a voltage null, which is the classic HB9CV built on a metal boom. With a centre feed the bond would short the feed points, so contact falls back to isolated plus a modelled boom.<br>Choices: isolated, contact |
| **Balun ratio**<br>`--balun-ratio`<br>Modes: G O M S | Ratio of the balun at the feed point. 1:1 (default) is a current balun or common-mode choke between the unbalanced coax and the balanced antenna; one is mandatory in a real build because the NEC source is an ideal balanced source. 4:1 changes the balanced-side reference impedance to 50/4 = 12.5 ohm (default target 12.5 ohm and maximum \|X_in\| of 10 ohm in optimize; the L-network in match/sweep is designed against 12.5 ohm). It is an option to evaluate, not a proven fix.<br>Choices: 1:1, 4:1 |

| Option | Explanation |
|---|---|
| **Target Z (ohm)**<br>`--target-impedance`<br>Modes: G O | Impedance, in ohm, that the balanced side of the antenna should present (the reference for VSWR). The default is 50 ohm, or 12.5 ohm when the balun ratio is 4:1. In generate mode it is informative only (written to the comments); in optimize it is the reference used to compute VSWR_vs_target and the VSWR penalty of each candidate.<br>Default shown: 50 |
| **Segments / element**<br>`--segments`<br>Modes: G O | Number of wire segments used for each element in the NEC model. It must be odd (so a segment sits exactly at the centre, where the source goes) and at least 5. More segments give a finer current distribution and more accurate results but increase the nec2c run time, especially in optimize. The program warns if the segments break the thin-wire rules (segment length at least 2 radii and at most lambda/10). Default 21, or 41 with the T-match feed, whose junctions need finer segments; the average gain test reports the accuracy.<br>Default shown: 21 / 41 |
| **Azimuth step (deg)**<br>`--az-step`<br>Modes: G O | Step in degrees of the azimuth radiation pattern written in the NEC file (RP card). It must divide 360 exactly (1, 2, 5, 10...) and be at least 0.01. A smaller step gives a smoother pattern and a finer front/back and beamwidth reading but increases run time. The GUI plots use their own full-sphere step (the 'step' selector on the left). |
| **NEC extended kernel (EK)**<br>`--extended-kernel`<br>Modes: G O | Adds NEC-2's extended thin-wire kernel (EK card). It is off by default: it only helps on thick, straight wires, NEC-2 turns it off at junctions and bends, and with the T-match junctions and short segments it can make the current solution unstable (negative radiated power, an efficiency above 100 %). Where it is stable the results differ from the thin-wire kernel by about 1 ohm and 0.01 dB. Every run is checked for a physical power budget and discarded if it fails. |
| **Conductivity (S/m)**<br>`--conductivity`<br>Modes: G O | Electrical conductivity of all wires, in siemens per metre, written as an LD 5 card so nec2c includes conductor losses. Default 2.5e7 (6061 aluminium); 6063 aluminium is about 3.0e7, brass 1.5e7, copper 5.8e7. 0 means perfect conductors (no LD card). With 'Run nec2c' and at the end of optimize the model is simulated again with perfect conductors and the change in gain, F/B and R_in is reported as an efficiency estimate. |
| **Output .nec file**<br>`--output`<br>Modes: G O | Name of the NEC input file to write. If left empty, generate derives a name from the parameters (for example hb9cv_145MHz_tl_isolated.nec) and optimize writes hb9cv_optimized.nec. Relative names are created inside the work folder. The GUI reads this file back to draw the antenna and compute the radiation patterns.<br>Default shown: (automatic) |
| **Build sheet (.md)**<br>`--build-sheet`<br>Modes: G O | Writes a build sheet: a markdown file with the name you give and the same data as JSON next to it. It contains the cut list of the elements (NEC and cut lengths), spacing and boom, the phasing line, feed point and balun, the predicted performance, a tuning procedure and a bill of materials. In generate the performance section needs 'Run nec2c'; in optimize the network must be tl or lc.<br>Default shown: (none) |
| **Run nec2c after generating**<br>`--run`<br>Modes: G | After writing the NEC file, run nec2c on it and print a summary: gain in dBi and dBd, take-off angle, half-power beamwidth, front-to-back ratio, element currents and the conductor-loss check (the model is simulated again with perfect conductors). Without it only the model file is generated. nec2c must be installed and reachable (PATH or the 'nec2c' field). |

## Antenna tab

| Option | Explanation |
|---|---|
| **Spacing (wavelengths)**<br>`--spacing-factor`<br>Modes: G | Distance between the rear and the front element, in wavelengths of the design frequency. The classic HB9CV uses 0.125 lambda (45 electrical degrees, about 258 mm at 145 MHz). The theoretical phase lead of the rear current is 180 minus the spacing in degrees (135 degrees for 0.125 lambda). Larger spacing usually adds gain bandwidth but lengthens the boom; the optimizer scans several values. |
| **Rear length factor**<br>`--rear-length-factor`<br>Modes: G | Scales the rear element length relative to the nominal 150/f metres (f in MHz), i.e. 1.0 gives about 1034 mm at 145 MHz. Allowed range 0.5 to 1.5. The rear element is normally a little longer than the front one. Mutual coupling and the element diameter shift the resonance, so the optimizer scans factors around 1.0. |
| **Front length factor**<br>`--front-length-factor`<br>Modes: G | Scales the front element length relative to the nominal 139/f metres (f in MHz), i.e. 1.0 gives about 959 mm at 145 MHz. Allowed range 0.5 to 1.5. Shortening the front element moves its resonance up, which changes the input impedance and the F/B; the optimizer scans factors around 1.0. |
| **Element diameter (mm)**<br>`--element-diameter`<br>Modes: G O | Diameter in mm of both elements (the default for each). It sets the wire radius in the model and therefore the electrical length and bandwidth: thicker tubes are shorter electrically and broader. The per-element diameters and the section lists override it. In optimize, 'Refine diameter' can move it within the diameter range. |
| **Rear diameter (mm)**<br>`--rear-diameter`<br>Modes: G O | Diameter in mm of the rear element only. If empty, the general element diameter is used. Cannot be combined with the rear sections list, which defines the diameters section by section. |
| **Front diameter (mm)**<br>`--front-diameter`<br>Modes: G O | Diameter in mm of the front element only. If empty, the general element diameter is used. Cannot be combined with the front sections list, which defines the diameters section by section. |
| **Rear sections**<br>`--rear-sections`<br>Modes: G O | Models a stepped (telescoped) rear element. Give ONE half only, from the centre outward, as LENGTH_MM:DIAMETER_MM pairs separated by commas; the last item has only a diameter and runs to the tip. Example 300:12,200:10,8 means 300 mm of 12 mm tube, then 200 mm of 10 mm, then 8 mm to the tip. Both halves are identical. Lengths are measured from the element centre (half the feed gap included). Needs the isolated mount and excludes the rear diameter.<br>Default shown: 300:12,200:10,8 |
| **Front sections**<br>`--front-sections`<br>Modes: G O | Same as the rear sections but for the front element: ONE half, from the centre outward, LENGTH_MM:DIAMETER_MM pairs separated by commas and a final diameter that runs to the tip (for example 280:12,180:10,8). Every model is checked against the thin-wire rules (neighbouring radius ratio up to 2:1) and warnings are printed. |
| **Reflectors (0-2)**<br>`--reflectors`<br>Modes: G O | Passive reflectors behind the rear element. Length and spacing are pre-calculated (minimum / nominal / maximum); optimize keeps them inside those limits.<br>Choices: 0, 1, 2 |
| **Directors (0-5)**<br>`--directors`<br>Modes: G O | Passive directors in front of the front element. Length and spacing are pre-calculated (minimum / nominal / maximum); optimize keeps them inside those limits.<br>Choices: 0, 1, 2, 3, 4, 5 |
| **Reflector lengths (x resonant)**<br>`--reflector-lengths`<br>Modes: G O | Factors of the resonant dipole length, F1,F2. Empty = pre-calculated nominal (1.05). In optimize: the starting point.<br>Default shown: 1.05,1.05 |
| **Reflector spacings (wavelengths)**<br>`--reflector-spacings`<br>Modes: G O | Each from the previous element (the first from the rear element). Empty = 0.15, 0.20.<br>Default shown: 0.15,0.20 |

| Option | Explanation |
|---|---|
| **Director lengths (x resonant)**<br>`--director-lengths`<br>Modes: G O | Factors of the resonant dipole length, F1..F5. Empty = pre-calculated nominal.<br>Default shown: 0.96,0.95,0.94,0.93,0.92 |
| **Director spacings (wavelengths)**<br>`--director-spacings`<br>Modes: G O | Each from the previous element (the first from the front element). Empty = 0.15, 0.20, 0.25, 0.30, 0.30.<br>Default shown: 0.15,0.20,0.25,0.30,0.30 |
| **Parasitic tube dia (mm)**<br>`--parasitic-diameter`<br>Modes: G O | Tube diameter of reflectors and directors. Default: the element diameter.<br>Default shown: (element diameter) |
| **Min gap between elements (mm)**<br>`--min-gap-mm`<br>Modes: G O | Smallest surface-to-surface gap between neighbouring elements (clamps, insulators). With 0.05 lambda it sets the minimum spacing of reflectors and directors. |
| **Boom overhang (mm)**<br>`--boom-overhang-mm`<br>Modes: G O | Boom beyond the outer elements at each end. With reflectors/directors the boom is lengthened automatically. |
| **Boom shape**<br>`--boom-shape`<br>Modes: G O | Cross-section of the boom: square or round. It only matters when the boom is modelled (isolated boom) and in the build sheet. A square boom of side a is represented in NEC by an equivalent wire of radius 0.59 times a; a round boom uses its true radius.<br>Choices: square, round |
| **Boom size (mm)**<br>`--boom-size`<br>Modes: G O | Boom dimension in mm: the side for a square boom or the diameter for a round one. It determines the equivalent radius of the modelled boom and is listed in the build sheet. Default 25 mm. |
| **Boom length (mm)**<br>`--boom-length`<br>Modes: G O | Total length of the boom in mm. It sets how far the modelled boom conductor extends along the beam axis and is reported in the build sheet; it should be at least the element spacing plus some margin at both ends. Default 700 mm. |
| **Model the boom (isolated)**<br>`--boom-isolated`<br>Modes: G O | Includes the boom in the NEC model as a metallic conductor with no electrical connection to the elements, 'Boom clearance' below their surfaces. By default the boom is not modelled at all. Turning it on shows its coupling effect (small shift of resonance and impedance). The boom is segmented by its own radius (segments of at least 4 radii), as the thin-wire kernel requires. |
| **Boom clearance (mm)**<br>`--boom-clearance-mm`<br>Modes: G O | Gap in mm between the surfaces of the elements (and T-bars) and the surface of a modelled boom (default 10). The common-mode shield wire runs beside the boom at the same gap. NEC-2 evaluates the fields on the wire axes, so conductors much closer than their radii couple inaccurately; every model is checked and conductors closer than half the larger radius are reported.<br>Default shown: 10 |
| **Feed gap (mm)**<br>`--feed-gap-mm`<br>Modes: G O | Gap in mm between the two halves of each element at the feed point (where the insulator is). In the cut list each half is (L_nec - g)/2 minus the end hardware; the gap is inside the tip-to-tip NEC length, so changing it does not change the electrical length of the model, but with the same tubes each extra millimetre of gap lengthens the element by 1 mm. See 'Model the feed gap in NEC'. |
| **Model the feed gap in NEC**<br>`--model-feed-gap`<br>Modes: G O | Also puts the feed gap in the NEC geometry: each element becomes two half wires running from plus/minus g/2 to the tips, plus a one-segment wire of length g at the centre that carries the source or the line port (NEC needs a connected segment for a port). The gap capacitance and the insulator are not modelled. The one-segment bridge must be at least 4 tube radii and lambda/200 long (thin-wire rules), so the option suits thin tubes or wide gaps. Not together with the common-mode feed-line wire. |
| **End cap (mm)**<br>`--end-cap-mm`<br>Modes: G O | Build sheet only. Length in mm of conductor added beyond each tube end by end hardware (caps, bolts); the cut list subtracts it so the finished element has the NEC length. 0 means flat ends. It does not change the NEC model. |

| Option | Explanation |
|---|---|
| **Tube wall (mm)**<br>`--tube-wall-mm`<br>Modes: G O | Build sheet only. Wall thickness in mm of the element tube, used in the bill of materials and to describe the telescoped sections. It does not change the NEC model, which uses solid wires. |
| **Boom material text**<br>`--boom-material`<br>Modes: G O | Build sheet only. Free text describing the boom and mount material (for example '25x25x2 mm aluminium square tube'), printed in the bill of materials. If empty, a text derived from the mount type is used. |
| **Feed style**<br>`--feed-style`<br>Modes: G O | Where the phase line and the feed connect. tmatch (the default with the tl and lc networks) is the classic HB9CV: each driven element is a continuous tube with a T-match (two stubs at the tap distance from the centre joined by a bar parallel to the element, on the side facing the other element); the line and the feed connect at the bar centres. The T-match transforms the element impedances so that a short, straight phase line can give the cardioid currents. center connects at the element centres (split, insulated elements) and is used automatically with stepped sections or a modelled feed gap; with a high-impedance open-wire line it cannot reach the cardioid currents with a short line.<br>Choices: auto, tmatch, center |
| **T-match tap (mm)**<br>`--tap-mm`<br>Modes: G O | Distance from the element centre to each tap (optimize moves it). |
| **T-bar offset (mm)**<br>`--tap-offset-mm`<br>Modes: G O | Distance from the element to the T-bar, toward the other element (closer bars lower the NEC accuracy). |
| **T-match wire (mm)**<br>`--tap-wire-mm`<br>Modes: G O | Diameter of the T-bar and its stubs (default: the element diameter).<br>Default shown: (element diameter) |

## Feed tab

| Option | Explanation |
|---|---|
| **Phase line type**<br>`--line-type`<br>Modes: G O | Physical type of the phase line used with network tl. openwire (default): a balanced air line, crossed by construction, velocity factor 0.95, with Z0 computed from the wire diameter and spacing (120*acosh(D/d), exact for round wires). coax: a coaxial cable, always straight because a coax cannot be crossed, velocity factor 0.66 and Z0 from 'Line Z0'; it needs a balun or choke at its element end, which the NEC TL card does not model. auto picks openwire.<br>Choices: auto, openwire, coax |
| **Crossed line / network**<br>`--crossed`<br>Modes: G O | Whether the phase line or LC network is crossed (the two conductors swap sides), which adds the extra 180 degrees of the classic HB9CV delay (180 + 45 = 225 degrees). In NEC a crossed TL is a negative Z0 and a crossed NT has an inverted Y12. default = crossed, except for coax lines which are always straight; yes with coax is an error. no gives a straight line.<br>Choices: default, yes, no |
| **Open-wire diameter (mm)**<br>`--line-wire-diameter`<br>Modes: G O | Open-wire line only: diameter d in mm of each of the two wires of the air line. Together with the spacing it sets the line impedance Z0 = 120*acosh(D/d). Default 3 mm. Below D/d = 3 the spacers and the proximity effect change Z0 and VF noticeably and a warning is printed.<br>Default shown: 3 |
| **Open-wire spacing (mm)**<br>`--line-spacing`<br>Modes: G O | Open-wire line only: centre-to-centre spacing D in mm between the two wires. Z0 = 120*acosh(D/d), exact for round wires in air. The default of 20 mm with 3 mm wire gives about 310 ohm. Wider spacing raises Z0; the line length in the model is not affected, only the impedance. D must be at least 1.2 d.<br>Default shown: 20 |
| **Line Z0 (ohm, coax)**<br>`--line-z0`<br>Modes: G O | Characteristic impedance in ohm of the phase line when it is a coaxial cable (default 50 ohm; use 75 for 75-ohm cable). Ignored for open-wire lines, where Z0 is computed from the wire diameter and spacing.<br>Default shown: 50 |

| Option | Explanation |
|---|---|
| **Line velocity factor**<br>`--line-vf`<br>Modes: G O | Velocity factor of the phase line, greater than 0 and at most 1. Default 0.95 for an open-wire air line and 0.66 for solid-polyethylene coax. The physical length of the line is the electrical length divided by this factor, so a lower factor gives a shorter cable for the same phase shift.<br>Default shown: 0.95 / 0.66 |
| **LC network Z0 (ohm)**<br>`--lc-z0`<br>Modes: G O | Characteristic impedance in ohm of the ideal pi network (network lc). It sets the L and C values that produce the required phase shift: L = Z0*sin(theta)/w and C = tan(theta/2)/(Z0*w). Default 50 ohm. |
| **Line electrical length (deg)**<br>`--line-deg`<br>Modes: G | Electrical length of the phase line or LC network in degrees (0 to 360). By default it is synthesized from the response of the elements (or, without synthesis, derived from theory) so that the far element current has the right amplitude and phase, and it is never shorter than the line can physically be. Typing a value forces it, which is useful to test sensitivity. With a transmission line, 'Line physical length' takes priority if both are given. |
| **Phase target (deg)**<br>`--phase-target-deg`<br>Modes: G | Desired delay in degrees of the current in the far element relative to the fed one. By default it follows cardioid theory: spacing in degrees plus 180 when the front element is fed (225 for 0.125 lambda), or 180 minus the spacing when the rear is fed. Mutual coupling shifts the real value, so the optimizer scans around it. |
| **Line physical length (m)**<br>`--line-length`<br>Modes: G | Physical length in metres of the phase line. If empty it is derived from the phase target: (degrees/360) * wavelength * velocity factor. Typing a length overrides the derived one, handy to model a line cut to a specific size. The NEC TL card receives the physical length divided by the velocity factor. |
| **LC series L (nH)**<br>`--lc-l-nh`<br>Modes: G | Series inductor of the pi network in nH. Must be given together with the shunt capacitor; both replace the values derived from the target delay. Use it to test real component values. The network is ideal: no losses, parasitics or component radiation. |
| **LC shunt C (pF)**<br>`--lc-c-pf`<br>Modes: G | Shunt capacitor at each port of the pi network in pF. Must be given together with the series inductor; both replace the values derived from the target delay. The network is ideal. |
| **Rear amplitude ratio**<br>`--rear-ratio`<br>Modes: G | Amplitude of the rear excitation relative to the front one when the network is ideal (two sources): of the element currents with 'Ideal drive' current (default), of the source voltages with voltage. 1.0 means equal amplitudes; the default 0.92 slightly reduces the rear drive. In optimize the 'Amplitude ratios' list scans several values. |
| **Ideal drive**<br>`--ideal-drive`<br>Modes: G O | Ideal network only. current (default): the amplitude ratio and the phase prescribe the element CURRENTS, as the cardioid theory does: each source gets a lossless series reactance of 1e6 ohm, so the currents follow the source voltages whatever the mutual coupling between the elements. voltage: they prescribe the source voltages; with mutual coupling the currents then differ strongly from the requested relation.<br>Choices: current, voltage |
| **Rear phase (deg)**<br>`--rear-phase`<br>Modes: G | Phase in degrees of the rear excitation for the ideal network (element current with 'Ideal drive' current, source voltage with voltage). Default 180 minus 360 times the spacing in wavelengths, i.e. 135 degrees for 0.125 lambda, the current relation that gives a cardioid with a null at the back for equal-amplitude currents. |
| **CM feed-line length (m)**<br>`--feedline-length-m`<br>Modes: G O | Common-mode test. Adds a wire of this length in metres representing the outer conductor of the coax, tied to the fed port and running rearward below the elements (beside a modelled boom, at 'Boom clearance' from it). It shows how current on the feed line can disturb the pattern when there is no good balun. 0 turns it off. The wire is an open-ended shield: an upper-bound sensitivity test, not a model of a real coax run or choke. |
| **CM feed-line dia (mm)**<br>`--feedline-diameter`<br>Modes: G O | Diameter in mm of the common-mode test wire that represents the coax outer conductor. RG-58 is about 5 mm. Only used when the common-mode feed-line length is greater than 0. |

| Option | Explanation |
|---|---|
| **Common-mode sensitivity**<br>`--cm-sensitivity`<br>Modes: G O | Simulates the model without and with the common-mode feed-line wire (length from 'CM feed-line length', default half a wavelength) and reports the change in front-to-back ratio and input impedance. A large change means the antenna depends on a good balun/choke. Needs nec2c. In generate it uses the model you give; in optimize, the best model found. |
| **Line allowance (mm)**<br>`--line-allowance-mm`<br>Modes: G O | Extra physical line length in mm for the connections at both ends, the crossing and the stubs (default 20 for an open-wire line, 40 for coax). A straight line cannot be shorter than the distance between its two ends; with this allowance the minimum becomes that distance plus the allowance. generate never picks a shorter line by default and warns when one is given; optimize skips such lines and reports how many it dropped.<br>Default shown: 20 / 40 |
| **Max line length (deg)**<br>`--max-line-deg`<br>Modes: G O | Longest line generate picks and optimize scans. Default: 1.5 x the straight run (or theory + largest offset).<br>Default shown: (automatic) |
| **Synthesize the line**<br>`--synthesize-line`<br>Modes: G O | Computes the phase line from the driven elements themselves: two nec2c probe runs (1 V on one feed port, no network) give how each port, each element current and the far field respond. For two elements the line length whose element currents come closest to the cardioid (equal amplitudes, rear leading by 180 minus the spacing in degrees) is chosen and the line impedance needed for an exact match is reported. With reflectors or directors in free space the two-element target does not hold: the line that maximizes the objective on the horizon ring, computed by superposing the two probe patterns, is chosen instead. generate uses it as the default line; optimize centres the line scan of every spacing, Z0 and tap on it and repeats it after the pre-tune. no uses the cardioid theory, never shorter than the buildable minimum.<br>Choices: default, yes, no |
| **Series feed C (pF)**<br>`--series-c-pf`<br>Modes: G O | Series capacitor in pF between the feed line and the fed port: the trimmer of the classic HB9CV. It is written to the NEC file as a comment and added, with its loss (ESR = \|X\|/Q, see 'Series C quality factor'), to every input impedance read from the output file (also by match and sweep, and by the pattern plots). auto chooses the value that cancels the input reactance at the design frequency (no capacitor when the reactance is capacitive); in optimize it is chosen for every candidate and then kept fixed in the Monte-Carlo.<br>Default shown: auto |
| **Series C quality factor**<br>`--series-c-q`<br>Modes: G O | Quality factor of the series feed capacitor. Its loss resistance ESR = \|X\|/Q is added to the input impedance together with its reactance. Default 500 (a good air or PTFE trimmer); 0 makes it lossless.<br>Default shown: 500 |
| **Line loss (dB/m)**<br>`--line-loss-db-per-m`<br>Modes: G O | Build sheet only, network tl: matched loss of the phasing line in dB per metre. The NEC TL card is lossless, so the build sheet reports the loss times the physical length as an upper bound (as if all the power crossed the line). 0 leaves it out of the report. |

## Ground tab

| Option | Explanation |
|---|---|
| **Ground**<br>`--ground`<br>Modes: G O | Ground model. free: free space (default); the pattern is a single azimuth ring at the horizon. perfect: perfectly conducting ground. average: finite ground with eps_r 13 and sigma 0.005 S/m. good: eps_r 20 and sigma 0.02 S/m. Over ground you must give a height; the pattern is then taken in two steps, an elevation cut to find the take-off angle and an azimuth ring at that angle, because horizontal polarization has no radiation at the horizon over ground.<br>Choices: free, perfect, average, good |
| **Height (m)**<br>`--height-m`<br>Modes: G O | Height above ground in metres. For horizontal elements it is the height of the element centres; for vertical it is the lowest tip of the longest element (driven or parasitic). Required whenever the ground is not free space. Height strongly affects the take-off angle, the gain and the input impedance because of the ground reflection. |
| **Height (wavelengths)**<br>`--height-lambda`<br>Modes: G O | Same as the height in metres but expressed in wavelengths of the design frequency (for example 1.0 = one wavelength, about 2.07 m at 145 MHz). Use one or the other. |
| **Orientation**<br>`--orientation`<br>Modes: G O | Polarization of the antenna. horizontal: the elements lie along Y and the spacing along X (beam toward +X). vertical: the elements lie along Z, with the spacing again along X. The two are identical in free space; over ground they behave very differently.<br>Choices: horizontal, vertical |
| **Ground eps_r**<br>`--ground-eps`<br>Modes: G O | Relative permittivity (eps_r) of the finite ground. Typing a value overrides the one from the ground preset (average 13, good 20). Together with the conductivity it defines the ground used by the Sommerfeld model (GN 2). |
| **Ground sigma (S/m)**<br>`--ground-sigma`<br>Modes: G O | Conductivity in S/m of the finite ground. Typing a value overrides the one from the ground preset (average 0.005, good 0.02). Wet soil is about 0.01 to 0.03; dry sand and rock are lower. |
| **Mast length (m)**<br>`--mast-length-m`<br>Modes: G O | Models a mast as a straight conductor of this length in metres. 0 turns it off. With horizontal elements it hangs below the boom centre, perpendicular to the boom and the elements; on the boom axis it lies in the symmetry plane of the balanced elements and is not excited, so use 'Mast side offset' for a side clamp. With vertical elements it is a side mount: parallel to the elements, behind the rear end of the boom, centred on the element centres. Not together with the feed-line wire. It is not galvanically connected: the 'Mast gap' sets the coupling distance. |
| **Mast diameter (mm)**<br>`--mast-diameter-mm`<br>Modes: G O | Diameter in mm of the modelled mast. Default 40 mm. A thicker mast couples more strongly to the antenna. Only used when the mast length (or the mast sensitivity list) is set. |
| **Mast gap (mm)**<br>`--mast-gap-mm`<br>Modes: G O | Air gap in mm between the lowest modelled conductor (the boom if modelled, otherwise the elements) and the top of the mast. Default 20 mm. 0 places the mast surface 10% of its radius from the conductor. A smaller gap means stronger coupling. |
| **Mast side offset (mm)**<br>`--mast-offset-mm`<br>Modes: G O | Horizontal: sideways offset of the mast from the boom axis (side clamp). Vertical: extra distance behind the boom. |
| **Mast sensitivity (m)**<br>`--mast-sensitivity`<br>Modes: G O | Comma-separated list of mast lengths in metres (for example 0.5,1,2). The model is simulated without a mast and with each length, using the mast diameter and gap above, and a table with the changes in gain, F/B and Zin is printed. Needs nec2c. In generate it uses the model you give; in optimize, the best model.<br>Default shown: 0.5,1,2 |
| **Height sweep (m)**<br>`--height-sweep`<br>Modes: G | Over ground, generate mode: simulates the model at N heights from MIN to MAX metres, written MIN,MAX,N (for example 4,12,5), and prints the take-off angle, gain, F/B and input impedance at each height. At one or two wavelengths the height changes the pattern more than any build tolerance. Needs nec2c.<br>Default shown: 4,12,5 |

| Option | Explanation |
|---|---|
| **Robustness table**<br>`--robustness`<br>Modes: G O | Prints one table with the change of gain, F/B and input impedance caused by parts the default model leaves out, each simulated alone against the model as given: a metallic boom (when the boom is not modelled), a mast placed where it breaks the symmetry of the elements (horizontal: 1 m on a 60 mm side clamp; vertical: a 2 m side-mounted mast) and an unchoked coax shield of half a wavelength. A conductor in the symmetry plane of the balanced elements is not excited by the model; such rows are marked with * and explained instead of being read as robustness. In optimize it runs on the best model. Needs nec2c. |
| **Segmentation convergence**<br>`--check-convergence`<br>Modes: G O | Simulates the model again with 2n+1 segments per element instead of n and prints the change of gain, F/B and input resistance. A warning is printed above 0.1 dB of gain, 1 dB of F/B or 5 % of resistance: the segmentation is then too coarse for the numbers to be trusted. In optimize it runs on the best model. Needs nec2c. |

## Generate tab

| Option | Explanation |
|---|---|
| **Frequency sweep**<br>`--fsweep`<br>Modes: G | Frequency sweep written as FMIN,FMAX,STEP in MHz (for example 140,150,0.5). nec2c then computes the input impedance Zin(f) at every frequency, which the match and sweep modes read to study bandwidth. The geometry and the line stay fixed at the design frequency. Without the pattern option the radiation pattern is omitted from the sweep.<br>Default shown: 140,150,0.5 |
| **Pattern vs frequency**<br>`--fsweep-pattern`<br>Modes: G | With the frequency sweep, also computes the radiation pattern at every frequency. With 'Run nec2c' it summarizes gain, front-to-back ratio and Zin versus frequency (taking the take-off angle per frequency over ground) and writes a CSV. Takes longer than an impedance-only sweep. |
| **Sweep CSV**<br>`--fsweep-csv`<br>Modes: G | Name of the CSV file that receives the pattern-versus-frequency summary. If empty it is <output file>_fsweep.csv. Only used together with the frequency sweep and the pattern-versus-frequency option.<br>Default shown: (automatic) |

## Optimize tab

| Option | Explanation |
|---|---|
| **Band FMIN,FMAX,N**<br>`--band`<br>Modes: O | Scores every candidate at N evenly spaced frequencies, written FMIN,FMAX,N (for example 144,146,5), and keeps the worst case: the minimum over frequency of the objective minus the VSWR penalty. This favours designs that hold up across the whole band instead of one frequency. The geometry and line length stay fixed at the design frequency. Empty means a single frequency. Costs N times more simulations.<br>Default shown: 144,146,5 |
| **Refine rounds**<br>`--refine`<br>Modes: O | Rounds of local search after the grid scan. Each round simulates all small moves around the best point in parallel (element lengths, phase or line length and, if enabled, spacing and diameter), keeps the best improvement and halves the step when nothing improves. 0 means grid only. Empty uses the default: 6 rounds with the tl/lc networks, 8 with reflectors or directors, 0 with ideal sources. A Nelder-Mead polish follows (see Nelder-Mead runs).<br>Default shown: 6 |
| **Nelder-Mead runs**<br>`--nm-evals`<br>Modes: O | Maximum number of nec2c runs of the Nelder-Mead polish that follows the local search (default 40; 0 turns it off). It moves the continuous parameters together (element lengths, spacing, line length or rear phase, diameter), so it can follow correlated changes such as a longer front element with a shorter line, which a one-axis-at-a-time search cannot. |

| Option | Explanation |
|---|---|
| **Pre-tune front length**<br>`--pretune`<br>Modes: O | With the tl/lc networks, before the grid the front element length of the central candidate is tuned for an input reactance near zero at the design frequency (a secant search of up to six nec2c runs), and the front-length grid is re-centred on that value. 1 % of element length moves the resonance by about 1 % of frequency, so a grid that starts detuned wastes most of its points. Default on.<br>Choices: default, yes, no |
| **F/B metric**<br>`--fb-metric`<br>Modes: O | Which front-to-back ratio the front-back and balance objectives (and the Monte-Carlo) use. point: forward gain minus the gain at exactly 180 degrees on the ring. sector: forward gain minus the worst point of the rear sector on the ring (default). cone: forward gain minus the worst direction within the sector half-angle of the back direction at any elevation, taken from a 5 degree rear half-sphere grid; it catches rear lobes above or below the ring.<br>Choices: point, sector, cone |
| **Parallel jobs**<br>`--jobs`<br>Modes: O | Number of nec2c processes run in parallel while scoring candidates. By default it equals the number of CPU cores. Lower it if the computer becomes unresponsive; raising it above the core count rarely helps. |
| **Scan CSV**<br>`--csv`<br>Modes: O | CSV file that receives every scanned candidate with its score, gain, F/B, input impedance and VSWR. The first lines start with '#' and record the arguments and the nec2c version; skip them (comment='#' in pandas) when reading. Default hb9cv_scan.csv in the work folder. |
| **Parasitic length offsets**<br>`--parasitic-length-offsets`<br>Modes: O | Grid offsets added to EVERY reflector/director length factor, e.g. -0.02,0,0.02. Clamped to the pre-calculated limits.<br>Default shown: -0.02,0,0.02 |
| **Parasitic spacing scales**<br>`--parasitic-spacing-scales`<br>Modes: O | Grid multipliers of EVERY reflector/director spacing, e.g. 0.9,1,1.1. Clamped to the pre-calculated limits.<br>Default shown: 0.9,1,1.1 |
| **Spacing values**<br>`--spacing-values`<br>Modes: O | List of element spacings to scan, in wavelengths, separated by commas (default 0.08,0.125,0.17). Every value is combined with the other lists, so the number of simulations is the product of the list sizes; keep the lists short. |
| **Rear length factors**<br>`--rear-length-values`<br>Modes: O | List of rear element length factors to scan, relative to 150/f metres (default 0.98,1.0,1.02). Combined with the other lists, so the number of simulations is the product of the list sizes. |
| **Front length factors**<br>`--front-length-values`<br>Modes: O | List of front element length factors to scan, relative to 139/f metres (default 0.98,1.0,1.02). With the pre-tune on, the list is shifted so that its middle value is the tuned front length. Combined with the other lists, so the number of simulations is the product of the list sizes. |
| **Line offsets (deg)**<br>`--line-offset-values`<br>Modes: O | Offsets in electrical degrees, added to the centre of the line scan of each spacing (the synthesized line, or the theoretical one without synthesis, moved up when the lowest offset would fall below the shortest buildable line), that the optimizer tries with the tl and lc networks (default -30 to +30 in 5 degree steps). Values outside the buildable range are dropped, never wrapped around 360. Replaced by 'Line lengths' if that is given. |
| **Line lengths (m)**<br>`--line-length-values`<br>Modes: O | Absolute physical line lengths in metres to scan. If given, it replaces the list of degree offsets. Useful when the cable length is the quantity you actually cut. |
| **Z0 values (ohm)**<br>`--z0-values`<br>Modes: O | Characteristic impedances in ohm of the TL phase line to scan, for example 50,75,300. On an open-wire line each value sets the wire spacing (the wire diameter stays fixed). Empty means the single Z0 of the configured line.<br>Default shown: 50,75,300 |
| **Amplitude ratios**<br>`--ratio-values`<br>Modes: O | Relative amplitudes of the rear source to scan (default 0.7,1.0,1.3). Only for the ideal network with two sources; the tl and lc networks fix the amplitude by their own physics. |

| Option | Explanation |
|---|---|
| **Phase offsets (deg)**<br>`--phase-offset-values`<br>Modes: O | Offsets in degrees from the theoretical rear phase (180 minus the spacing in degrees) to scan for each spacing, for the ideal network (default -60 to +60 in 15 degree steps). Replaced by 'Absolute phases' if given. |
| **Absolute phases (deg)**<br>`--phase-values`<br>Modes: O | Absolute rear source phases in degrees to scan with the ideal network. If given, it replaces the list of phase offsets. |
| **Score through the L-network**<br>`--match-score`<br>Modes: O | Scores every optimize candidate after the ideal L-network designed at the design frequency against the target impedance, built with components of the given Q: the VSWR penalty uses the matched VSWR over the band and the gain objectives use the gain after the network loss and residual mismatch. The front-back objective keeps the pure F/B and pays 'Network loss weight' dB of score per dB of network loss (an efficiency penalty). It lets the optimizer use the low or high input resistances where the best patterns are, paying for the matching network. Default on with tl/lc; no searches for a direct connection.<br>Choices: default, yes, no |
| **Match: inductor Q**<br>`--match-q-l`<br>Modes: O | Q of the scoring L-network inductor. |
| **Match: capacitor Q**<br>`--match-q-c`<br>Modes: O | Q of the scoring L-network capacitor. |
| **Admissible R_in (min,max)**<br>`--zin-r-range`<br>Modes: O | Admissible range of the input resistance in ohm, written min,max. Default 2,500 when candidates are scored through the L-network (the network loss already penalizes extreme values) and 8,80 for a direct connection. Candidates outside the range are excluded from the best choice (tl and lc only; with a band, at every band frequency).<br>Default shown: 2,500 / 8,80 |
| **Stop if no Zin match**<br>`--strict-zin`<br>Modes: O | What optimize does when no candidate meets the Zin window (admissible R_in and max \|X_in\| at every band frequency). Off (default): the closest candidate is still written (model, build sheet, JSON) and reported with a clear warning, so you can see how far the window is from reach. On: stop with an error and write no model. |
| **Max \|X_in\| (ohm)**<br>`--zin-x-max`<br>Modes: O | Maximum admissible absolute input reactance \|X_in\| in ohm (tl and lc only). Default 500 when candidates are scored through the L-network; for a direct connection 30, or 10 with a 4:1 balun.<br>Default shown: 500 / 30 |
| **VSWR weight (dB)**<br>`--vswr-weight`<br>Modes: O | dB subtracted from the score for each unit of VSWR above 1 (worst VSWR over the band, against the target impedance). Default 3 with tl/lc and 0 otherwise. 0 only reports VSWR without penalizing it. Increase it to prefer well-matched designs over slightly better pattern.<br>Default shown: 3 |
| **Realized gain**<br>`--realized-gain`<br>Modes: O | In the gain-based objectives use realized gain, which discounts the mismatch loss against the target impedance, instead of plain gain. Only meaningful with the tl and lc networks. default is on for tl/lc; no uses the plain gain.<br>Choices: default, yes, no |
| **Weight: gain**<br>`--w-gain`<br>Modes: O | Weight of the gain term in the balance objective. The balance score is w_gain times the gain plus w_fb times the robust front-to-back ratio. Default 1.0. Increase it to favour gain over rejection. |
| **Weight: F/B**<br>`--w-fb`<br>Modes: O | Weight of the robust front-to-back term in the balance objective (default 0.5). Increase it to favour rear rejection over forward gain. |
| **Rear sector (deg)**<br>`--rear-sector`<br>Modes: O | Rear sector in degrees of azimuth, written start,end (default 150,210). The robust front-to-back ratio is the forward gain minus the worst (highest) point inside this sector, so a design that only has a null exactly at 180 degrees is not rewarded. |

| Option | Explanation |
|---|---|
| **Refine spacing**<br>`--refine-spacing`<br>Modes: O | When refining, also move the element spacing (step 0.01 wavelength, halved when no move improves). default follows the program setting, yes forces it and no keeps the spacing fixed. Letting it move can improve the result but changes the boom length.<br>Choices: default, yes, no |
| **Refine diameter**<br>`--refine-diameter`<br>Modes: O | When refining, also move the element diameter: to the next or previous purchasable value of 'Tube diameters', or in 1 mm steps (halved when no move improves) when that list is 'any', always inside the diameter range. default follows the program setting, yes forces it and no keeps the diameter fixed.<br>Choices: default, yes, no |
| **Diameter range (mm)**<br>`--diameter-range`<br>Modes: O | Lower and upper limits in mm for the element diameter during refinement, written min,max (default 6,16). Use the tube sizes you can really buy. |
| **Tube diameters (mm)**<br>`--diameter-values`<br>Modes: O | Element tube diameters in mm that you can buy, comma separated (default 6,8,10,12,14,16). The starting element diameter is snapped to the nearest one and 'Refine diameter' steps through this list, so the result is always a real tube size. 'any' moves the diameter continuously within the diameter range. |
| **T-match taps (mm)**<br>`--tap-values`<br>Modes: O | Tap distances scanned by the grid (default: the T-match tap).<br>Default shown: 80,120,160 |
| **Monte-Carlo samples**<br>`--tolerance-mc`<br>Modes: O | Monte-Carlo tolerance analysis. After the scan, the same N random perturbations are applied to each of the best candidates: element lengths and spacings, line phase, and with their own tolerances the T-match taps and bar offsets of each element, the line velocity factor and Z0 and the series capacitor. The series capacitor and the matching network stay as designed for the nominal candidate unless 'MC: re-tune per sample' is on. The candidates are re-ranked by the 10th percentile score, so you pick a design that tolerates build errors; a bootstrap interval of that percentile is printed. 0 turns it off; use 100 or more. It multiplies the run time. |
| **MC: distribution**<br>`--tol-dist`<br>Modes: O | Distribution of the Monte-Carlo errors. uniform: any value within plus or minus the tolerance is equally likely (a worst-case style assumption). normal: Gaussian with the tolerance as two standard deviations, clipped at three.<br>Choices: uniform, normal |
| **MC: top-K candidates**<br>`--tol-top-k`<br>Modes: O | Monte-Carlo: how many of the best candidates of the scan are analysed with random perturbations (default 5). More candidates give a better chance to find a robust design but cost proportionally more simulations. |
| **MC: length tol (mm)**<br>`--tol-length-mm`<br>Modes: O | Monte-Carlo: element length tolerance, plus or minus this many mm with uniform distribution (default 2). Set it to the accuracy you can really achieve when cutting the tubes. |
| **MC: spacing tol (mm)**<br>`--tol-spacing-mm`<br>Modes: O | Monte-Carlo: element spacing tolerance, plus or minus this many mm with uniform distribution (default 3). It reflects how accurately the elements are positioned on the boom. |
| **MC: line tol (deg)**<br>`--tol-line-deg`<br>Modes: O | Monte-Carlo: tolerance of the phase line (or of the rear phase with the ideal network), plus or minus this many electrical degrees with uniform distribution (default 2). It reflects the accuracy of the cable or line length. |
| **MC: seed**<br>`--tol-seed`<br>Modes: O | Monte-Carlo: seed of the random number generator (default 12345). Using the same seed repeats exactly the same perturbations, so results are reproducible; change it to try a different random set. |
| **MC: tap tol (mm)**<br>`--tol-tap-mm`<br>Modes: O | Monte-Carlo: tolerance of the T-match tap position, plus or minus this many mm, drawn for each driven element on its own (default 2). Only with the T-match feed. |
| **MC: T-bar tol (mm)**<br>`--tol-bar-mm`<br>Modes: O | Monte-Carlo: tolerance of the T-bar offset from its element, plus or minus this many mm, drawn for each driven element on its own (default 1). Only with the T-match feed. |

| Option | Explanation |
|---|---|
| **MC: line VF tol (%)**<br>`--tol-vf-pct`<br>Modes: O | Monte-Carlo: tolerance of the phasing-line velocity factor in percent (default 2): spacers, insulation and moisture change it. The cut length stays, so its electrical length changes. |
| **MC: line Z0 tol (%)**<br>`--tol-z0-pct`<br>Modes: O | Monte-Carlo: tolerance of the phasing-line Z0 (or of the LC network characteristic impedance) in percent (default 3): wire spacing or cable tolerance. |
| **MC: series C tol (%)**<br>`--tol-c-pct`<br>Modes: O | Monte-Carlo: tolerance of the series feed capacitor setting in percent (default 2). Only when a series capacitor is used. |
| **MC: re-tune per sample**<br>`--tol-retune`<br>Modes: O | Monte-Carlo: re-tune the series capacitor (auto) and re-design the matching network for every sample, as if they were adjusted on the bench after the build. Off by default: they stay as designed for the nominal candidate, so their mistuning counts in the spread. |
| **Network loss weight**<br>`--loss-weight`<br>Modes: O | With 'Score through the L-network' and the front-back objective: dB of score subtracted per dB of matching-network loss (default 1). It is an efficiency penalty, the loss does not change the F/B; 0 ranks by the pure F/B. |

## Match tab

| Option | Explanation |
|---|---|
| **NEC .out file**<br>`--nec-out`<br>Modes: M S | nec2c output file (.out) from which the antenna input impedance is read. It must contain a single EX source. For the sweep mode it must contain a frequency sweep, which you obtain with generate, a frequency sweep and 'Run nec2c'. Alternatively, type R and X by hand in match mode.<br>Default shown: hb9cv_145MHz_tl_isolated.out |
| **Load R (ohm)**<br>`--r`<br>Modes: M S | Resistive part R of the load impedance in ohm, typed manually instead of reading a NEC file. It must be accompanied by the reactance X. In sweep mode it is only used with 'Assume constant Z'. |
| **Load X (ohm)**<br>`--x`<br>Modes: M S | Reactive part X of the load impedance in ohm (positive inductive, negative capacitive), typed manually. It must be accompanied by the resistance R. In sweep mode it is only used with 'Assume constant Z'. |
| **Reference Z0 (ohm)**<br>`--z0`<br>Modes: M S | Reference (unbalanced) impedance of the source or cable the L-network must match to, in ohm. Default 50. With a 4:1 balun the balanced-side reference becomes 12.5 ohm instead. |
| **Inductor Q**<br>`--q-l`<br>Modes: M S | Quality factor Q of the inductor. It adds a series loss resistance ESR = X/Q so the loss in the matching network is included. Empty means an ideal inductor without loss. Typical air coils at VHF have Q of 100 to 200. |
| **Capacitor Q**<br>`--q-c`<br>Modes: M S | Quality factor Q of the capacitor. It adds a series loss resistance ESR = X/Q to account for loss in the capacitor. Empty means an ideal capacitor. Good RF ceramic capacitors have Q of several hundred. |
| **Also print balanced version**<br>`--balanced`<br>Modes: M | Also prints the L-network as a balanced network, with the series elements split between the two legs, together with a warning about parasitics above about 100 MHz. Use it when the matching network sits between a balanced antenna feed point and the coax. |
| **L-network solution**<br>`--lnet-solution`<br>Modes: M S | Every load can be matched by two L-networks of the same topology, one per root of the design equation. plus (default) and minus pick a root; lowpass picks the solution with a series inductor and a shunt capacitor and highpass the one with a series capacitor and a shunt inductor (when no solution has those kinds, the plus or minus root is used and a note is printed); best picks the one with the lower worst VSWR over the NEC sweep (match: when the file holds a sweep; otherwise the one with the smaller spread between the two reactances). The other solution is always printed.<br>Choices: plus, minus, lowpass, highpass, best |

| Option | Explanation |
|---|---|
| **Capacitor ESL per lead (nH)**<br>`--esl-nh`<br>Modes: M S | Lead inductance in nH of each lead of a capacitor (default 2; a capacitor has two leads). Used to estimate the self-resonant frequency of every capacitor of the L-network: a warning is printed when it falls inside or below the band, a note when it is below 1.5 times the highest frequency. |
| **Coil stray C (pF)**<br>`--coil-cp-pf`<br>Modes: M S | Stray (inter-turn) capacitance in pF of an inductor (default 0.7, typical of a small air coil). Used for the self-resonance check of every inductor of the L-network, as for the capacitors. |
| **Design frequency (MHz)**<br>`--design-freq`<br>Modes: S | Frequency in MHz at which the L-network is designed in sweep mode (default 145). The component values are fixed at this frequency, and then VSWR and impedance are evaluated across the swept range. |
| **Assume constant Z**<br>`--assume-constant-z`<br>Modes: S | Instead of reading Zin(f) from a NEC sweep, assume the typed R and X are constant over the range given by Fmin, Fmax and Step. It is a quick idealization that does not represent the real antenna, whose impedance changes with frequency. |
| **Fmin (MHz)**<br>`--fmin`<br>Modes: S | Lowest frequency of the sweep in MHz (default 140). Only used with 'Assume constant Z'; otherwise the frequencies come from the NEC file. |
| **Fmax (MHz)**<br>`--fmax`<br>Modes: S | Highest frequency of the sweep in MHz (default 150). Only used with 'Assume constant Z'; otherwise the frequencies come from the NEC file. |
| **Step (MHz)**<br>`--step`<br>Modes: S | Frequency step of the sweep in MHz (default 0.25). Only used with 'Assume constant Z'. A smaller step gives a smoother curve and more rows in the CSV. |
| **Commercial values**<br>`--commercial`<br>Modes: S | Commercial component values to evaluate, written SERIES,SHUNT in nH for an inductor or pF for a capacitor, according to the ideal network. Separate several pairs with a semicolon, for example 12,3.3;10,3.9. If empty, the nearest E24 values are used. Each pair is plotted as its own curve so you can compare how real values degrade the match.<br>Default shown: 12,3.3;10,3.9 |
| **Sweep CSV**<br>`--sweep-csv`<br>Modes: S | CSV file that receives the L-network sweep: frequency, load impedance, input impedance with the network, VSWR and the network name for every point. Default lnetwork_sweep.csv in the work folder. The GUI reads this file to draw the sweep plot. |
| **Inductor tolerance (%)**<br>`--tol-l`<br>Modes: S | Inductor tolerance as a percentage (for example 5 for plus or minus 5 %; 0 turns it off). The worst-case VSWR over the four corners of the tolerance box is reported at every frequency of the sweep, for every network (ideal and commercial) around its own nominal values. |
| **Capacitor tolerance (%)**<br>`--tol-c`<br>Modes: S | Capacitor tolerance as a percentage (for example 5 for plus or minus 5 %; 0 turns it off). Reports the worst-case VSWR over the tolerance range together with the inductor tolerance. |

# Appendix B — Files created by the program

All files are written in the work folder. Use the Open folder button to see them.

| File | Created by | Content |
|---|---|---|
| ~/.local/share/hb9cv-studio (Linux), %LOCALAPPDATA%\Programs\HB9CV-Studio (Windows) | the installer | The installed program, its Python environment (venv) and, when compiled, bin/nec2c. |
| name.nec | generate, optimize | The NEC model (geometry and simulation cards). Can be opened with Open .nec… or by other NEC programs. |
| name.out | generate (Run), optimize | The nec2c result: currents, impedance, patterns, power budget. |
| name.freqs.json | generate, optimize | Machine-readable summary of the results used by the window. |
| name_build.md | generate, optimize (Build sheet) | The construction document (chapter 13). |
| name_build.json | generate, optimize (Build sheet) | The same data in JSON format. |
| hb9cv_scan.csv | optimize (Scan CSV) | Every candidate tried, with its dimensions and score. Opens in a spreadsheet. |
| hb9cv_optimized.nec / .out | optimize | The best model when no output name is given. |
| sweep CSV | generate (Sweep CSV) | Impedance and VSWR per frequency of a frequency sweep. |
| lnetwork_sweep.csv | sweep | VSWR and impedances of the matching network per frequency. |
| preset .json | Save preset | All the settings of the window. |
| ~/.hb9cv_gui.json | the window | Remembered language and scale. |

End of the manual. Good building and 73!